# V9968

# MSX-VIDEO

## TECHNICAL DATA BOOK

V9968 TECHNICAL DATA BOOK

2026.10

# Contents


| Item | Page |
|---|---|
| Chapter 1 Introduction | 3 |
| Chapter 2 I/O Ports and Access Methods | 6 |
| Chapter 3 Control Registers | 12 |
| Chapter 4 Status Registers | 34 |
| Chapter 5 Screen Modes | 39 |
| Chapter 6 Sprites | 57 |
| Chapter 7 Color Palette and Color Representation | 73 |
| Chapter 8 VDP Commands | 81 |
| Chapter 9 Geo3D | 113 |
| Chapter 10 Interrupts | 127 |
| Chapter 11 VRAM Access and Timing | 135 |
| Chapter 12 Compatibility and Migration Notes | 139 |
| Appendix A Control Register Summary | 143 |
| Appendix B Status Register Summary | 164 |
| Appendix C Screen Mode Summary | 168 |
| Appendix D VDP Command Code Summary | 185 |
| Appendix E I/O Port Summary | 186 |
| Index | 188 |

# Chapter 1 Introduction

## 1.1 About This Manual

This manual is the programming manual for the "V9968", a Video Display Processor (VDP) developed for the MSX2++. It describes all of the functions of the V9968 for developers of MSX software that performs screen display using the V9968.

Chapter 2 first describes the basic procedures for accessing the V9968 from the CPU, and the chapters that follow describe each function in detail. To quickly check only the bit-level specifications of the registers, see Chapter 3, Chapter 4, and Appendix A.

## 1.2 Notation

This manual uses the following notation.

- Control registers are written as **R#n** (for example, R#7), and status registers are written as **S#n** (for example, S#0).
- Hexadecimal numbers are written with a trailing h (for example, 98h, 3FFFh). If the number would begin with a letter, a leading 0 is added (for example, 0FFh).
- For bits, bit7 is the most significant bit and bit0 is the least significant bit. Bit ranges are written as in [7:0].
- The VRAM address signals are written as A17–A0.
- To indicate the origin of a function, a function introduced in the V9958 may be marked "9958 Compatible". Likewise, a function introduced in the V9938 is noted as such.
- "Compatible mode" and "extended mode" refer to the operating modes of the V9968 that are switched by the V58 bit of R#21 (Section 1.4).

## 1.3 Overview of the V9968

The V9968 is a Video Display Processor (VDP) developed for the MSX2++.

The VDPs of the MSX series have been passed down from the TMS9918 of the original MSX, through the

V9938 of the MSX2, to the V9958 of the MSX2+ and MSX turboR. The V9968 is the successor in this lineage; it maintains compatibility with the TMS9918 / V9938 / V9958 while providing new functions. Most software written for the earlier VDPs will, in principle, run as-is on the V9968.

At the same time, the V9968 also provides extended functions that the earlier VDPs did not have. This manual distinguishes between the compatible functions and the extended functions in its descriptions, so when using an extended function, check the relevant chapter carefully.

## 1.4 Compatibility and the "9958 Compatible" Mode

The V9968 has a **V58 bit** (bit0 of R#21) that switches the operating mode.

- **Compatible mode (V58 = 1)**: The state at reset. In this mode, operation is equivalent to that of the V9958. In this mode, A17 of the table base addresses is treated as 0, and the new VDP commands (Section 1.5) cannot be used.
- **Extended mode (V58 = 0)**: The mode in which the extended functions of the V9968 are enabled. VRAM of 256KB (18-bit address) becomes available, and the new commands, the extended palette, and other features are enabled.

R#21, which contains the V58 bit, and R#20, another extended register, are protected by a lock mechanism (LockExtReg) to prevent erroneous operation, and writes are ignored immediately after reset. The method for releasing the lock is described in Chapter 2.

In this manual, a function that operates as before in the V9958 compatible mode may be called "9958 Compatible" (Section 1.2).

## 1.5 Functional Overview

### Screen Modes

The screen modes of the V9968 are equivalent to those of the V9958; there are no new screen modes. The available modes are Text1 (SCREEN 0 WIDTH 40), Text2 (SCREEN 0 WIDTH 80), Multicolor (SCREEN 3), Graphic1–Graphic7 (SCREEN 1, 2, 4–8), Graphic7-YAE (SCREEN 10, 11), and Graphic7-YJK (SCREEN 12). The setting method for each mode, the use of VRAM, and the detailed behavior are described in Chapter 5.

## Sprites

In addition to the conventionally compatible Sprite mode1 and Sprite mode2, the sprites provide **Sprite mode3**, newly introduced in the V9968.

In Sprite mode3, coloring can be performed dot by dot, and enlargement / reduction (scaling), horizontal flipping, and vertical flipping are possible. Up to 64 sprites can be displayed, with up to 16 sprites per line. Details are given in Chapter 6.

## Color Palette

The color palette has 256 entries, and each color is held with 5 bits for each of R, G, and B. In addition to writes in the conventional format (3 bits for each of R, G, and B; 16 colors), 5-bit values can be written directly in the extended palette mode (EPAL). Details are given in Chapter 7.

## VDP Commands

In addition to the 13 types of VDP commands available up to the V9958, three new commands, **LRMM, LFMC, and LFMM**, which can be used only in extended mode, have been introduced. The command-related registers have also been extended, including the rotation parameters (R#47–R#50) and the window specification for clipping (R#51–R#58). Details are given in Chapter 8.

## Geo3D

The V9968 incorporates a 3D drawing engine, "Geo3D". Details are given in Chapter 9.

## Interrupts

In addition to the vertical sync interrupt (frame interrupt) and the line interrupt, a command completion interrupt (CEIE, R#20), which reports the completion of a VDP command, is provided. Details are given in Chapter 10.

## VRAM

In extended mode, VRAM can be accessed up to 256KB (18 bits, address A17–A0). In compatible mode, operation is within the conventionally compatible address range. The access method is described in Chapter 2, and the timing and restrictions of access are described in Chapter 11.

# Chapter 2 I/O Ports and Access Methods

All access to the V9968 is performed through I/O ports. This chapter describes the port assignments and the basic access procedures for VRAM, the control registers, the palette, and the status registers. For the bit specifications of each register, see Chapter 3 and Chapter 4.

## 2.1 Port Assignments

The base address of the V9968 is the value stored in the BIOS work area at 0006h / 0007h (normally 98h). Relative to the base address, the following five ports are assigned.

| Port | Write | Read |
|---|---|---|
| Port#0 (base+0) | VRAM data write | VRAM data read |
| Port#1 (base+1) | Two-byte pair protocol (Section 2.3) | The status register pointed to by R#15 |
| Port#2 (base+2) | Palette data write | Returns FFh (no function) |
| Port#3 (base+3) | Indirect register write via the R#17 pointer | Returns FFh (no function) |
| Port#4 (base+4) | Interrupt flag clear + LockExtReg setting | Interrupt status |

The use of each port is described in order below.

In a configuration where the V9968 is connected as a second VDP (2nd VDP), the base address is 88h. In this case, the work area at 0006h / 0007h retains the value of the built-in VDP (98h), so the program must access the device by specifying 88h directly. The handling of the vertical sync interrupt of the 2nd VDP also differs from that of the built-in VDP (see the note in Section 10.8).

## 2.2 Accessing VRAM

Reads and writes of VRAM are performed through Port#0. The address to be accessed is set in advance using the two-byte pair protocol of Port#1 (Section 2.3).

The VRAM address of the V9968 is 18 bits (A17–A0), and the address register is shared between reads and writes. The lower 14 bits of the address (A13–A0) are set with the two-byte pair protocol, and the upper bits, A17–A14, are specified separately in R#14.

## Behavior After the Address Is Set

- If the address was set for writing, the data written to Port#0 is written to that address, and the address is incremented. However, the address is not incremented on the first access immediately after the address is set.
- If the address was set for reading, the first read of Port#0 returns the data at the set address. The read address is incremented at the time of the response from VRAM.
- Bit6 of the second byte of the pair distinguishes between reading (0) and writing (1) (Section 2.3).

## Address Wraparound

The range of the carry (wraparound) when the address is incremented varies with the operating conditions, as follows.

| Condition | Wrap range |
|---|---|
| When the screen mode {M5, M4} is of the 00 group (Graphic1 / Text1 / Multicolor / Graphic2), or when VR of R#8 is 0 | 14 bits (only A13–A0 advance) |
| Otherwise, in compatible mode (V58 = 1) | 17 bits |
| Otherwise, in extended mode (V58 = 0) | 18 bits |

In the group that wraps at 14 bits, for example in Text1 mode, the address after 3FFFh returns to 0000h. This is an intentional specification provided for compatibility with machines equipped with the TMS9918. By specifying A17–A14 in R#14, the area from 4000h onward can also be accessed, but note that because the carry from bit13 to bit14 is blocked, the address after 7FFFh returns to 4000h.

A17 set in R#14 is treated as fixed at 0 when the VR bit of R#8 is 0, and is also masked in compatible mode (V58 = 1).

### Access Restrictions While a Command Is Executing

While the MXC bit of R#45 is 1, VRAM access from the CPU is masked. During this time, writes to VRAM are not executed, and reads return FFh. For MXC, see Chapter 8.

## 2.3 The Port#1 Two-Byte Pair Protocol

Writes to Port#1 are always treated as a pair of two bytes. The value (data or the lower part of an address) is written as the first byte, and its meaning is specified by bit7 / bit6 of the second byte.

| Bit7 of the second byte | Bit6 of the second byte | Meaning |
|---|---|---|
| 1 | — | Control register write. The value of the first byte is written to the register (R#0–R#63) specified by bit5–bit0 |
| 0 | 1 | VRAM write address setting. Bit5–bit0 become A13–A8 |
| 0 | 0 | VRAM read address setting. Bit5–bit0 become A13–A8 |

In either case, the first byte is the lower 8 bits of the VRAM address (A7–A0), or the data to be written to a register.

### Aborting a Pair

After the first byte of a two-byte pair has been written, if Port#0 is accessed, or if any port is read, before the second byte is written, the pair being written is discarded. Do not leave the state with only the first byte written.

Writes to Port#3 (Section 2.4) do not affect the state of this two-byte pair.

## 2.4 Writing to the Control Registers

All control registers are write-only, and can be written by two methods: direct specification and indirect specification.

### Direct Specification (Port#1)

This is the method of writing the value as the first byte and 80h + the register number as the second byte, using the protocol of Section 2.3. Any register can be rewritten with a single pair.

### Indirect Specification (Port#3)

This is the method of writing by writing a value to Port#3, which writes to the register pointed to by R#17 (control register pointer). It is suited to feeding values in sequence to consecutive registers.

- Bit5–bit0 of R#17 are the pointer, and bit7 is AII (auto-increment inhibit).
- When AII = 0, the pointer is incremented each time Port#3 is written. When AII = 1, the pointer is fixed.
- R#17 itself can be set only by direct specification (via Port#1). Even if the pointer points to 17 via Port#3, that value is ignored (however, if AII = 0, the pointer alone is incremented).

## 2.5 Accessing the Palette

Palette data is written through Port#2. The palette number to be written is specified in advance in R#16 (color palette pointer). Writing R#16 returns the phase of the palette write (which byte is next) to 0.

There are two write formats, depending on the EPAL bit of R#20.

### Normal Format (EPAL = 0)

The number (0–15) is specified in bit3–bit0 of R#16, and two bytes are written to Port#2.

| Byte | Bit composition |
|---|---|
| First byte | [0][R2][R1][R0][0][B2][B1][B0] |
| Second byte | [0][0][0][0][0][G2][G1][G0] |

Each 3-bit value of R, G, and B is expanded internally to 5 bits, in a form that replicates the upper bits into the lower bits, and stored. Each time one pair is written, the palette number is automatically incremented within the range of [3:0] (the upper 4 bits are fixed at 0).

### Extended Format (EPAL = 1)

The number (0–255) is specified in bit7–bit0 of R#16, and three bytes are written to Port#2. Each byte is a 5-bit value of the form [0][0][0][value4][value3][value2][value1][value0], representing R, G, and B in order. Each time one set is written, the palette number is automatically incremented in 8 bits.

The palette RAM is organized as 256 entries × 5 bits for each of R, G, and B. For details of color representation, see Chapter 7.

At reset, only the first 16 entries are initialized, to the V9958-compatible initial colors. The initial values of entries 16 and onward are undefined.

## 2.6 Reading the Status Registers

The status registers are read-only. By specifying a number in R#15 (status register pointer) and reading Port#1, the value of that status register, S#0–S#9, is obtained. The pointer at reset is 0. If S#10–S#15 is specified, FFh is returned.

Some status registers have flags and the like cleared when they are read. The principal ones are as follows.

| Register | Side effect on read |
|---|---|
| S#0 | The interrupt flag F, the sprite collision flag C, and the overmap flag 5S are cleared |
| S#1 | The line interrupt flag FH is cleared |
| S#5 | The sprite collision coordinates (X / Y) are cleared |
| S#9 | BD of S#2 is cleared |

For the bit specifications of each status register and details of the side effects, see Chapter 4.

## 2.7 Interrupt Control Port (Port#4)

Port#4 is the port that handles the checking and clearing of interrupt sources, and the lock of the extended registers (LockExtReg).

## Read

The bit composition of the value read is as follows.

| bit | Name | Meaning |
|---|---|---|
| bit7 | LockExtReg | Lock state of the extended registers (1 at reset) |
| bit6–bit3 | — | 0 |
| bit2 | CI | 1 if the command completion interrupt source is active |
| bit1 | LI | 1 if the line interrupt source is active |
| bit0 | FI | 1 if the frame interrupt source is active |

## Write

- Writing 1 to bit2 / bit1 / bit0 clears the corresponding interrupt flag (command completion / line / frame). Writing 0 has no effect.
- Bit7 is a write to LockExtReg itself.

## Locking the Extended Registers (LockExtReg)

LockExtReg is 1 (locked state) at reset. While locked, **writes to the extended registers R#20 and R#21 are ignored**.

To use the extended functions, release the lock by the following procedure, and then perform the settings.

1. Write a value with bit7 = 0 to Port#4 (LockExtReg becomes 0, and the lock is released).
2. Set R#20 / R#21, for example by setting the V58 bit of R#21 to 0 to switch to extended mode.

# Chapter 3 Control Registers in Detail

This chapter describes the V9968 control registers R#0–R#27 and the VDP command registers R#32–R#58 in detail, one register at a time.

All control registers are **write-only**. There is no way to read them back (reading Port#1 reads a status register, as described in the next chapter). For how to access each register (direct specification / indirect specification), see Chapter 2.

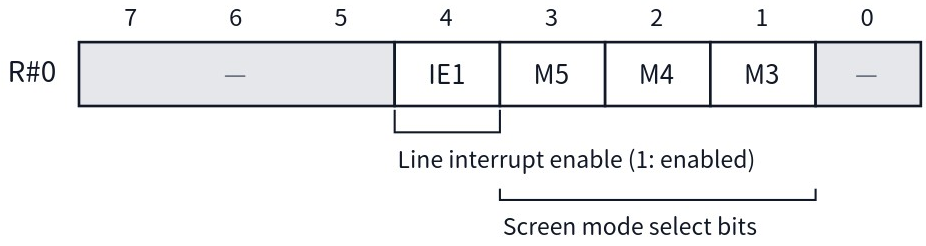
Two prerequisites run through this chapter and are important throughout:

- **The V58 bit (R#21 bit0)**: At reset this is 1, and this state is the **V9958 compatible mode**. Setting it to 0 selects the **extended mode**, which enables the new commands (LRMM / LFMC / LFMM) and also puts VRAM into 256KB mode. In this chapter, the two states are referred to as "compatible mode (V58=1)" and "extended mode (V58=0)".
- **LockExtReg (Port#4 bit7)**: At reset this is 1, and while it is 1, **writes to R#20 and R#21 are ignored**. To change R#20 / R#21, first set LockExtReg to 0 via Port#4 to release the lock (see Chapter 2 for Port#4).

In each register heading, in addition to the name, the access attribute and the reset value are shown. Bits described as "unused" are not retained in the register when a value is written, and do not affect operation.

## 3.1 Mode Setting Registers

### R#0 Mode0

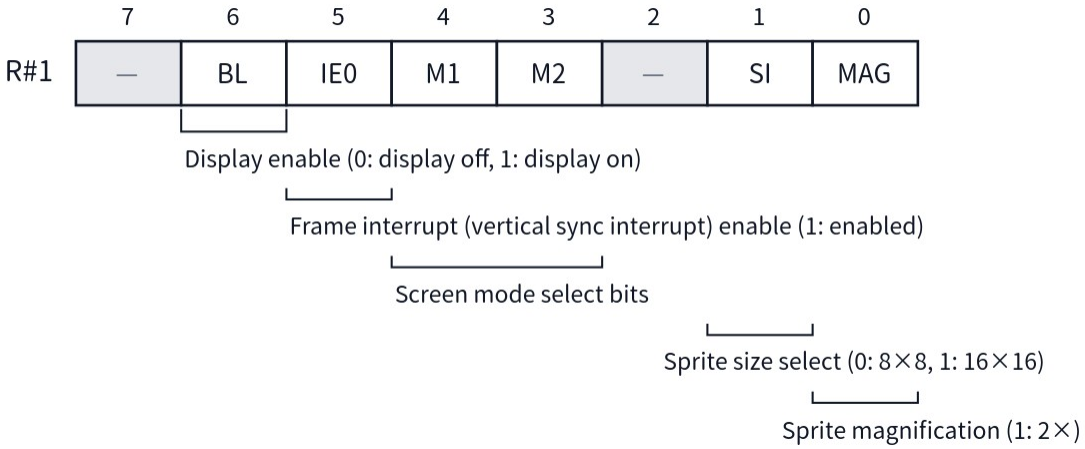



- Access: Write-only
- Reset value: M5=M4=M3=0, IE1=0

The screen mode is determined by a total of five bits: M5–M3 in R#0 and M2, M1 in R#1. For the correspondence between the combinations and the screen modes, see the table in the section on R#1.

As a side effect, writing to R#0 clears the line interrupt flag.

## R#1 Mode1



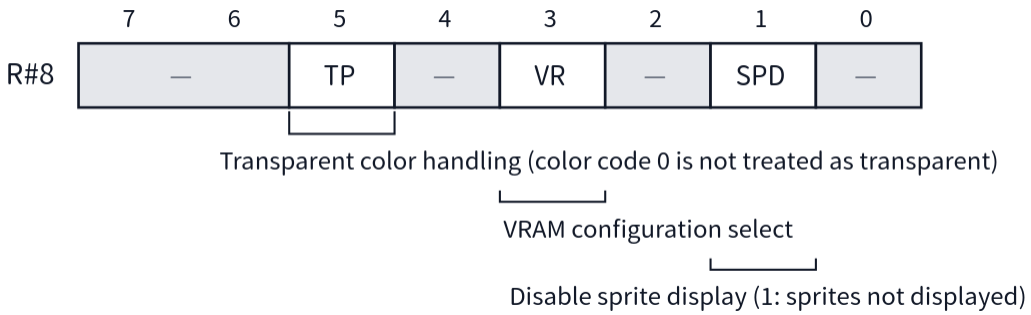

- Access: Write-only
- Reset value: BL=0, IE0=0, SI=0, MAG=0

The correspondence between the screen mode selection bits {M5, M4, M3, M2, M1} and the screen modes is as follows (for details of each mode, see Chapter 5).

| M5 M4 M3 M2 M1 | Screen Mode |
|---|---|
| 00000 | Graphic1 (SCREEN1) |
| 00001 / 00101 | Text1 (SCREEN0 WIDTH40) *M3 is a don't-care |
| 00010 / 00110 | Multicolor (SCREEN3) *M3 is a don't-care |
| 00100 | Graphic2 (SCREEN2) |
| 01000 | Graphic3 (SCREEN4) |
| 01100 | Graphic4 (SCREEN5) |
| 10000 | Graphic5 (SCREEN6) |

| M5 M4 M3 M2 M1 | Screen Mode |
|---|---|
| | |
| 10100 | Graphic6 (SCREEN7) |
| 11100 | Graphic7 (SCREEN8 / 10 / 11 / 12) |
| 01001 | Text2 (SCREEN0 WIDTH80) |

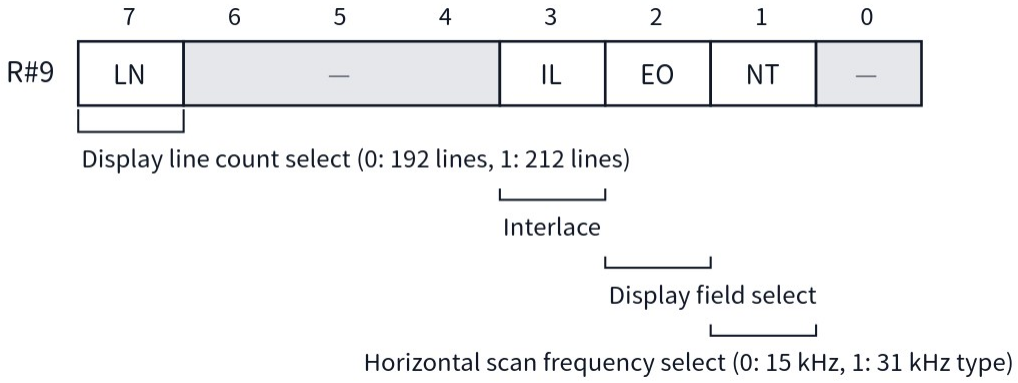
## R#8 Mode2




- Access: Write-only
- Reset value: TP=0, VR=1, SPD=0

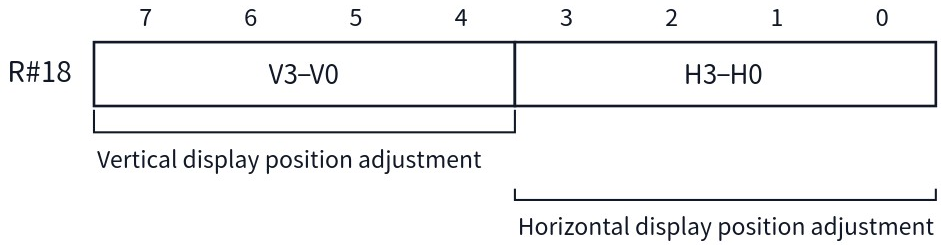
VR concerns VRAM access. When VR=0, the upper bits (A17–A14) of the VRAM address specified by R#14 are fixed at 0, and address incrementing also wraps within the 14-bit range (see also Chapter 2).

## R#9 Mode3




- Access: Write-only
- Reset value: All 0

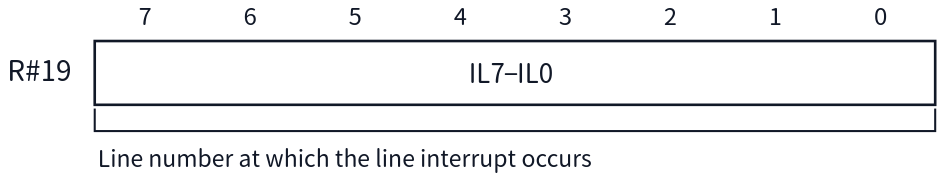
## R#18 Display Adjust




- Access: Write-only
- Reset value: 0

This fine-adjusts the display position of the entire screen. The interpretation of the value (sign and direction) follows the specifications of the display system.

## R#19 Interrupt Line



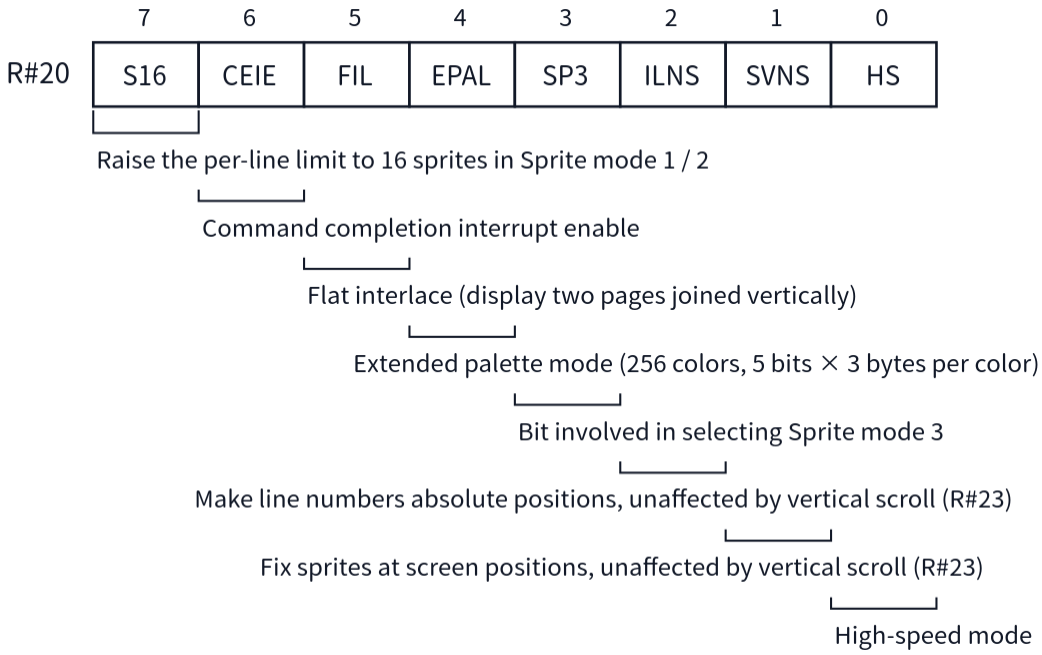

- Access: Write-only
- Reset value: 0

When the display line reaches this number, the line interrupt flag is set (an interrupt actually occurs when IE1=1 in R#0).

As a side effect, writing to R#19 clears the line interrupt flag (writing to R#0 also clears it).

Note that the line number in R#19 is normally affected by the vertical scroll offset (R#23). Setting ILNS=1 in R#20 makes it interpreted as an absolute position with the top of the screen as 0, unaffected by R#23 (see the section on R#20 and Chapter 10).

### R#20 Mode5 [New in V9968]




- Access: Write-only (but writes are ignored while LockExtReg=1)
- Reset value: All 0

This is a mode register added in the V9968. EPAL switches the format of palette writes via Port#2 (see Chapter 2 and Chapter 7). Using CEIE allows the completion of a VDP command to be reported by an interrupt.

S16 is a bit that extends the number of sprites displayed per line (initial value 0). In Sprite Mode 3, up to 16 sprites are always displayed per line, so S16 has no effect. In Sprite Mode 2, up to 8 sprites can be displayed when S16=0, for compatibility with earlier chips, and up to 16 when S16=1. In Sprite Mode 1, up to 4 sprites can be displayed when S16=0, for compatibility with earlier chips, and up to 16 when S16=1 (for details, see Chapter 6).

HS is ORed with the board-side input signal (force_highspeed), so even with HS=0, board-side conditions may cause high-speed mode to be entered.

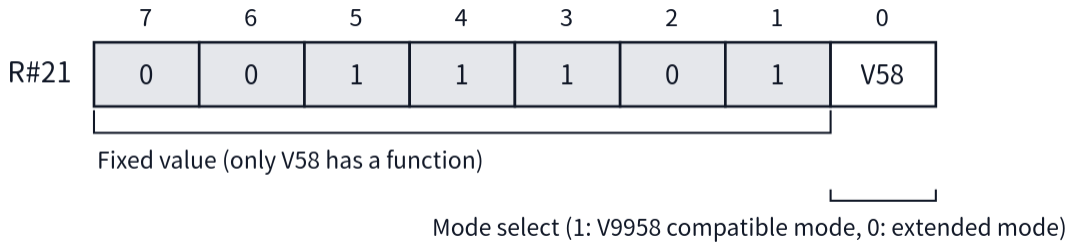
FIL is the flat interlace mode. In the conventional interlace mode, page0 and page1 (or page2 and page3) are displayed alternately, shifted by half a pixel in the vertical direction, but the two pages remain separate screens, so a line drawn across the pages does not connect. Setting FIL=1 makes it possible to display page0 and page1 as if joined vertically in VRAM, so they can be treated as a single vertically long screen. VDP commands can also be used on this combined high-resolution screen (see Chapter 5 and

Chapter 8).

ILNS switches the interpretation of the line number for the line interrupt. Normally, the line number for the line interrupt (R#19) is affected by the vertical scroll offset (R#23). Setting ILNS=1 makes the line number in R#19 interpreted as an absolute position based on the top of the screen (R#19=0), unaffected by the value of R#23. This is used when an interrupt is wanted at a fixed position on the screen while the vertical scroll (R#23) is being moved (see Chapter 10).

SVNS switches the relationship between the sprite display position and the vertical scroll (R#23). Normally, sprites are affected by the vertical scroll of R#23 together with the background. Setting SVNS=1 makes sprites unaffected by the R#23 specification, so that a sprite's Y coordinate refers to an absolute position on the screen (with Y=0, the sprite is displayed at the very top of the screen). This is used when only the background is to be scrolled while the sprites are kept fixed on the screen (see Chapter 6).

### R#21 Mode6 [New in V9968]




- Access: Write-only (but writes are ignored while LockExtReg=1)
- Reset value: V58=1

Bit7–bit1 are the fixed values shown above in the current bit assignment. Only bit0, V58, has a function.

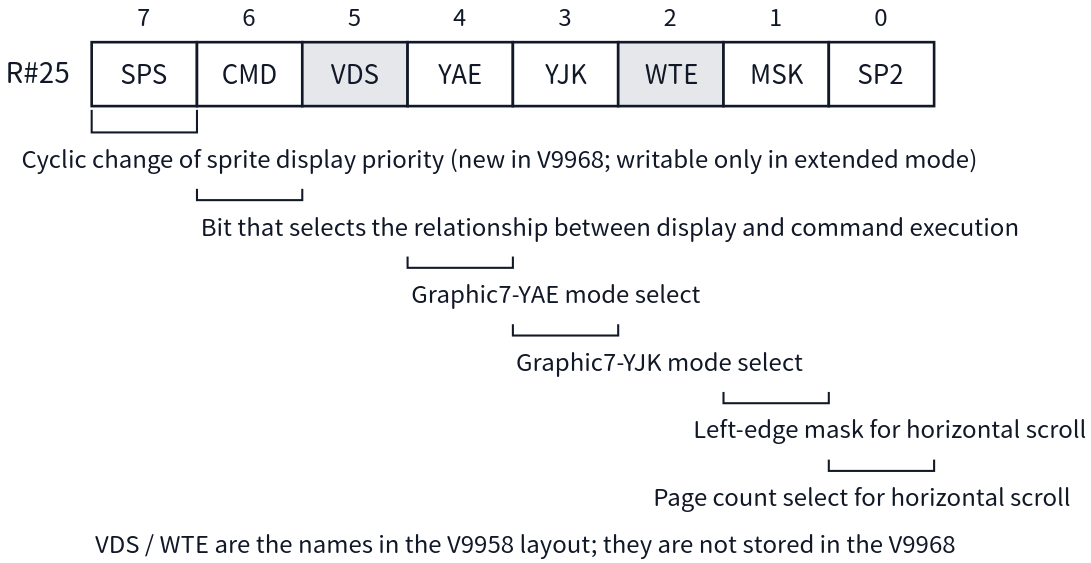
V58 is a bit that switches the overall operating mode of the V9968.

- **V58=1 (compatible mode)**: The new commands (LRMM / LFMC / LFMM) are disabled. The A17 of addresses written to R#2 / R#4 / R#6 / R#10 / R#11 is retained as 0.
- **V58=0 (extended mode)**: The new commands are enabled, and VRAM enters 256KB mode.

As a side effect, writing 1 to R#21 (returning to compatible mode) clears bit17 of all five table base addresses (pattern name / pattern generator / sprite pattern generator / color / sprite attribute) to 0.

Note that some older materials refer to "V58 in R#20" in places, but V58 is bit0 of this register, R#21.

### R#25 Mode4




- Access: Write-only
- Reset value: All 0

When using YJK / YAE in Graphic7 (equivalent to SCREEN8), in addition to the screen mode selection (R#0 / R#1), the color representation format is selected with the YJK / YAE bits of this register (see Chapter 5).

The amount of horizontal scroll is specified by R#26 / R#27 (see the sections on each register). The MSK and SP2 bits of this register control how that appears.

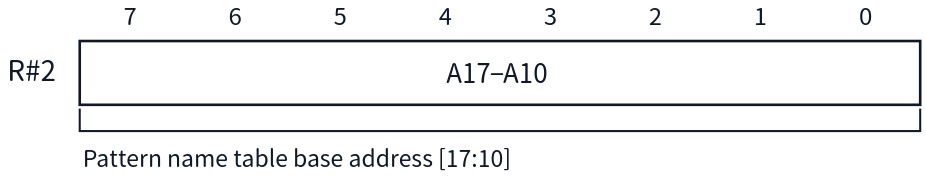
- **MSK**: When 0, the leftmost 8 dots are not masked (initial value). When 1, the leftmost 8 dots are masked and the border color is displayed. If the value of R#27 is 0, there is no need to mask (in Graphic5 and Graphic6, the mask is 16 dots).
- **SP2**: When 0, the horizontal screen size is 1 page (initial value). Scrolling takes place within 1 page, and on the screen the hidden part on the left appears on the right. When 1, the horizontal screen size is 2 pages, scrolling takes place over 2 pages, and the back page appears as a result of scrolling.

- **SPS**: This is a bit newly established in the V9968. It can be written only in extended mode (V58=0); in compatible mode (V58=1), writes are ignored. Setting it to 1 makes the sprite display priority change cyclically. It can be used in all of Sprite Mode 1 / 2 / 3 (see Chapter 6).

## 3.2 Table Address Registers

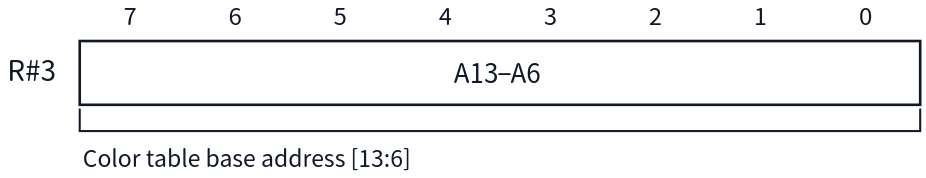
The following registers specify the base address of each table in VRAM. All are write-only, and the reset value is 0. In compatible mode (V58=1), the A17 written to a register that includes A17 is retained as 0.

### R#2 Pattern Name Table Address




Pattern name table base address [17:10]

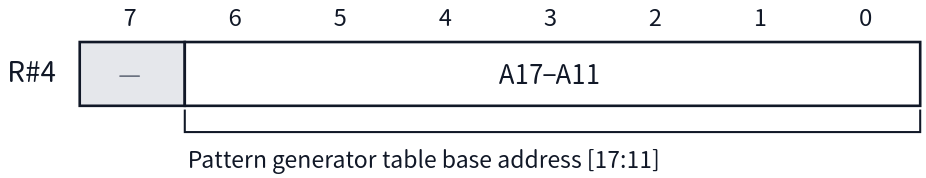
### R#3 Color Table Address (L)




Color table base address [13:6]

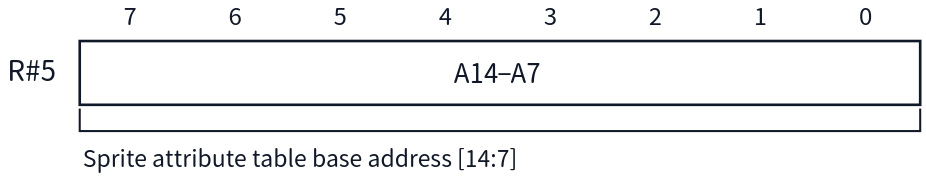
The base address of the color table is composed of this register (lower) and R#10 (upper).

### R#4 Pattern Generator Table Address




Pattern generator table base address [17:11]

### R#5 Sprite Attribute Table Address (L)



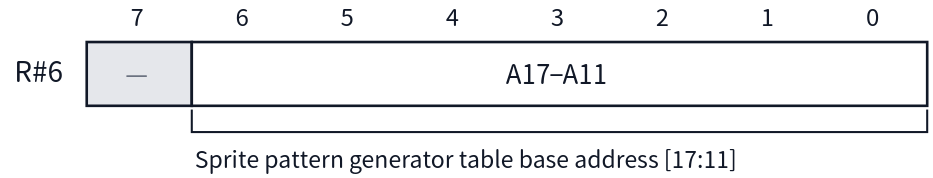

Sprite attribute table base address [14:7]

The base address of the sprite attribute table is composed of this register (lower) and R#11 (upper).

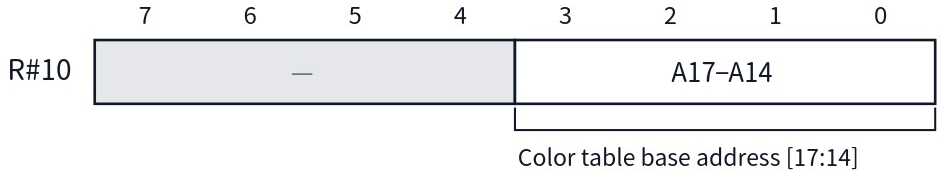
Bit1 / bit0 of this register (bits [8:7] of the base address) are also used as an AND mask with the upper bits

of the plane number when generating attribute addresses in Sprite Mode 2 / 3. This is a mask function for making the attribute table appear smaller (see Chapter 6).

### R#6 Sprite Pattern Generator Table Address




### R#10 Color Table Address (H)

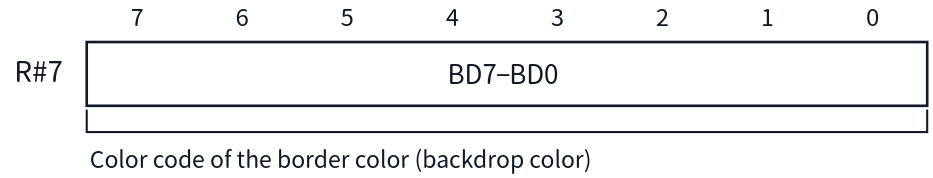



### R#11 Sprite Attribute Table Address (H)




## 3.3 Color and Display Control Registers

### R#7 Background Color

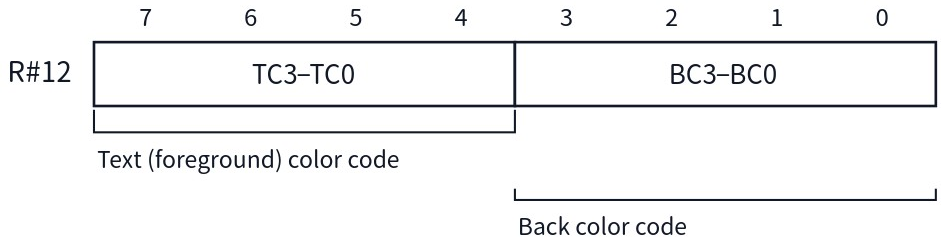



- Access: Write-only
- Reset value: 0

The entire byte written is retained in the register. The interpretation on the display side differs for each

screen mode; for example, in some screen modes only the upper nibble is used (see Chapter 5).

### R#12 Text Color / Back Color

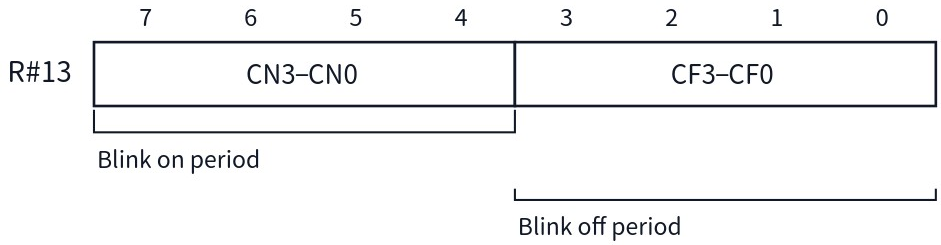



- Access: Write-only
- Reset value: 0

During the blink display period, the color of this register is used instead of R#7. The blink period is set by R#13.

[TBC] The notation of the bit names for the text color field of this register is not consistent across the source materials (the notation in the RTL comments appears to contain an error). Please confirm the official bit names.

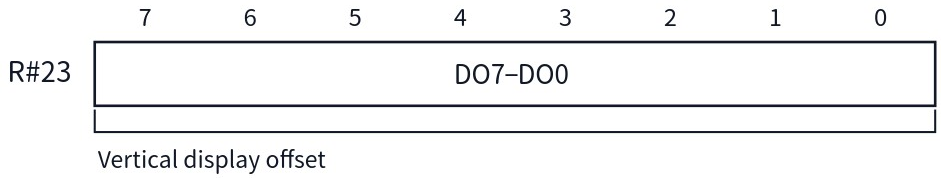
### R#13 Blinking Period




- Access: Write-only
- Reset value: 0

This sets the length of the period during which the color of R#12 is displayed (on) and the period during which the display returns to the color of R#7 (off).

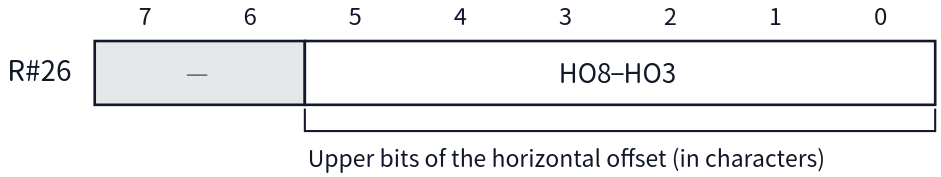
## R#23 Display Offset




- Access: Write-only
- Reset value: 0

This shifts the display start position of the screen in line units (vertical scroll).

## R#26 Horizontal Offset (By Character)




- Access: Write-only
- Reset value: 0

The amount of horizontal scroll is specified by 9 bits combining this register (upper) and R#27 (lower).

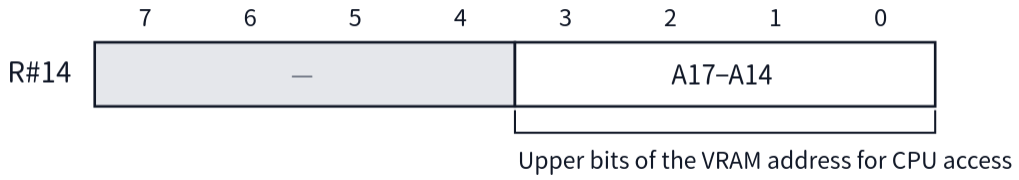
## R#27 Horizontal Offset (By Dot)




- Access: Write-only
- Reset value: 0

## 3.4 Access Control Registers

### R#14 VRAM Access Base Address

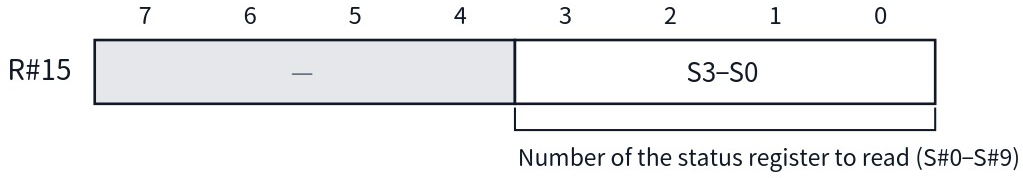



- Access: Write-only
- Reset value: 0 (0 for the VRAM address as a whole)

This specifies the upper bits of the address when the CPU accesses VRAM via Port#0. The lower bits (A13–A0) are set by the Port#1 protocol (see Chapter 2). The address register is 18 bits, common to reads and writes.

When the VR bit of R#8 is 0, A17–A14 are fixed at 0. Also, in compatible mode (V58=1), A17 is masked.

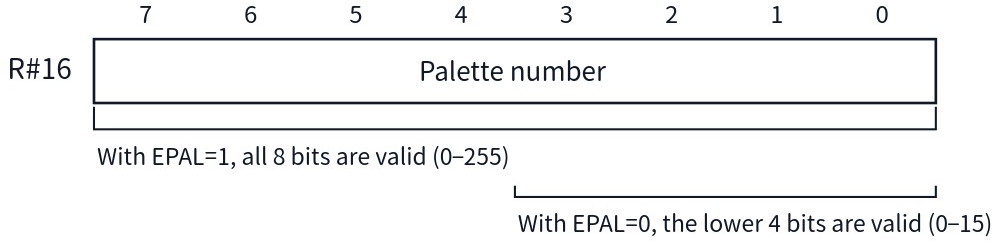
### R#15 Status Register Pointer




- Access: Write-only
- Reset value: 0

Reading Port#1 returns the value of the status register specified by this register (see Chapter 4). At reset it points to 0 (S#0). If S#10–S#15 is specified, FFh is returned.

### R#16 Color Palette Pointer

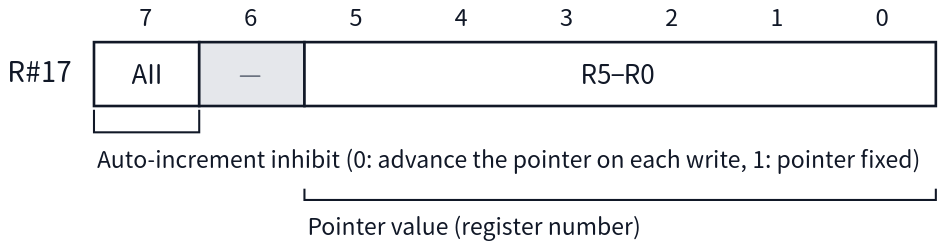


- Access: Write-only
- Reset value: Palette address 0

This specifies the starting number when writing palette data via Port#2. When EPAL (R#20 bit4) is 0, only the lower 4 bits are valid; when it is 1, all 8 bits are valid.

As a side effect, writing to R#16 returns the phase of the palette write (which byte is being accepted) to the beginning. If the number is changed in the middle of a palette write, always rewrite this register before sending the data.

### R#17 Control Register Pointer




- Access: Write-only
- Reset value: Pointer 0, AII=0

This is the pointer used for indirect writes via Port#3. Note that this register itself can be set only by direct specification via Port#1. Even if the pointer points to R#17 via Port#3, that written value is ignored (but when AII=0, the pointer alone is incremented).

### Unimplemented Registers (R#22, R#24, R#28–R#31)

R#22 and R#24 have no number assigned to them in the first place, and R#28–R#31 are also not implemented. Writes to these numbers are ignored.

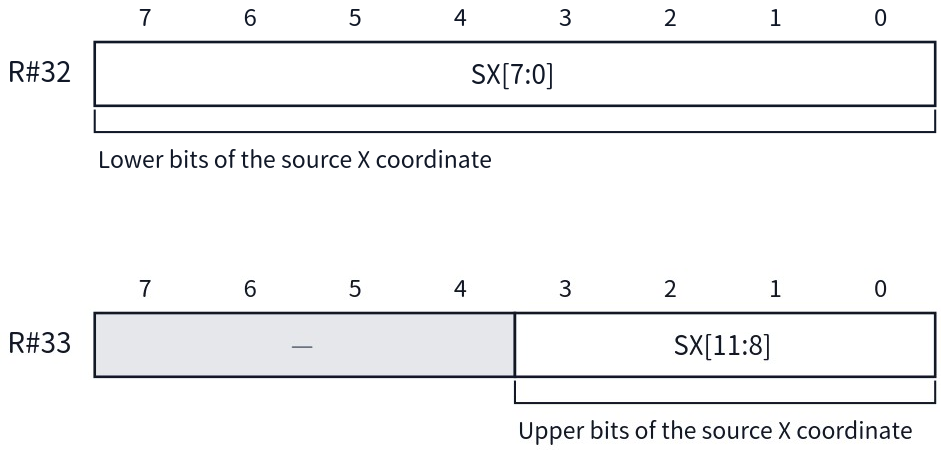
## 3.5 Command Registers

R#32–R#58 are registers for the parameters and execution instructions of VDP commands. All are write-only. For the operation of each command itself, see Chapter 8. This section describes their specifications as registers (bit layout, reset value, side effects).

Coordinates and dot counts are composed as multi-bit values spanning multiple registers. In what

follows, "(L)" is the register that holds the lower byte, and "(H)" is the register that holds the upper bits.

### R#32 Source X (L) / R#33 Source X (H)

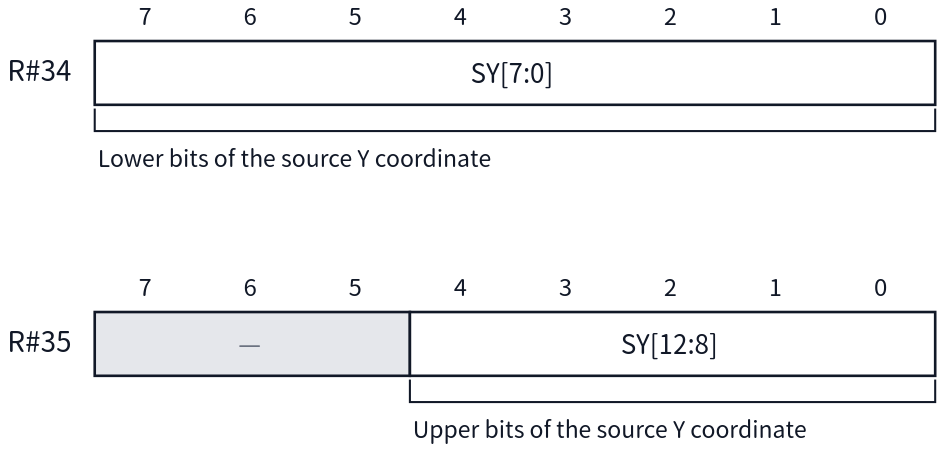



- Reset value: 0

SX is retained up to 12 bits, but in commands other than LRMM, SX[11:9] is cleared to 0 at the start, so the effective width is 9 bits.

As a side effect, writes to R#32 / R#33 are also written simultaneously to [7:0] / [15:8] of the font address register for LFMC / LFMM (see also R#34 in the next item).

### R#34 Source Y (L) / R#35 Source Y (H)

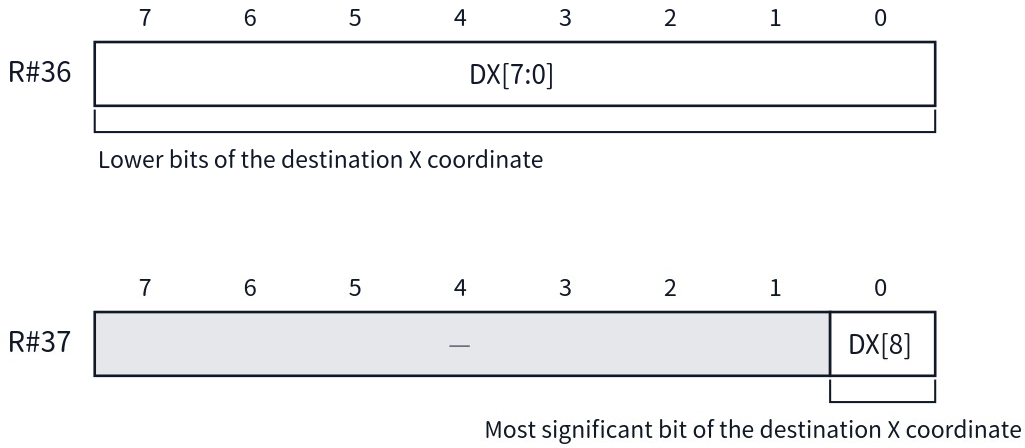



- Reset value: 0

SY retains 13 bits. At the start of a command, SY[12:11] is cleared in 256KB mode and SY[12:10] is cleared in 128KB mode (the effective width is 11 bits / 10 bits, respectively).

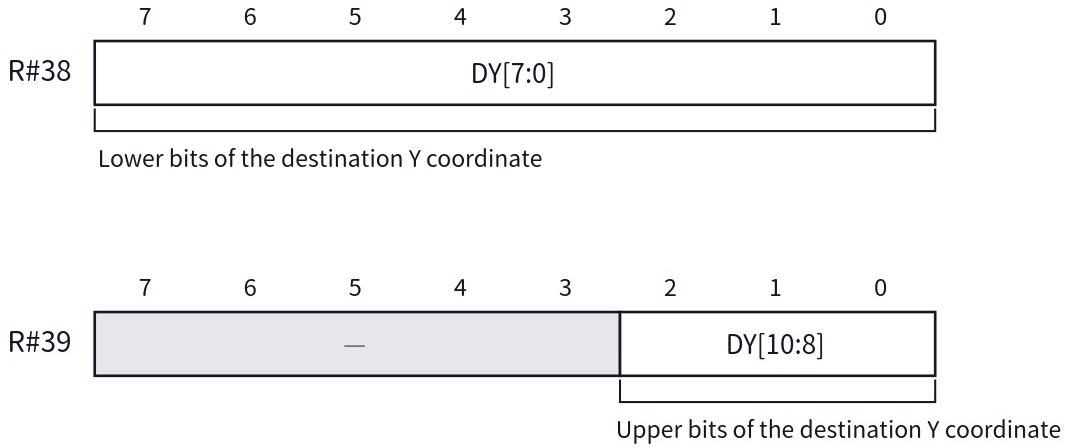
As a side effect, a write to R#34 is also written simultaneously to [17:16] of the font address register. That is, note that the three registers R#32–R#34 overwrite the font address (18 bits).

## R#36 Destination X (L) / R#37 Destination X (H)




- Reset value: 0

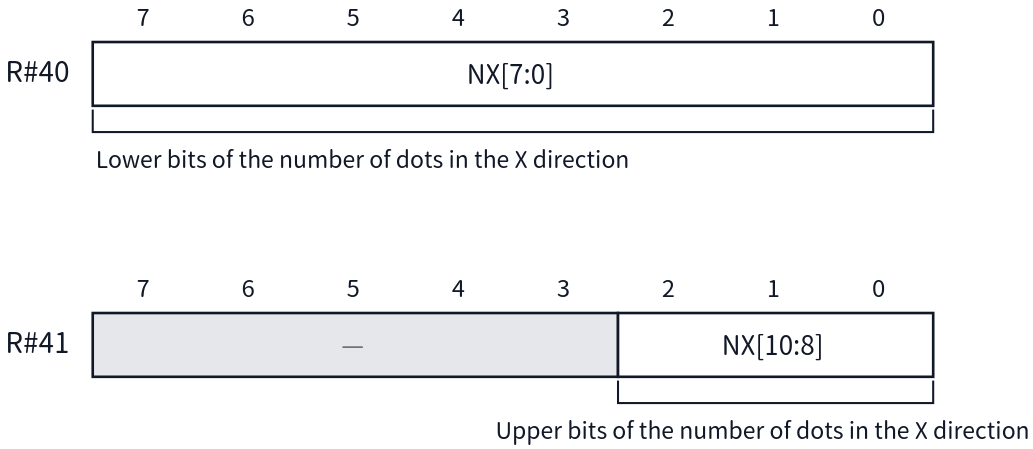
## R#38 Destination Y (L) / R#39 Destination Y (H)




- Reset value: 0

The valid bits of R#39 differ depending on the mode. In 256KB mode, the 3 bits bit2–bit0 are valid, but in 128KB mode only bit1–bit0 are valid, and DY[10] is fixed at 0.

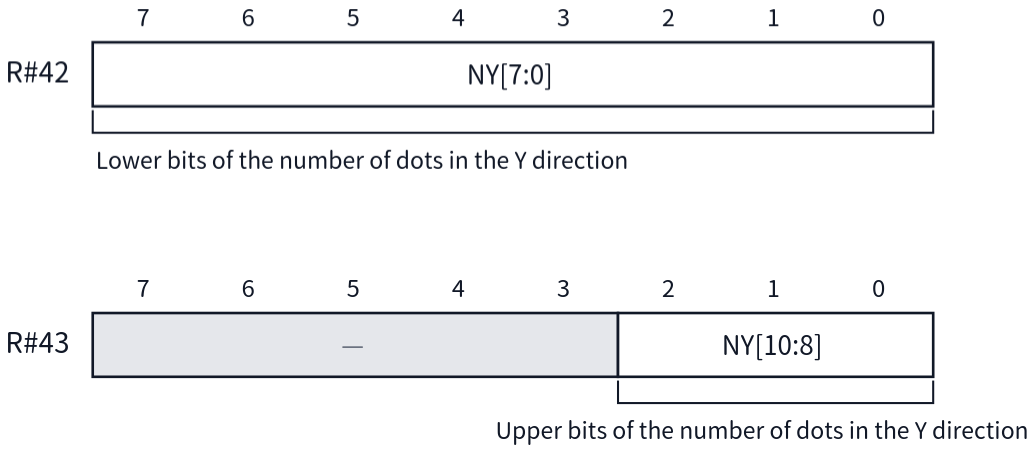
## R#40 Number Of Dots X (L) / R#41 Number Of Dots X (H)




- Reset value: 0

NX retains 11 bits, but in commands other than LRMM, NX[10:9] is cleared to 0 at the start, so the effective width is 9 bits.

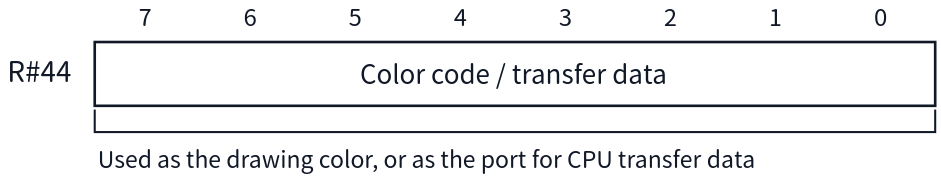
## R#42 Number Of Dots Y (L) / R#43 Number Of Dots Y (H)




- Reset value: 0

The valid bits of R#43 are mode-dependent, as with R#39 (3 bits in 256KB mode; the upper bits are restricted in 128KB mode).

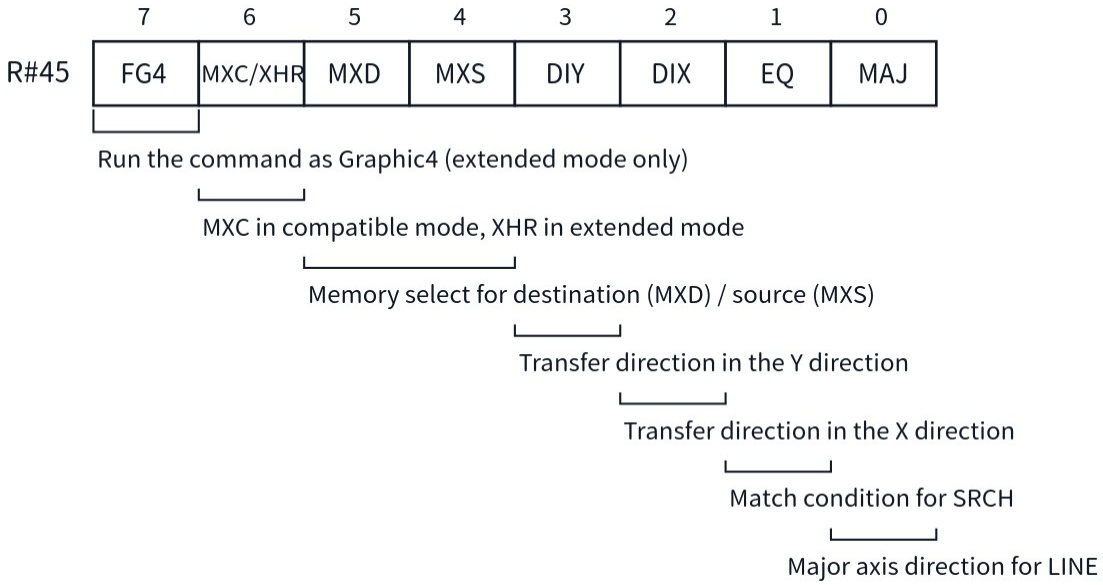
## R#44 Color




- Reset value: 0

In HMMV / LMMV / PSET and similar commands, this is used as the drawing color. In HMMC / LMMC / LFMC, it serves as the receiving port for data transferred from the CPU. Conversely, while LMCM is executing, the value of the pixel read out is stored in this register and can be read via S#7 (see Chapter 4).

## R#45 Argument




- Reset value: All 0

| Bit | Name | Meaning |
|---|---|---|
| 7 | FG4 | [New in V9968] When 1, VDP commands operate as Graphic4 (SCREEN 5) regardless of the screen mode specification. Valid only in extended mode (in compatible mode it is masked to 0 and retained) |

| Bit | Name | Meaning |
|---|---|---|
| 6 | MXC / XHR | In compatible mode it is taken in as MXC; in extended mode as XHR (in LRMM, pixels are treated as vertically long) |
| 5 | MXD | Destination memory selection (in the V9968, extended RAM has been abolished, so the target is VRAM only) |
| 4 | MXS | Source memory selection (same as above) |
| 3 | DIY | Transfer direction in the Y direction (0: downward, 1: upward) |
| 2 | DIX | Transfer direction in the X direction (0: rightward, 1: leftward) |
| 1 | EQ | SRCH match condition (equal to / not equal to the border color) |
| 0 | MAJ | Major-axis direction of LINE (0: X direction, 1: Y direction) |

MXC has meaning only in compatible mode. While MXC=1, VRAM access from the CPU is masked (writes are not executed, and reads return FFh).

Bit6 is a doubly defined bit whose meaning changes depending on the mode. In compatible mode (V58=1) it is taken in as MXC; in extended mode (V58=0) it is taken in as XHR. Also, FG4 in bit7 is masked to 0 and retained unless in extended mode.

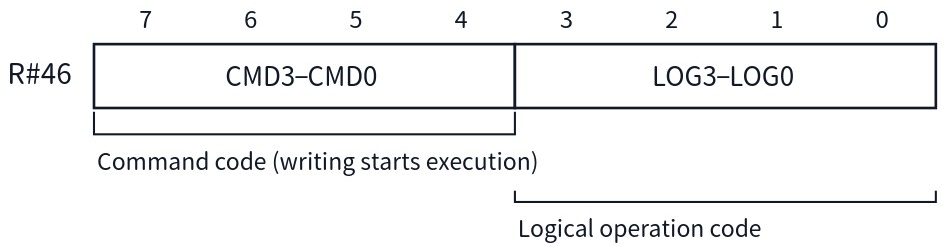
XHR has meaning only in extended mode. When XHR=1, LRMM treats pixels as vertically long pixels. This is a mechanism for performing rotated display with the correct aspect in Graphic5 and Graphic6 (SCREEN 6, 7) (see Chapter 8).

MXS / MXD are extended RAM selection bits originating from the V9938 / V9958, but in the V9968 extended RAM has been abolished, so only VRAM can be selected as the transfer source and transfer destination.

FG4 has meaning only in extended mode. When FG4=1, VDP commands operate as Graphic4 (SCREEN 5) whatever the screen mode specification may be. Including SCREEN 1–4, whatever screen mode the

display is in, the command side is processed in the Graphic4 coordinate and color format. This function was provided in order to draw into the Sprite Mode 3 sprite patterns (whose address calculation is in Graphic4 format) using VDP commands; for example, it allows lines to be drawn into a sprite pattern while the display remains SCREEN 6, 7, or 8 (see Chapter 6 and Chapter 8). Note that if the display itself is Graphic4 (SCREEN 5), the commands operate as Graphic4 to begin with, so the operation does not change with FG4.

## R#46 Command




- Reset value: Command code 0 (STOP), logical operation 0

Writing to this register starts execution of the command at that moment. Set the other parameter registers (R#32–R#45, R#47–R#58) before writing R#46.

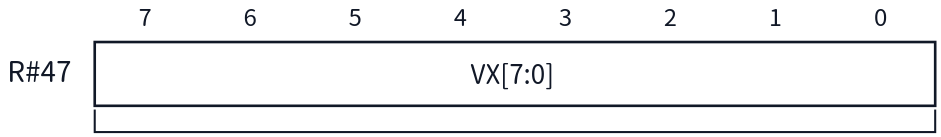
The assignment of the command codes is as follows. LRMM / LFMC / LFMM are commands newly established in the V9968, and are valid only in extended mode (V58=0).

| Code | Command | Code | Command |
|---|---|---|---|
| 1111 | HMMC | 0111 | LINE |
| 1110 | YMMM | 0110 | SRCH |
| 1101 | HMMM | 0101 | PSET |
| 1100 | HMMV | 0100 | POINT |
| 1011 | LMMC | 0011 | LRMM [New in V9968] |
| 1010 | LMCM | 0010 | LFMC [New in V9968] |

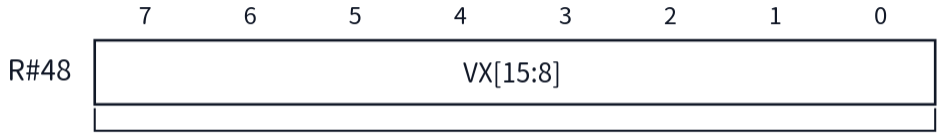
| Code | Command | Code | Command |
|---|---|---|---|
| 1001 | LMMM | 0001 | LFMM [New in V9968] |
| 1000 | LMMV | 0000 | STOP |

Note that some older materials describe an "ECOM bit" as the condition for enabling the new commands. ECOM was a bit once provided to enable the new commands, but in the current design it has been integrated into V58, and no independent ECOM bit exists. The enabling condition is extended mode (V58=0).

## R#47 Rotation Vector X (L) / R#48 Rotation Vector X (H) [New in V9968]




Lower bits of the rotation vector X component

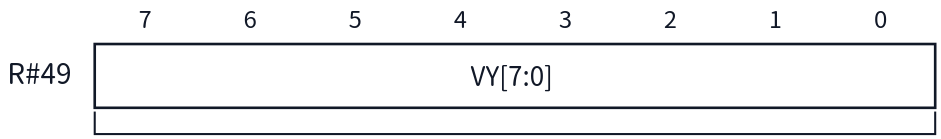



Upper bits of the rotation vector X component

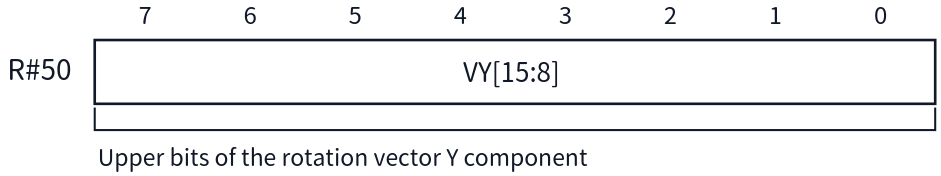
- Reset value: 0

This is the X component of the rotation vector used by the LRMM command. VX is a signed 16-bit value and, when LRMM executes, is sign-extended as a fixed-point number with 8 fractional bits (8.8 format) and added to the coordinates.

## R#49 Rotation Vector Y (L) / R#50 Rotation Vector Y (H) [New in V9968]



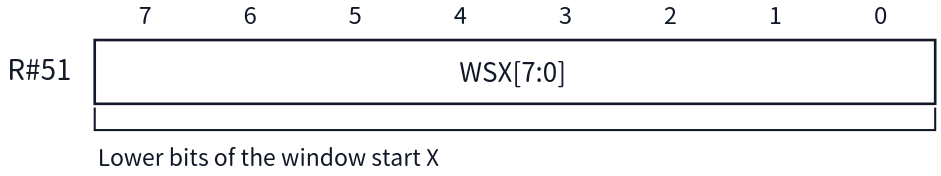

Lower bits of the rotation vector Y component

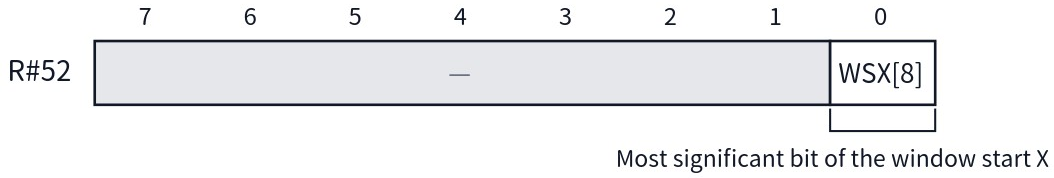



- Reset value: 0

This is the Y component of the rotation vector. The format is the same as VX.

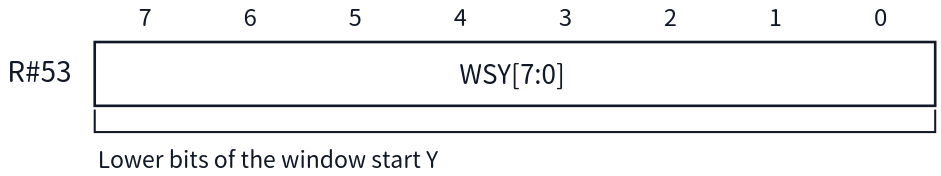
### R#51 Window Start X (L) / R#52 Window Start X (H) [New in V9968]

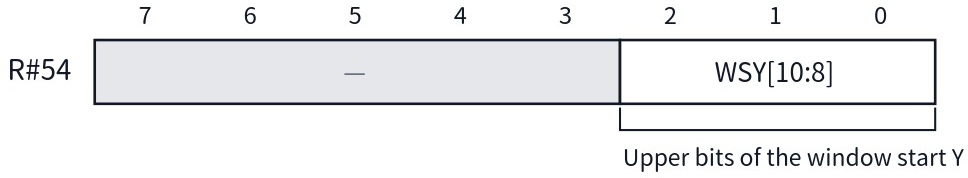



- Reset value: 0

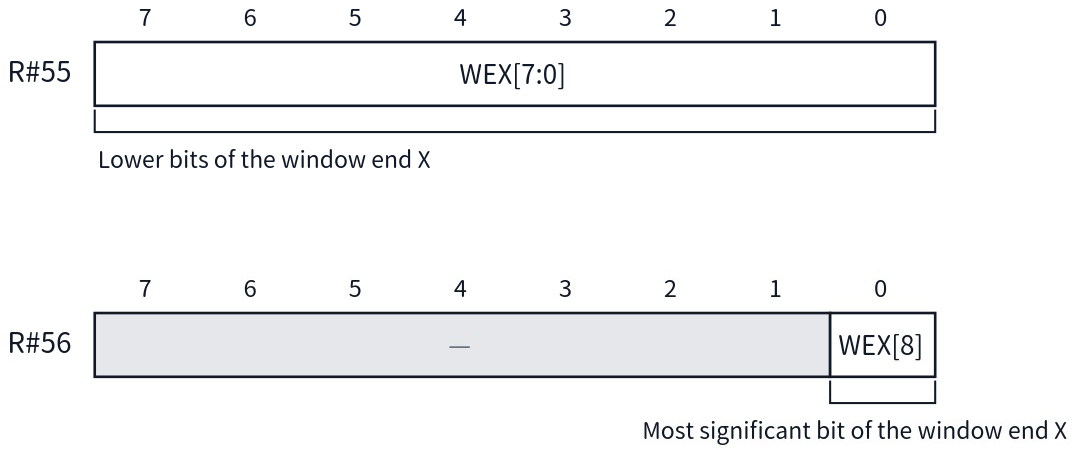
### R#53 Window Start Y (L) / R#54 Window Start Y (H) [New in V9968]



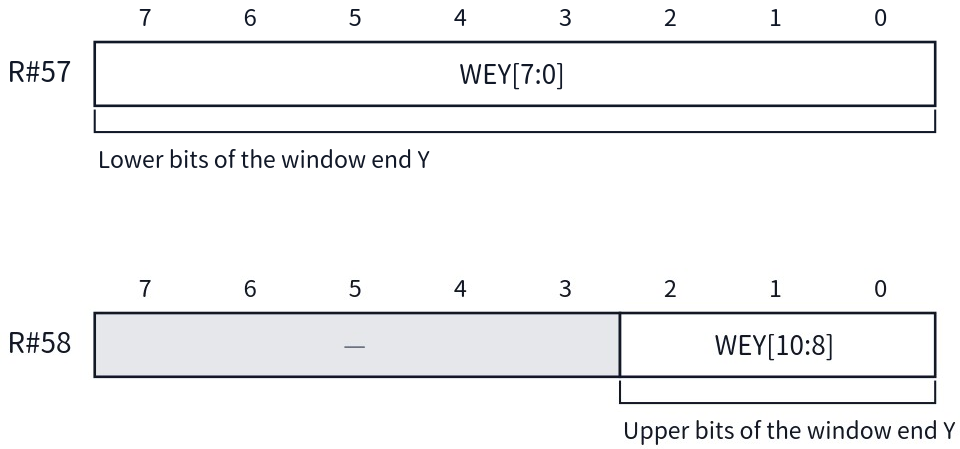

- Reset value: 0

## R#55 Window End X (L) / R#56 Window End X (H) [New in V9968]




- Reset value: 1FFh for WEX as a whole

## R#57 Window End Y (L) / R#58 Window End Y (H) [New in V9968]




- Reset value: 7FFh for WEY as a whole

The window registers (R#51–R#58) specify the start point and end point of the LRMM source reference window. These registers are effective only for LRMM and have no effect on other commands. The window is the range including both the start point and the end point; dots whose reference point lies inside (WSX, WSY)–(WEX, WEY) are read from the transfer source, and dots that fall outside are replaced with the color of R#44 (CLR). Because the reset values of the end points are the maximum values (WEX=1FFh, WEY=7FFh), immediately after reset the entire area is inside the window.

## Unimplemented Registers (R#59–R#63)

R#59–R#63 are not implemented. Writes to these numbers are ignored.

# Chapter 4 Status Registers in Detail

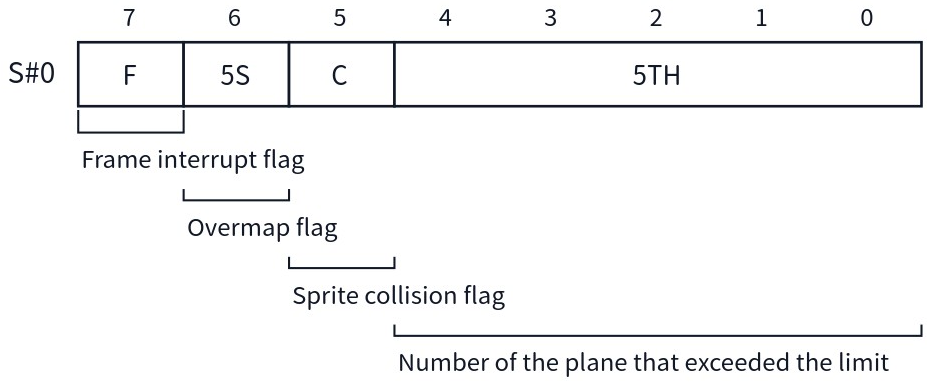
This chapter describes the status registers S#0–S#9 of the V9968 in detail, one register at a time.

All status registers are **read-only**. A register is read by specifying its number in R#15 (Status Register Pointer) and then reading Port#1 (see Chapter 2 and Chapter 3). At reset, the pointer points to 0 (S#0).

Some registers have their **flags cleared when they are read**. Each register's section states this explicitly as a "side effect on read", so take care when monitoring the state by polling. Bits marked as "fixed" in the tables in this chapter are always read as that value.

If S#10–S#15 is specified and read, FFh is always returned (see the final section).

### S#0




Side effect on read: **F, C, and 5S are cleared** (the value of 5TH is retained).

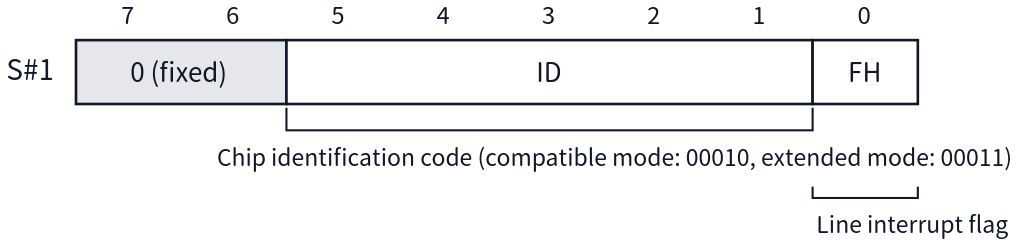
5S and 5TH are the mechanism that reports that the limit on the number of sprites that can be displayed on one line has been exceeded. The limit, and the count at which 5S is set, differ depending on the sprite mode.

| Condition | Limit per line | Count at which 5S is set |
|---|---|---|
| Sprite mode1 | 4 | The 5th sprite |
| Sprite mode2 | 8 | The 9th sprite |

| Condition | Limit per line | Count at which 5S is set |
|---|---|---|
| Sprite mode3, or mode1 / 2 when S16=1 | 16 | The 17th sprite |

5TH contains the number of the plane that exceeded the limit. Because 5TH is 5 bits, if the plane number is 32 or greater, 1Fh is stored.

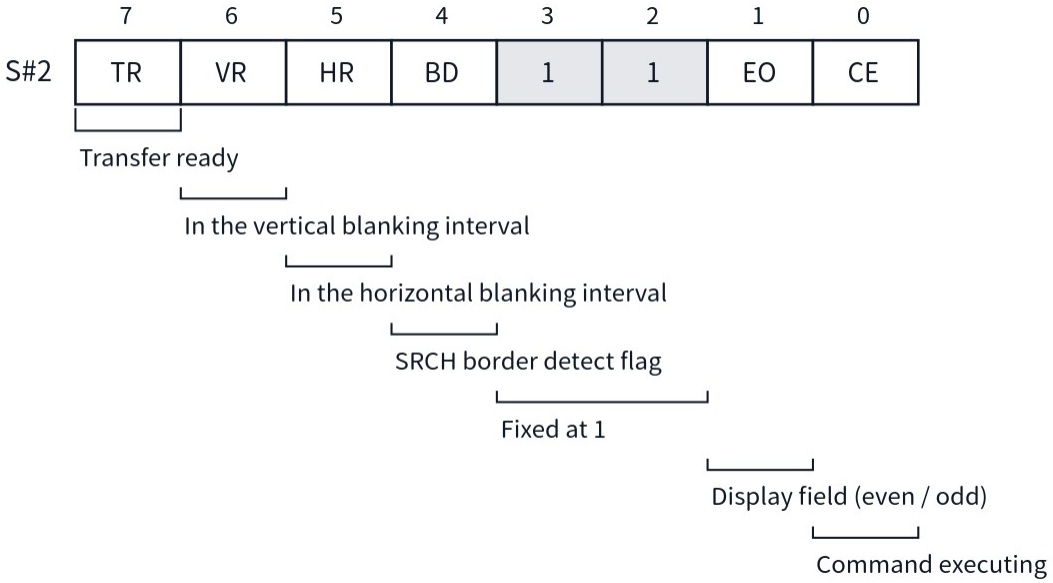
## S#1




The value of ID is `00010` in compatible mode (V58=1) and `00011` in extended mode (V58=0). This provides a means of determining whether the software is operating as the equivalent of a V9958, or in the extended mode of the V9968.

Side effect on read: **FH is cleared**.

## S#2

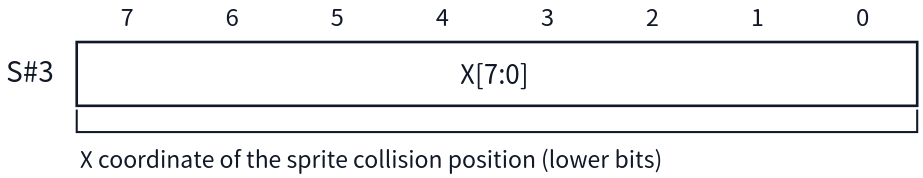



Side effect on read: None.

CE is 1 while a VDP command is executing. Waiting for command completion can be done by polling CE, or can be reported by an interrupt by using CEIE of R#20.

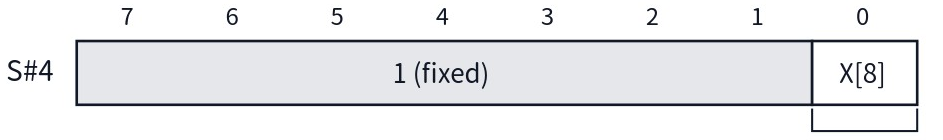
Bit3 and bit2 are always read as 1. Earlier documents did not describe these two bits, but unused bits are not necessarily 0, so software must not use their values in any decision.

### S#3

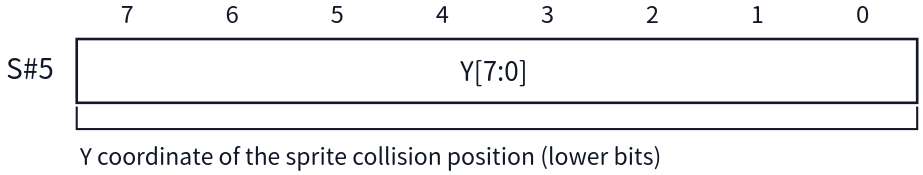



This is the lower 8 bits of the X coordinate of the position where sprites collided with each other. The upper bits are in S#4. For the clearing of the coordinates, see the section on S#5.

### S#4



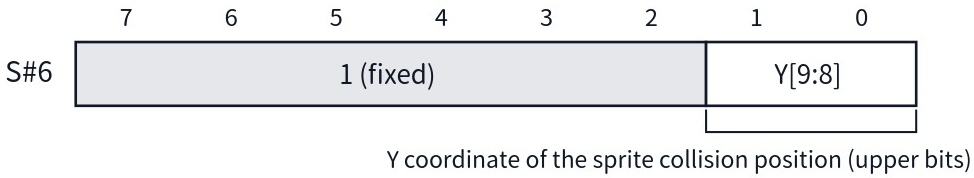

### S#5



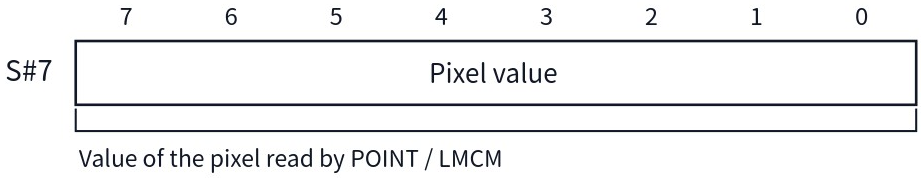

This is the lower 8 bits of the Y coordinate of the position where sprites collided with each other. The upper bits are in S#6.

Side effect on read: **The collision coordinates (both X and Y) are cleared**. When reading the collision coordinates, read S#3 / S#4 first, and read S#5 last.

### S#6



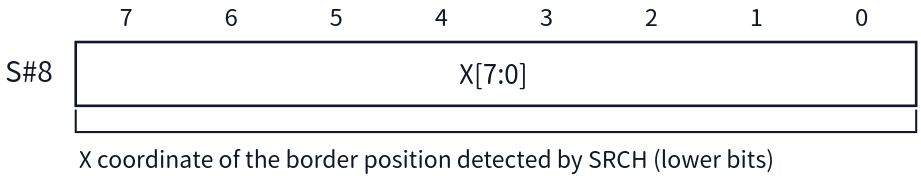

### S#7




A pixel value read by a POINT command or an LMCM command is stored in R#44, and is then read via this register, S#7.

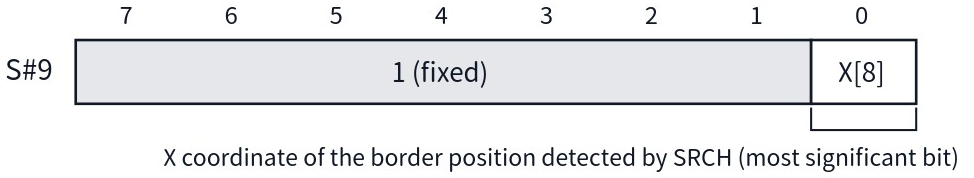
Reading this register is used as the "read complete" signal for the transfer control (TR control) on the command engine side. In consecutive reads with LMCM, reading S#7 is the condition for advancing to the next pixel.

### S#8




The upper bits are in S#9. For the clearing of the BD flag, see the section on S#9.

### S#9




Side effect on read: **BD (S#2 bit4) is cleared**.

### S#10–S#15

S#10–S#15 are not implemented. If these numbers are specified in R#15 and Port#1 is read, **FFh** is always returned.

## Relationship to the Interrupt Output

Finally, the relationship between the status flags and the interrupt output is summarized here. The interrupt output of the V9968 is the OR of the following three conditions (negative logic).

- The line interrupt flag (FH of S#1) is set, and IE1 of R#0 is 1
- The frame interrupt flag (F of S#0) is set, and IE0 of R#1 is 1
- A command completion interrupt source is present, and CEIE of R#20 is 1

The occurrence state of each source can also be checked by reading Port#4 (see Chapter 2). To use the command completion interrupt, because CEIE is in R#20, LockExtReg must be released at Port#4 before R#20 is set.

# Chapter 5 Screen Modes

## 5.1 About the Screen Modes

The screen modes of the V9968 are equivalent to those of the V9958, and no new screen modes have been added. All 12 modes can be used as they are, from the text modes and pattern (character) based modes inherited from the TMS9918, through the bitmap modes added in the V9938, to the YJK based modes added in the V9958.

This chapter describes the list of modes and how to select them, and the table layout in VRAM for each mode. The description of each mode assumes the operation of the earlier V9938/V9958, but where the detailed behavior was not covered in detail in the earlier documentation (the V9938/V9958 technical data books) and the operation of the V9968 has been established, this manual states it explicitly. In addition, behavior that was undefined in the V9958 but is actually usable is described here as an official function of the V9968, upgraded from undefined behavior. Such items are marked "**9958 Compatible**".

Note that what this chapter calls the "palette" is the 256-entry palette described in Chapter 7 (5 bits for each of the R, G and B components of each color). The color numbers specified in the earlier modes refer to entries of this palette. For the initial palette values immediately after reset, see Chapter 7.

## 5.2 Selecting a Mode

A screen mode is selected by the combination of a total of 5 bits: bits 3–1 (M5, M4, M3) of control register R#0, and bit 4 (M1) and bit 3 (M2) of R#1. The correspondence between the combinations and the modes is as follows.

| Mode | Abbreviation | M5 | M4 | M3 | M2 | M1 | Corresponding BASIC SCREEN | Screen | Colors | Sprites |
|---|---|---|---|---|---|---|---|---|---|---|
| Graphic1 | G1 | 0 | 0 | 0 | 0 | 0 | SCREEN 1 | 32×24 patterns | Palette, 16 colors | Mode 1 |
| Text1 | T1 | 0 | 0 | 0 | 0 | 1 | SCREEN 0 (WIDTH 40) | 40×24 characters | 2 colors | None |

| Mode | Abbreviation | M5 | M4 | M3 | M2 | M1 | Corresponding BASIC SCREEN | Screen | Colors | Sprites |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | (6×8 dots) | | |
| Multicolor | MC | 0 | 0 | 0 | 1 | 0 | SCREEN 3 | 64×48 blocks | Palette, 16 colors | Mode 1 |
| Graphic2 | G2 | 0 | 0 | 1 | 0 | 0 | SCREEN 2 | 256×192 dots | Palette, 16 colors | Mode 1 |
| Graphic3 | G3 | 0 | 1 | 0 | 0 | 0 | SCREEN 4 | 256×192 dots | Palette, 16 colors | Mode 2 |
| Graphic4 | G4 | 0 | 1 | 1 | 0 | 0 | SCREEN 5 | 256×212 dots | Palette, 16 colors | Mode 2 |
| Text2 | T2 | 0 | 1 | 0 | 0 | 1 | SCREEN 0 (WIDTH 80) | 80×24 characters (6×8 dots) | 2 colors + blink colors | None |
| Graphic5 | G5 | 1 | 0 | 0 | 0 | 0 | SCREEN 6 | 512×212 dots | Palette, 4 colors | Mode 2 |
| Graphic6 | G6 | 1 | 0 | 1 | 0 | 0 | SCREEN 7 | 512×212 dots | Palette, 16 colors | Mode 2 |
| Graphic7 | G7 | 1 | 1 | 1 | 0 | 0 | SCREEN 8 | 256×212 dots | 256 colors (direct specification) | Mode 2 |
| Graphic7 -YAE | G7-YAE | 1 | 1 | 1 | 0 | 0 | SCREEN 10/11 | 256×212 dots | YJK + palette, 16 colors | Mode 2 |
| Graphic7 -YJK | G7-YJK | 1 | 1 | 1 | 0 | 0 | SCREEN 12 | 256×212 dots | YJK (approx. 19,268 colors) | Mode 2 |

YAE and YJK, the variants of Graphic7, are switched not by the mode bits but by bit 3 (YJK) and bit 4 (YAE)

of R#25 (Sections 5.13 and 5.14).

**9958 Compatible (the M3 bit in Text1 / Multicolor)** In Text1 and Multicolor, M3 (bit 1 of R#0) is a don't-care. That is, Text1 can be selected with {M5,M4,M3,M2,M1} = 00001 or 00101, and Multicolor with 00010 or 00110. This behavior was not guaranteed on the V9958, but on the V9968 it is an official specification.

In addition, the following settings are used in combination with the modes. For details, see Chapter 3.

- Number of display lines: Bit 7 (LN) of R#9 switches the vertical display of the bitmap modes between 192 lines (LN=0) and 212 lines (LN=1), and the number of rows of Text2 between 24 rows and 26.5 rows. Unless otherwise noted, the memory sizes in this chapter are for 212 lines (24 rows for Text2).
- Display position adjustment: The horizontal scroll (R#26, R#27) and the display start line (R#23) change the display position of the screen; the table layout in VRAM does not change.
- V58 bit (bit 0 of R#21): Immediately after reset this is 1, and the chip operates in V9958 compatible mode. Setting it to 0 selects the extended mode, in which the full 256KB of VRAM can be used. The table layouts themselves in this chapter are common to both modes, but the bits that can be used for the base addresses change (Section 5.15).

## Table Base Address Registers

The start address of each table is specified by the following registers (for the sprite related tables, see Chapter 6). Only the upper bits of the address can be written to a register; the lower bits are treated as 0. The positions at which a table can be placed are therefore limited to boundaries determined by each register.

| Register | Table | Address bits supplied |
|---|---|---|
| R#2 | Pattern name table / bitmap | A17–A10 |
| R#3 | Color table / blink table (lower) | A13–A6 |
| R#10 | Color table / blink table (upper) | A17–A14 |
| R#4 | Pattern generator table | A17–A11 |

| Register | Table | Address bits supplied |
|---|---|---|
| | | |
| R#5, R#11 | Sprite attribute table (Chapter 6) | Equivalent to A17–A7 |
| R#6 | Sprite pattern generator table (Chapter 6) | A17–A11 |

The start address of the color table and the blink table is specified by combining R#3 and R#10, as A17–A6 (in 64-byte units).

## 5.3 Text1 Mode

This is a text mode of 40×24 characters. Characters are 6×8 dots, and 256 kinds of characters can be displayed. Sprites cannot be used in this mode. In MSX-BASIC it corresponds to SCREEN 0 (WIDTH 40).

| Item | Description |
|---|---|
| Screen | 40 characters × 24 rows |
| Characters | 256 kinds, 6×8 dots, font definable for each character |
| Colors | 2 colors common to all characters (foreground color / background color) |
| Memory required | Pattern generator table 2,048 bytes, pattern name table 960 bytes |

### Pattern Name Table (R#2)

One byte corresponds to one character on the screen, and its value is the character code (0–255) as it is. The address is the base address plus "row×40 + column".

The start address can be specified in 1KB units with R#2.

| R#2 value | 00h | 01h | 02h | … | 7Fh | 80h | … | FEh | FFh |
|---|---|---|---|---|---|---|---|---|---|
| Start address | 00000h | 00400h | 00800h | … | 1FC00h | 20000h | … | 3F800h | 3FC00h |

Start address = R#2 value × 400h. The MSX-BASIC initial value is 00000h.

### Pattern Generator Table (R#4)

This table defines the character fonts. It uses 8 bytes per character, arranged in character code order from the start. Of each byte, the upper 6 bits are displayed on the screen; the lower 2 bits are not displayed (because the character width is 6 dots).

Address = base address + character code×8 + line (0–7)

The start address can be specified in 2KB units with R#4.

| R#4 value | 00h | 01h | 02h | … | 7Eh | 7Fh | |---|---|---|---|---|---|---| | Start address | 00000h | 00800h | 01000h | … | 3F000h | 3F800h |

Start address = R#4 value × 800h. The MSX-BASIC initial value is 00800h.

### Specifying Colors

Colors are specified with R#7, and are common to all characters. The upper 4 bits give the foreground color (the "1" parts of the font pattern) and the lower 4 bits give the background color (the "0" parts); each is a color number in the palette. The border color is the same as the background color.

### VRAM Address Wraparound

Care is required with address wraparound when the CPU accesses VRAM in Text1 mode (and also in Multicolor, Graphic1 and Graphic2). After 3FFFh, the lower 14 bits of the address register return to 0000h (the carry from bit 13 to bit 14 is blocked; this is a specification for maintaining compatibility with the TMS9918). If, on the other hand, the upper address is specified with R#14, the range 04000h–3FFFFh can also be accessed, but in that case as well a wraparound remains, in which after 7FFFh the address returns to 4000h. For details of VRAM access, see Chapter 11.

## 5.4 Text2 Mode

This is a text mode of 80×24 characters. Characters are 6×8 dots, and each character can be made to blink. Sprites cannot be used in this mode either. In MSX-BASIC it corresponds to SCREEN 0 (WIDTH 80).

| Item | Description |
|---|---|
| Screen | 80 characters × 24 rows (26.5 rows with LN=1 in R#9) |
| Characters | 256 kinds, 6×8 dots, font definable for each character, blinking possible for each character |
| Colors | 2 colors (foreground color / background color) + 2 colors when blinking |
| Memory required | Pattern generator table 2,048 bytes, pattern name table 1,920 bytes, blink table 240 bytes (at 26.5 rows, 2,160 bytes and 270 bytes, respectively) |

### Pattern Name Table (R#2)

As in Text1, one byte (a character code) corresponds to one character. The address is the base address plus "row×80 + column". The start address can be specified in 4KB units with R#2 (of address = R#2 value × 400h, the bits that are actually effective are A17–A12).

### Pattern Generator Table (R#4)

This has exactly the same structure and function as in Text1. See Section 5.3.

### Blink Table (R#3, R#10)

This table records which characters are to blink; one bit corresponds to one character on the screen. A character whose corresponding bit is "1" is given the blink attribute. Bits are assigned to characters in order from the start of the pattern name table, starting from the upper bit of each byte.

Address = base address + (row×80 + column) ÷ 8

The start address can be specified in 512-byte units with R#3 and R#10.

### Specifying Colors and Blink

The normal colors are specified with R#7, as in Text1 (upper 4 bits = foreground color, lower 4 bits = background color). A character given the blink attribute is displayed by repeatedly alternating between the color of R#7 and the blink color specified with R#12 (likewise, upper = foreground, lower = background). The period of the repetition is set with R#13: the upper 4 bits give the time for which the color of R#7 is displayed, and the lower 4 bits give the time for which the blink color is displayed, each in

units of 1/6 second. Setting either one to 0 means that color is not displayed during that period (for details, see Chapter 3).

## 5.5 Multicolor Mode

This is a low-resolution graphic mode in which the area of one character (8×8 dots) is divided into four 4×4 dot blocks, and a color is specified for each block. The screen consists of 64×48 blocks. In MSX-BASIC it corresponds to SCREEN 3, and sprite mode 1 can be used.

| Item | Description |
|---|---|
| Screen | 64×48 blocks (each block is 4×4 dots) |
| Colors | Palette, 16 colors, specifiable for each block |
| Memory required | Pattern generator table (color data) 2,048 bytes, pattern name table 768 bytes |

### Pattern Name Table (R#2)

This is a table of 768 bytes, with one byte corresponding to each pattern position on the screen (32×24). The start address can be specified in 1KB units with R#2.

### Pattern Generator Table (R#4)

In this mode, the pattern generator table holds the color data as it is. One pattern is 8 bytes, and each pair of 2 bytes represents the colors of the four blocks: top, bottom, left and right. The upper 4 bits of each byte give the color of the left block, and the lower 4 bits give the color of the right block (a palette number).

The byte displayed is determined by the display line. The byte position within the pattern is "the group number for every 4 lines, when the display line is viewed in a 32-line cycle" (0–7). In other words, even for the same pattern name, a different pair of 2 bytes is displayed depending on the vertical position on the screen (in the 32-line cycle).

Address = base address + pattern number×8 + byte position (0–7)

The start address can be specified in 2KB units with R#4. The border color is specified with the lower 4

bits of R#7.

## 5.6 Graphic1 Mode

This is a character graphic mode in which 8×8 dot patterns are arranged in a 32×24 array. 256 kinds of patterns can be displayed, and colors are specified for each group of 8 patterns. In MSX-BASIC it corresponds to SCREEN 1, and sprite mode 1 can be used.

| Item | Description |
|---|---|
| Screen | 32×24 patterns (256×192 dots) |
| Patterns | 256 kinds, 8×8 dots, figure definable for each pattern |
| Colors | Palette, 16 colors, specifiable for each group of 8 patterns |
| Memory required | Pattern generator table 2,048 bytes, color table 32 bytes, pattern name table 768 bytes |

### Pattern Name Table (R#2)

This is a table of 768 bytes, corresponding one-to-one with the pattern positions on the screen; its values are pattern numbers (0–255). The start address can be specified in 1KB units with R#2.

### Pattern Generator Table (R#4)

Each pattern uses 8 bytes, and each byte corresponds to one horizontal line of the pattern; dots that are "1" are displayed in the foreground color and dots that are "0" in the background color.

Address = base address + pattern number×8 + line (0–7)

The start address can be specified in 2KB units with R#4.

### Color Table (R#3, R#10)

One byte is used for each group of 8 patterns; the upper 4 bits specify the foreground color of that pattern group, and the lower 4 bits specify the background color.

Address = base address + pattern number ÷ 8

The start address can be specified in 64-byte units with R#3 and R#10. The border color is specified with the lower 4 bits of R#7.

## 5.7 Graphic2 Mode

This is a mode in which the pattern generator table and the color table of Graphic1 are extended to three sets, for the top, middle and bottom thirds of the screen. Because a separate set of 256 patterns can be used in each third, 768 kinds of patterns can effectively be displayed, and the mode is used for pseudo 256×192 dot graphic display. Up to 2 colors can be specified for each horizontal line (8 dots) of a pattern. In MSX-BASIC it corresponds to SCREEN 2, and sprite mode 1 can be used.

| Item | Description |
|---|---|
| Screen | 32×24 patterns (256×192 dots), 3-bank configuration |
| Patterns | Effectively 768 kinds, 8×8 dots |
| Colors | Palette, 16 colors, up to 2 colors for each horizontal line |
| Memory required | Pattern generator table 6,144 bytes, color table 6,144 bytes, pattern name table 768 bytes |

### Pattern Name Table (R#2)

This is a table of 768 bytes, with the same layout as Graphic1. Each row group — the top third (rows 0–7), the middle third (rows 8–15) and the bottom third (rows 16–23) — refers to the patterns of the corresponding bank. The start address can be specified in 1KB units with R#2.

### Pattern Generator Table (R#4)

This is 6,144 bytes, for the three banks.

Address = base address + bank×2,048 + pattern number×8 + line (0–7)

The bank is determined by the group of display rows (row ÷ 8), but an AND is taken with the bits corresponding to A12 and A11 of the base address. Therefore, if the corresponding bit of R#4 is set to 0, that bank refers to the same pattern set as the other banks. The start address can be specified in 8KB

units with R#4.

### Color Table (R#3, R#10)

This has the same size as the pattern generator table (6,144 bytes), and specifies, for each horizontal line of a pattern, the upper 4 bits = foreground color and the lower 4 bits = background color.

Address = base address + bank×2,048 + pattern number×8 + line (0–7)

On the color table side as well, a mask is applied by AND with the corresponding bits of the base address, for bank selection and pattern group selection. Setting a bit on the base side to 0 allows colors to be shared over the corresponding range of patterns. The start address can be specified in 8KB units with R#3 and R#10. The border color is specified with the lower 4 bits of R#7.

## 5.8 Graphic3 Mode

The table layout in VRAM is exactly the same as Graphic2. The only difference is that the sprites are in mode 2. In MSX-BASIC it corresponds to SCREEN 4. For the layout, see Section 5.7.

## 5.9 Graphic4 Mode

This is a bitmap mode of 256×212 dots. One byte corresponds to 2 dots, and a color can be specified for each dot from the palette of 16 colors. VDP commands can be used. In MSX-BASIC it corresponds to SCREEN 5, and sprite mode 2 can be used.

| Item | Description |
|---|---|
| Screen | 256×212 dots (192 lines with LN=0) |
| Colors | Palette, 16 colors, specifiable for each dot |
| Memory required | 24,576 bytes (6000h, at 192 lines) / 27,136 bytes (6A00h, at 212 lines) |

### Bitmap Layout (R#2)

The address of dot (X, Y) is obtained from the following formula.

Address = base address + Y×128 + X÷2

For a dot with an even X, the upper 4 bits of the byte give the color (palette number); for a dot with an odd X, the lower 4 bits do. The size of one page is 32,768 bytes (8000h), and the base address can be specified in 32KB units with R#2. In compatible mode, four locations can be selected: 00000h, 08000h, 10000h and 18000h; in extended mode, A17 is also effective, and eight locations can be selected, adding 20000h–38000h. The border color is specified with the lower 4 bits of R#7.

## 5.10 Graphic5 Mode

This is a bitmap mode of 512×212 dots. One byte corresponds to 4 dots, and a color can be specified for each dot from a palette of 4 colors. In MSX-BASIC it corresponds to SCREEN 6, sprite mode 2 can be used, and VDP commands can also be used.

| Item | Description |
|---|---|
| Screen | 512×212 dots (192 lines with LN=0) |
| Colors | Palette, 4 colors, specifiable for each dot |
| Memory required | 24,576 bytes (6000h, at 192 lines) / 27,136 bytes (6A00h, at 212 lines) |

### Bitmap Layout (R#2)

Address = base address + Y×128 + X÷4

The 4 dots within one byte correspond, in the order of the remainder of X÷4 being 0, 1, 2 and 3, to the 2 bits at bits 7–6, 5–4, 3–2 and 1–0, respectively. The method of specifying the base address is the same as in Graphic4.

Note that in this mode, tiling is applied to the color specification of the border color and the sprites. Of the 4 bits of a color specification, the upper 2 bits give the color of the dots with even X, and the lower 2 bits give the color of the dots with odd X (the border color specification by R#7 is also in this format).

## 5.11 Graphic6 Mode

This is a bitmap mode of 512×212 dots. One byte corresponds to 2 dots, and 16 palette colors can be specified for each dot. In MSX-BASIC it corresponds to SCREEN 7, sprite mode 2 can be used, and VDP commands can also be used.

| Item | Description |
|---|---|
| Screen | 512×212 dots (192 lines with LN=0) |
| Colors | Palette, 16 colors, specifiable for each dot |
| Memory required | 49,152 bytes (0C000h, at 192 lines) / 54,272 bytes (0D400h, at 212 lines) |

### Bitmap Layout (R#2)

Address = base address + Y×256 + X÷2

For a dot with an even X, the upper 4 bits of the byte give the color; for a dot with an odd X, the lower 4 bits do. One screen fits within a 64KB (10000h) range, and the base address is selected with R#2 from 00000h or 10000h (in extended mode, 20000h and 30000h are also possible). The border color is specified with the lower 4 bits of R#7.

## 5.12 Graphic7 Mode

This is a bitmap mode of 256×212 dots. One byte corresponds directly to one dot, and 256 colors are specified directly. The palette is not involved. In MSX-BASIC it corresponds to SCREEN 8, sprite mode 2 can be used, and VDP commands can also be used.

| Item | Description |
|---|---|
| Screen | 256×212 dots (192 lines with LN=0) |
| Colors | 256 colors (direct RGB specification) |
| Memory required | 49,152 bytes (0C000h, at 192 lines) / 54,272 bytes (0D400h, at 212 lines) |

## Bitmap Layout (R#2)

Address = base address + Y×256 + X

The breakdown of one byte is: bits 7–5 are the luminance information for green (3 bits), bits 4–2 for red (3 bits), and bits 1–0 for blue (2 bits). The method of specifying the base address is the same as in Graphic6.

Note that, because Graphic7 is a mode of SCREEN 7 and above, the physical layout of VRAM is interleaved (even addresses in DRAM0, odd addresses in DRAM1). However, when sprite mode 3 is used, the layout returns to flat. For details of the physical layout, and for the caution that switching changes the arrangement of the data, see Section 11.1. The same applies to SCREEN 10–12 (Sections 5.13 and 5.14).

[TBC] The interpretation of R#7 (border color) in Graphic7 mode. Earlier documentation states that the border color is specified with R#7, as in the other modes, but whether in this mode all 8 bits of R#7 are interpreted in GRB format, or only the lower 4 bits are used, could not be determined from the manuscripts and materials.

## Sample Program

The sample program `g7_demo.asm` is an example that displays all 256 colors of Graphic7. By dividing the screen into 8×8 blocks, varying green (G) with the block column, red (R) with the block row, and blue (B) with the four stripes within a block, all 256 color combinations can be viewed at a glance. Figure 5.1 shows the execution result.

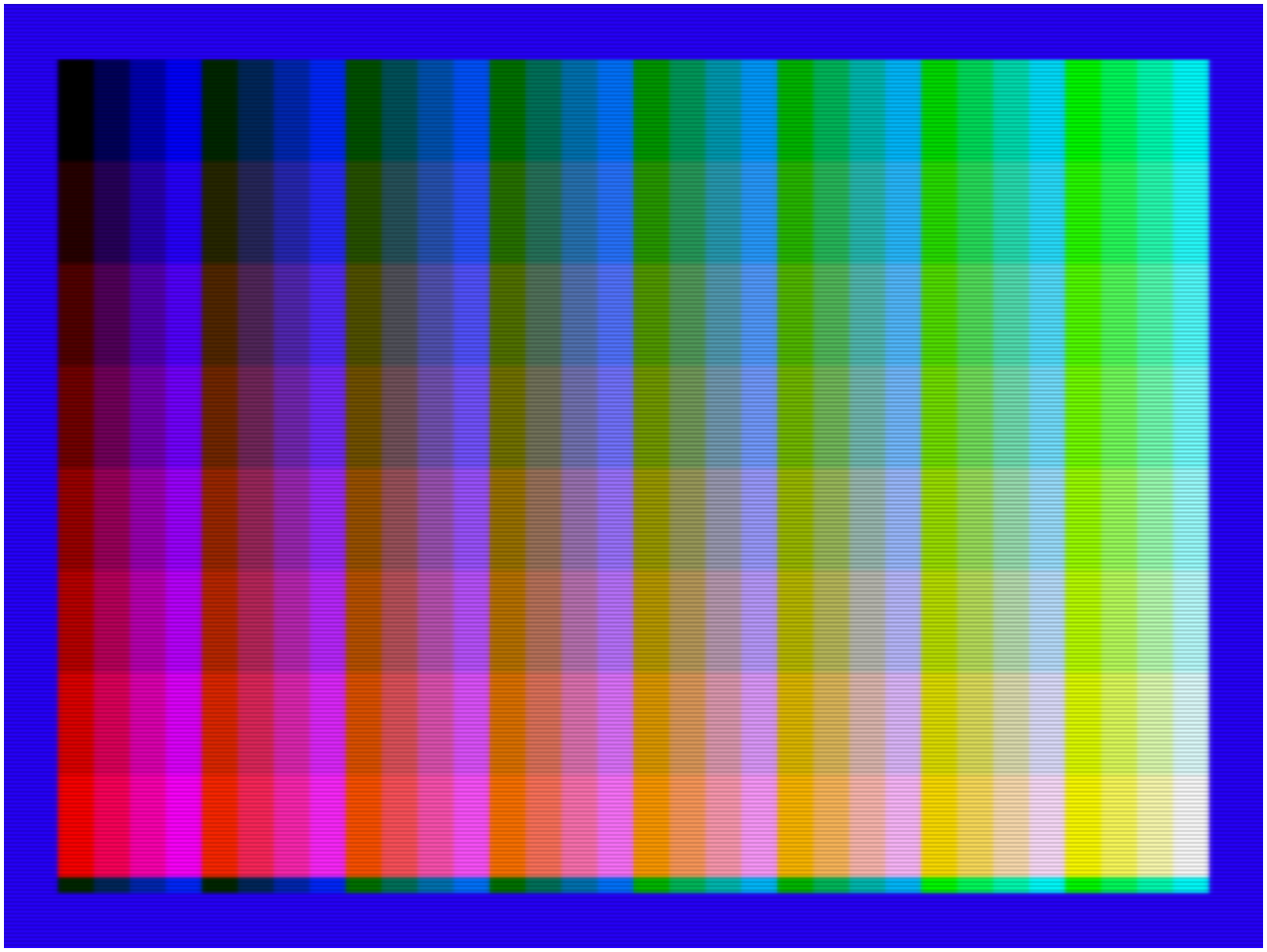


Figure 5.1 Execution result of g7_demo.asm (the 256 colors of Graphic7)

In Graphic7, one byte corresponds directly to one dot, so an image can be displayed simply by setting the write address to the start and sequentially writing 54,272 bytes (at 212 lines) from the CPU to Port#0 (98h). The palette is not involved. The writing of one line is as follows: it composes a color byte from the block column (G), the block row (R) and the stripe within the block (B), and writes runs in which the same color continues together.

```
draw_line::
        ld      c, 0            ; C = G (block column, 0..7)
g_loop:
        ld      l, 0            ; L = B (stripe, 0..3)
b_loop:
        ; color byte = (G << 5) | (R << 2) | B
        ld      a, c
        sla     a
        sla     a
        sla     a
```

```
        sla     a
        sla     a           ; A = G << 5
        ld      h, a
        ld      a, e
        sla     a
        sla     a           ; A = R << 2
        or      a, h
        or      a, l            ; A = the dot's color byte
        ld      h, 8            ; one stripe is 8 dots wide
px_loop:
        out     [0x98], a       ; write one dot (address auto-increments)
        dec     h
        jr      nz, px_loop
        inc     l
        ld      a, l
        cp      a, 4
        jr      nz, b_loop
        inc     c
        ld      a, c
        cp      a, 8
        jr      nz, g_loop
        ret
```

The block row advances R every 26 lines, and draws 208 lines in 8 block rows. The remaining 4 lines (lines 208–211) are drawn with R = 7 unchanged, so a thin stripe can be seen at the bottom edge of Figure 5.1 (for the calling loop, see the source file).

## 5.13 Graphic7-YAE Mode (SCREEN 10/11)

This is a variant of Graphic7 in which natural image display in the YJK system and display with the 16 palette colors can be mixed, dot by dot. In MSX-BASIC it corresponds to SCREEN 10 and SCREEN 11. The two have the same display method; the difference is whether the VDP commands interpret the data as RGB (palette) or as YJK (Chapter 8).

To set this mode, in addition to the Graphic7 mode bits, set both bit 3 (YJK) and bit 4 (YAE) of R#25 to 1. Note that the combination YJK=0, YAE=1 is a setting that is not defined by the specification, so do not

use it.

The layout in VRAM is exactly the same as Graphic7 (address = base address + Y×256 + X). What differs is the interpretation of one byte: four horizontally adjacent dots are treated as one group.

| Byte | Bits 7–4 | Bit 3 | Bits 2–0 |
|---|---|---|---|
| 1st dot | Y or palette number | A | Lower 3 bits of K |
| 2nd dot | Y or palette number | A | Upper 3 bits of K |
| 3rd dot | Y or palette number | A | Lower 3 bits of J |
| 4th dot | Y or palette number | A | Upper 3 bits of J |

- A dot whose bit 3 (A) is 0 is displayed in the YJK system. In this case, bits 7–4 are the upper 4 bits of the luminance Y, and Y takes 16 steps of even values (0, 2, 4, ⋯, 30).
- A dot whose bit 3 (A) is 1 has its bits 7–4 interpreted as a palette number (0–15), and is displayed in the palette color.

The meaning of J and K, and the conversion to RGB, are the same as in Section 5.14.

## 5.14 Graphic7-YJK Mode (SCREEN 12)

This is a variant of Graphic7, a natural image mode in which colors are represented in the YJK system. By giving each dot its own luminance component Y, and sharing the color difference components J and K among 4 horizontal dots, simultaneous display of approximately 19,268 colors is achieved with a limited number of bits. In MSX-BASIC it corresponds to SCREEN 12.

To set this mode, in addition to the Graphic7 mode bits, set bit 3 (YJK) of R#25 to 1 and bit 4 (YAE) to 0. The layout in VRAM is exactly the same as Graphic7.

The breakdown of each byte of a 4-dot group is as follows.

| Byte | Bits 7–3 | Bits 2–0 |
|---|---|---|
| 1st dot | Y | Lower 3 bits of K |

| Byte | Bits 7–3 | Bits 2–0 |
|---|---|---|
| | | |
| 2nd dot | Y | Upper 3 bits of K |
| 3rd dot | Y | Lower 3 bits of J |
| 4th dot | Y | Upper 3 bits of J |

- Y is a 5-bit unsigned value (0–31), and can be specified for each dot.
- J and K are each 6-bit two's complement values (−32–31), shared among 4 dots. J represents the red color difference, and K the green color difference.
- Conversion to RGB is performed by R = Y + J, G = Y + K, B = 5/4×Y − J/2 − K/4, and the results are rounded into the range 0–31 (the internal color signals are 5 bits per component).

## 5.15 Extensions and Differences in the V9968

Finally, the position of the V9968 with respect to the screen modes is summarized.

- **There are no new screen modes.** In all 12 modes, assets up to the V9958 operate as they are.
- **Formalization of the M3 don't-care (9958 Compatible)**: In Text1 and Multicolor, the mode can be selected regardless of the value of M3 (Section 5.2).
- **256KB VRAM (extended mode)**: In extended mode, with the V58 bit (bit 0 of R#21) set to 0, the A17 written to R#2, R#4, R#6, R#10 and R#11 is effective as it is. In compatible mode (V58=1), the A17 written to these registers is held as 0. As a result, the number of pages of the bitmap modes increases (8 pages in Graphic4/5, and 4 base locations can be selected in Graphic6/7). Note that writing 1 to R#21 returns the A17 of the base addresses of the above five tables all to 0.
- **VRAM address wraparound**: The width of the address wraparound when the CPU accesses VRAM differs by mode: 14 bits, 17 bits or 18 bits. In modes with {M5,M4}=00, such as Text1, wraparound is at 14 bits. For details, see Chapter 11.

- **Flat Interlace (FIL)**: In the conventional interlaced display, even if page0 and page1 (or page2 and page3) were displayed alternately, shifted by half a dot in the vertical direction, the two pages remained separate screens, and a line drawn across the pages did not connect. When FIL

(bit 5 of R#20) is set to 1, page0 and page1 can be displayed as a single vertically long screen, joined top to bottom in VRAM, and the VDP commands also treat it as a combined high-resolution screen (see Chapter 3 and Chapter 8).

- **The dot arrangement is the same as before**: The dot arrangement (bit assignment) of Graphic5/Graphic6/Graphic7 is the same as before (formulas 4.2–4.4 of the MSX2 Technical Handbook).

---

## Sources for This Chapter

- Original manuscript "V9968 Programmer's Manual (Screen Modes)": the chapter structure and the details of Text1, and the policy on compatibility.
- RTL verification notes (v9968_registers.md): the mode selection bits and the M3 don't-care, the bit assignments of the base address registers, and the operation of the A17 mask.
- MSX2 Technical Handbook (Technical Handbook wiki, "Chapter 3: MSX2 Screen Modes"): the specifications of the earlier modes (Text1 to Graphic7), the table layouts, and the formulas for obtaining an address from coordinates.
- For the YJK encoding format, content that agrees among the earlier materials (Grauw, "The YJK screen modes", and published materials of the MSX2+ generation) was adopted.

# Chapter 6 Sprites

Sprites are a function for displaying character patterns at arbitrary positions on the screen, independently of the background (the pattern plane). This chapter describes, for the three sprite modes of the V9968, the data layout in VRAM, the display priority, collision detection, and the limits of each mode.

For the bit definitions of the registers themselves, see Chapter 3; for details of the status registers that appear in this chapter (S#0, S#3–S#6), see Chapter 4.

## 6.1 Overview of the Sprite Modes

The V9968 has the following three sprite modes.

| Mode | Origin | Screen modes in which it can be used | Number of planes | Number that can be displayed on one line |
|---|---|---|---|---|
| Sprite Mode 1 | TMS9918 compatible | Graphic1, Graphic2, Multicolor | 32 planes (#0–#31) | 4 planes (16 planes when S16=1) |
| Sprite Mode 2 | Added in the V9938 | Graphic3–Graphic7 | 32 planes (#0–#31) | 8 planes (16 planes when S16=1) |
| Sprite Mode 3 | Added in the V9968 (new mode) | Graphic1–Graphic7, Multicolor | 64 planes (#0–#63) | 16 planes |

Mode 1 and Mode 2 are selected automatically according to the screen mode that is set (the same as on earlier chips). Sprites are not displayed in the text modes (Text1, Text2).

Mode 3 is a new mode that becomes effective, in place of Mode 1 / Mode 2, by setting the SP3 bit of R#20 to 1. It replaces the "two kinds of sprites determined by the screen mode" of the earlier modes, and can be used in all of Graphic1–Graphic7 and Multicolor.

The registers common to each mode are as follows.

| Register | Role |
|---|---|
| R#5, R#11 | Base address of the sprite attribute table |
| R#6 | Base address of the sprite pattern generator table |
| R#1 bit1 (SI), bit0 (MAG) | Sprite size (8×8 / 16×16) and enlargement specification for Mode 1 / 2 |
| R#8 bit1 (SPD) | When set to 1, disables sprite display as a whole (common to all modes) |
| R#20 bit3 (SP3) | When set to 1, Sprite Mode 3 becomes effective |
| R#20 bit7 (S16) | When set to 1, the display limit per line in Mode 1 / 2 becomes 16 planes (see Section 6.7) |
| R#20 bit1 (SVNS) | When set to 1, sprites are not affected by the vertical scroll (R#23) (see Section 6.2) |

## 6.2 Common Conventions

### Coordinates and Display Position

The Y coordinate of a sprite points to "one line above the line on which it is to be displayed". That is, relative to the value of Y written in the attribute, the sprite begins to be displayed from line Y+1. To display it on the topmost line of the screen (line 0), write Y = 255 (-1). This is the same convention as on earlier chips (TMS9918 / V9938).

The X coordinate is 0 at the left edge of the screen. In Mode 1 / 2, using the EC (Early Clock) bit can shift the display position 32 dots to the left, so that a display that gradually appears from the left edge of the screen is possible.

Note that in the high-resolution screen modes with 512 horizontal dots (Graphic5, Graphic6), one dot of a sprite corresponds to 2 horizontal dots of the screen, and the absolute size of the sprite is unchanged from the other modes. This, too, is the same as on earlier chips.

Normally, the display position of a sprite is affected by the vertical scroll (R#23) and scrolls together with the background. When the SVNS bit of R#20 is set to 1, sprites are no longer affected by the specification

of R#23, and the Y coordinate comes to point to an absolute position on the screen (with Y=0, the sprite is displayed at the very top of the screen). Use this when you want to scroll only the background and keep the sprites fixed on the screen (see the section on R#20 in Chapter 3).

### Priority

The smaller the plane number, the higher the priority. When several planes overlap on the same dot, in principle the dot of the plane with the smallest number is displayed (the exception due to the CC bit of Mode 2 is described in Section 6.4.3).

There is a limit, for each mode, on the number of planes that can be displayed on one line (see Table 6.1), and any excess is not displayed on that line. Because this judgment is made line by line, between sprites that are offset slightly from each other vertically, "only a part may disappear". Notification when the limit is exceeded (the 5S flag and the plane numbers of the fifth and subsequent planes) is described in Section 6.6.2.

### Terminating Display

Writing a specific value in the Y coordinate of an attribute terminates the scanning from that plane onward, and the subsequent planes are not displayed. The termination value differs by mode.

| Mode | Termination value |
|---|---|
| Mode 1 | Y = 208 (D0h) |
| Mode 2, Mode 3 | Y = 216 (D8h) |

To erase unneeded planes reliably, using this termination is reliable (the usage is the same as on earlier chips).

## 6.3 Sprite Mode 1

Mode 1 is the sprite of the TMS9918 itself. It is used in Graphic1, Graphic2, and Multicolor.

### 6.3.1 Attribute Table

There are 4 bytes per plane, arranged in plane-number order from the base address (R#5, R#11).

| Offset | Contents |
|---|---|
| +0 | Y coordinate |
| +1 | X coordinate |
| +2 | Pattern number (0–255) |
| +3 | bit7: EC (Early Clock) / bit3-0: color code |

The color code specifies only one color for the whole plane. The parts where the pattern bits are 1 are drawn in this color (for how the color is determined, see Chapter 7). Setting EC to 1 shifts the display position 32 dots to the left.

### 6.3.2 Pattern Table

The pattern is 1 bit per dot, and only the parts that are 1 have a color.

- 8×8 dots (SI=0): 8 bytes per pattern. Pattern numbers 0–255 are used as they are.
- 16×16 dots (SI=1): Four 8×8 patterns are combined and used (in the order upper left, upper right, lower left, lower right). The lower 2 bits of the specified pattern number are ignored, and the group of patterns with the number rounded down to a multiple of 4 is displayed.

Setting MAG=1 (R#1 bit0) displays each dot enlarged 2 times vertically and horizontally. The size and enlargement are settings common to all planes.

### 6.3.3 Display Limits

- Up to 32 planes can be displayed at the same time, and up to 4 planes per line in order of higher priority (up to 16 planes when S16=1. See Section 6.7).
- Planes exceeding the limit are not displayed on that line (see Section 6.6.2).

## 6.4 Sprite Mode 2

Mode 2 is the mode added in the V9938, and is used in Graphic3–Graphic7. The main differences from Mode 1 are the following three points: (1) a color can be specified for each line, (2) up to 8 planes can be displayed on one line (in the V9968 this can be extended to 16 planes by S16. See Section 6.7), and (3) the

coordinates at which a collision occurred can be known.

### 6.4.1 Attribute Table

As in Mode 1, there are 4 bytes per plane.

| Offset | Contents |
|---|---|
| +0 | Y coordinate |
| +1 | X coordinate |
| +2 | Pattern number (0–255) |
| +3 | Not used for color specification in Mode 2 (except for the extension in Section 6.4.4) |

The format of the pattern table and the handling of SI / MAG are the same as in Mode 1.

### 6.4.2 Sprite Color Table

The color information is placed in a sprite color table separate from the attribute table. The color table is placed automatically at the position determined by the same R#5, R#11 as the attribute table. Specifically, whereas the attribute table address uses the base down to bit9, in the color table bit9 is fixed to 0. For this reason, so that the two do not overlap, A9 of the base address must always be specified as 1 (this is the same point to note as on earlier chips).

The color table is 16 bytes per plane, and corresponds one byte to each line of the pattern (in the 8×8 case the first 8 bytes are used; in the 16×16 case all 16 bytes are used).

| Bit | Contents |
|---|---|
| bit7 | EC (Early Clock. Shifts only this line 32 dots to the left) |
| bit6 | CC (Composition of priority. See Section 6.4.3) |
| bit5 | IC (Lines that are 1 do not generate a collision. See Section 6.6.1) |
| bit3-0 | Color code (the color of the parts of this line's pattern that are 1) |

| Bit | Contents |
| --- | --- |
| | |

### 6.4.3 The CC Bit and OR Composition of Colors

The CC bit is a mechanism for composing the colors of overlapping sprites.

A plane with CC=1 does not, by itself, have a display priority, and is treated with the same priority as "the plane that has higher priority than itself, has CC=0, and is closest to itself". At a dot where such planes overlap, the color displayed is the logical sum (OR) of the color codes of each plane. When planes with CC=1 are chained together like beads, all of them are composed in a form hanging from the first CC=0 plane.

Using this mechanism, even in Mode 2 it is possible to "draw a multicolored character by overlapping several planes". Note that a dot overlapped by CC composition is not regarded as a collision.

### 6.4.4 V9968 Extension: Palette Sets with EPAL

In the extended palette mode (EPAL = 1 in R#20, see Chapter 7), the sprites of Mode 2 can also have a palette set (Chapter 7). The lower 4 bits of the fourth byte of the attribute (offset +3) are used for the palette set. When EPAL = 0, and also when this byte is 0 even if EPAL = 1, palette set 0 (palette numbers 0–15) is used, as before.

Note that in Graphic7 (SCREEN 8), unlike the V9938, the sprite colors also go through the palette. The same 256-color palette as the background is divided into groups of 16 colors each (palette sets), and the sprites are displayed with the 16 colors within the group. Because the first color of each group is treated as transparent, the colors that can actually be used are 15 (see Sections 7.4.3 / 7.4.4).

### 6.4.5 Display Limits

- Up to 32 planes can be displayed at the same time, and up to 8 planes per line in order of higher priority (up to 16 planes when S16=1. See Section 6.7).
- Planes exceeding the limit are not displayed on that line (see Section 6.6.2).

### 6.4.6 Sample Program

The sample program `spr_demo.asm` is an example that displays four 16×16-dot balls in SCREEN 5 (Graphic 4, Sprite Mode 2). Figure 6.1 shows the execution result.



Figure 6.1 Execution result of spr_demo.asm (Sprite Mode 2)

In the attribute table (7600h), the Y coordinate, X coordinate, and pattern number are arranged in the order in which they are displayed, and the list is finally terminated with an entry with Y = 216.

```
attribute_table:
        db      60, 40, 0, 0        ; plane 0: rainbow ball
        db      40, 110, 0, 0       ; plane 1: yellow ball
        db      90, 180, 0, 0       ; plane 2: light blue ball
        db      140, 70, 0, 0       ; plane 3: magenta ball
        db      216, 0, 0, 0        ; plane 4: Y=216 ends the list
```

Only for the ball at the left end (plane 0), a different color is set in all 16 bytes of the color table (7400h), making a striped pattern in which the color changes line by line. This is a feature of Mode 2: the color is

specified not per sprite but per line (→ Section 6.4.2). The remaining three planes have the same color on all lines (see the source file).

```
; plane 0: a different color on every line
db      0x06, 0x08, 0x08, 0x09, 0x09, 0x0b, 0x0b, 0x0a
db      0x0a, 0x0c, 0x02, 0x02, 0x03, 0x03, 0x05, 0x05
```

In the pattern table (7800h), 32 bytes of the ball picture are placed (for the data, see the source file). Note that, to make it 16×16 dots, the SI bit of R#1 is set to 1.

## 6.5 Sprite Mode 3 (New Mode in V9968)

Mode 3 is a sprite mode newly established in the V9968. It was designed to resolve the dissatisfactions with the earlier modes — that only one color can be used per plane (Mode 1) / per line (Mode 2), and that the CC composition for overdrawing is hard to handle — and has the following features.

- There are 64 planes (#0–#63), and up to 16 planes can be displayed on one line.
- The pattern has a 4-bit color code for each dot, and can be colored dot by dot.
- Enlargement / reduction (arbitrary magnification) can be performed for each plane.
- Horizontal flip and vertical flip can be performed for each plane.
- Translucency (blending with the background) can be performed for each plane.
- A palette set (Chapter 7) can be specified for each plane, and the whole 256-color palette can be used.

In Mode 3, SI / MAG (R#1) are not used. The size and magnification are specified for each plane in the attribute.

### 6.5.1 Attribute Table

There are 8 bytes per plane, arranged in plane-number order from the base address (R#5, R#11). Note that in Mode 3, too, address generation has the same mechanism as before, and bit8 and bit7 of the base address of the attribute table act as an AND mask with the upper bits of the plane number. Place the base address aligned to a 512-byte boundary.

| Offset | Contents |
|---|---|
| +0 | Y7–Y0 (the lower 8 bits of the Y coordinate) |
| +1 | bit7-6: SZ1, SZ0 (pattern size) / bit5-2: 0 (unused. Ignored even if written) / bit1-0: Y9, Y8 (the upper 2 bits of the Y coordinate. Y is 10 bits in total) |
| +2 | MGY7–MGY0 (the number of vertical display lines) |
| +3 | bit7-6: TP1, TP0 (translucency) / bit5: RVY (vertical flip) / bit4: RVX (horizontal flip) / bit3-0: PS3–PS0 (palette set) |
| +4 | X7–X0 (the lower 8 bits of the X coordinate) |
| +5 | bit7: 0 (unused) / bit6-4: PTS2, PTS1, PTS0 (the page number of the pattern sheet. One page is 256 lines = 32 KB) / bit3-2: 0 (unused) / bit1-0: X9, X8 (the upper 2 bits of the X coordinate. X is 10 bits in total) |
| +6 | MGX7–MGX0 (the number of horizontal display dots) |
| +7 | bit7-4: PY3–PY0 / bit3-0: PX3–PX0 (pattern number. PX and PY are tile coordinates on the pattern sheet, the upper nibble being PY and the lower nibble PX. See Section 6.5.2) |

The meaning of each field is as follows.

- **Y coordinate (10 bits)**: The convention for the display position is the same as before, and the sprite is displayed from line Y+1 for MGy lines. Writing Y = 216 terminates the scanning from that plane onward (see Section 6.2).
- **X coordinate (10 bits)**: 0 is the left edge of the screen. The sprite is displayed in the range of MGx dots from X. Giving X a large value (a value corresponding to a negative number when seen in 10 bits) allows it to be displayed with a part visible from the left edge.
- **SZ (pattern size)**: Determines the number of vertical dots of the pattern. The horizontal width of the pattern is always 16 dots.

| SZ | Pattern size (width × height in dots) | Bytes per pattern |
|---|---|---|
| 0 | 16×16 | 128 bytes |

| SZ | Pattern size (width × height in dots) | Bytes per pattern |
|---|---|---|
| | | |
| 1 | 16×32 | 256 bytes |
| 2 | 16×64 | 512 bytes |
| 3 | 16×128 | 1024 bytes |

- **MGx, MGy (display size)**: The number of dots (horizontal) and the number of lines (vertical) with which the pattern is displayed on the screen. The pattern is displayed enlarged / reduced to this rectangle. To display it at the same size, specify MGx = 16 and MGy = the number of vertical dots of the pattern (the value corresponding to SZ).
- **RVX, RVY**: Horizontal flip and vertical flip of the pattern.
- **Palette set**: The palette set (0–15) to apply to the color codes of this plane. The actual palette number is the 8 bits {palette set, color code} (see Chapter 7).
- **Pattern number and PTS**: The pattern is specified by the pattern number in the +7 byte (the upper nibble is PY, the lower nibble is PX). PX and PY are tile coordinates on the pattern sheet, and PTS in the +5 byte is its page number, so the position of the pattern is at the sheet coordinates (PX×16, PY×16 + PTS×256) (see Section 6.5.2).

The translucency specification is made in bit7-6 (TP1, TP0) of the +3 byte.

| bit7-6 | Transmittance | Composition of the displayed color |
|---|---|---|
| 0 | 0% | The sprite color as it is (opaque) |
| 1 | 25% | Sprite 75% + background 25% |
| 2 | 50% | Sprite 50% + background 50% |
| 3 | 75% | Sprite 25% + background 75% |

## 6.5.2 Pattern Table

The pattern of Mode 3 is 16 dots wide × 4 bits (a color code for each dot). One line is 8 bytes, and the

upper nibble of each byte represents the left dot and the lower nibble the right dot. A dot with color code 0 is transparent (it is not drawn, and does not collide either).

The pattern data of Mode 3 is easy to understand if it is regarded as being placed as a "pattern sheet" 256 dots wide (4 bits / dot). On the sheet, tiles of 16×16 dots are arranged 16 across horizontally, and they continue vertically. PX and PY in the +7 byte of the attribute are the tile coordinates on this sheet, and PTS (bit6-4, PTS2–PTS0) in the +5 byte is the number of the page obtained by dividing the sheet every 256 lines. Note that the value of the +7 byte can also be read as a "pattern number", in which case pattern number = PY × 16 + PX.

The address calculation of the pattern is always the calculation equivalent to Graphic4 (SCREEN 5), regardless of the current screen mode. That is, the pattern sheet is placed in VRAM with the pattern generator table base (R#6) at its head, in exactly the same arrangement as a bitmap of 256 dots per line × 4 bits. For this reason, even when the display is in another screen mode such as SCREEN 6, 7, or 8, setting FG4 of R#45 to 1 allows this pattern data to be drawn directly with a VDP command (see Chapters 3 and 8).

Because the pattern of Mode 3 is read out as a continuous address range, when SP3 is enabled, the physical layout of VRAM in SCREEN 7 and above also returns to flat, rather than interleaved (see Section 11.1). For this reason, switching SP3 in SCREEN 7 and above changes the physical arrangement of the VRAM data. When data other than that of Mode 3, such as a background bitmap, is used together, note that the arrangement of the data shifts before and after the switching.

Let the position of a dot within the pattern be (x, y). x is the horizontal position and takes the range 0–15; y is the vertical position and takes the range 0–127 (the actual upper limit of y used is the number of vertical dots of the pattern determined by SZ, minus 1). The sheet coordinates (X, Y) of this dot are as follows.

- X = PX × 16 + x
- Y = ((PY × 16 + y) & 255) + PTS × 256

In the calculation of Y, the position within the page (PY × 16 + y) is first masked with 255 (& 255), and then the head of the page (PTS × 256) is added. For this reason, when the lower end of a pattern crosses a page boundary, its continuation does not enter the next page, but is placed returning to the head of the same page.

The byte address of a dot is the dot address of Graphic4 itself (at 4 bits / dot, 2 dots make 1 byte, hence >> 1).

```
Address = Base
+ ((PX × 16 + x) + (((PY × 16 + y) & 255) + PTS × 256) × 256) >> 1
```

Which nibble within the byte it is is determined by the least significant bit of X (that is, the least significant bit of x); an even dot is the upper nibble and an odd dot is the lower nibble.

When the pattern does not cross a page boundary, the formula above agrees with the following expanded form. Each term is determined from the dimensions of the sheet (256 dots per line × 4 bits = 128 bytes).

```
Address of the head of the line = Base
+ PTS × 32768        (page advance: 256 lines × 128 bytes)
+ PY × 2048          (tile row advance: 16 lines × 128 bytes)
+ y × 128            (one line of the sheet)
+ PX × 8             (horizontal position of the tile: 16 dots × 4 bits)
(Within the line, 8 bytes are continuous. The 8 dots of the right half are from +4)
```

Because 16 tiles are arranged horizontally in each line of the sheet, the pattern data has a layout interleaved in the row direction in VRAM.

y is the row position within the pattern that corresponds to the display line. When enlargement / reduction is applied, the row corresponding to "display offset × number of pattern rows ÷ MGy" is selected as y (in the RTL this is obtained by division using a reciprocal table), and when RVY = 1, the top and bottom are flipped.

### 6.5.3 How Scaling Looks

Specifying in MGx and MGy values different from the size of the pattern displays the pattern stretched to fill the rectangle, or reduced. When reduced, dots are thinned out; when enlarged, the same dot is displayed continuously. Sampling is clamped at the edges of the pattern, so the pattern never protrudes outside the rectangle.

When MGx = 0 is specified, the pattern is displayed enlarged to the maximum of 256 dots horizontally (0 does not mean "no display"). Similarly, when MGy = 0 is specified, it is displayed enlarged to the

maximum of 256 lines vertically.

### 6.5.4 Display Limits

- Up to 64 planes can be displayed at the same time, and up to 16 planes per line in order of higher priority.
- The 17th and subsequent planes are not displayed on that line (see Section 6.6.2).

### 6.5.5 Sample Program

The sample program `spr3_demo.asm` is an example in SCREEN 5 (Graphic 4, Sprite Mode 3) that uses two kinds of tiles, a 16×16-dot arrow and a ball, and sets a different display magnification, flip, and transparency for each plane. Figure 6.2 shows the execution result. The upper row shows the arrow displayed at ×1, ×2, and ×3; the lower left is a horizontal flip (RVX), the lower middle is a vertical flip (RVY), and on the right a ball is overlapped with TP = 2 (50% blend) on a magenta rectangle.

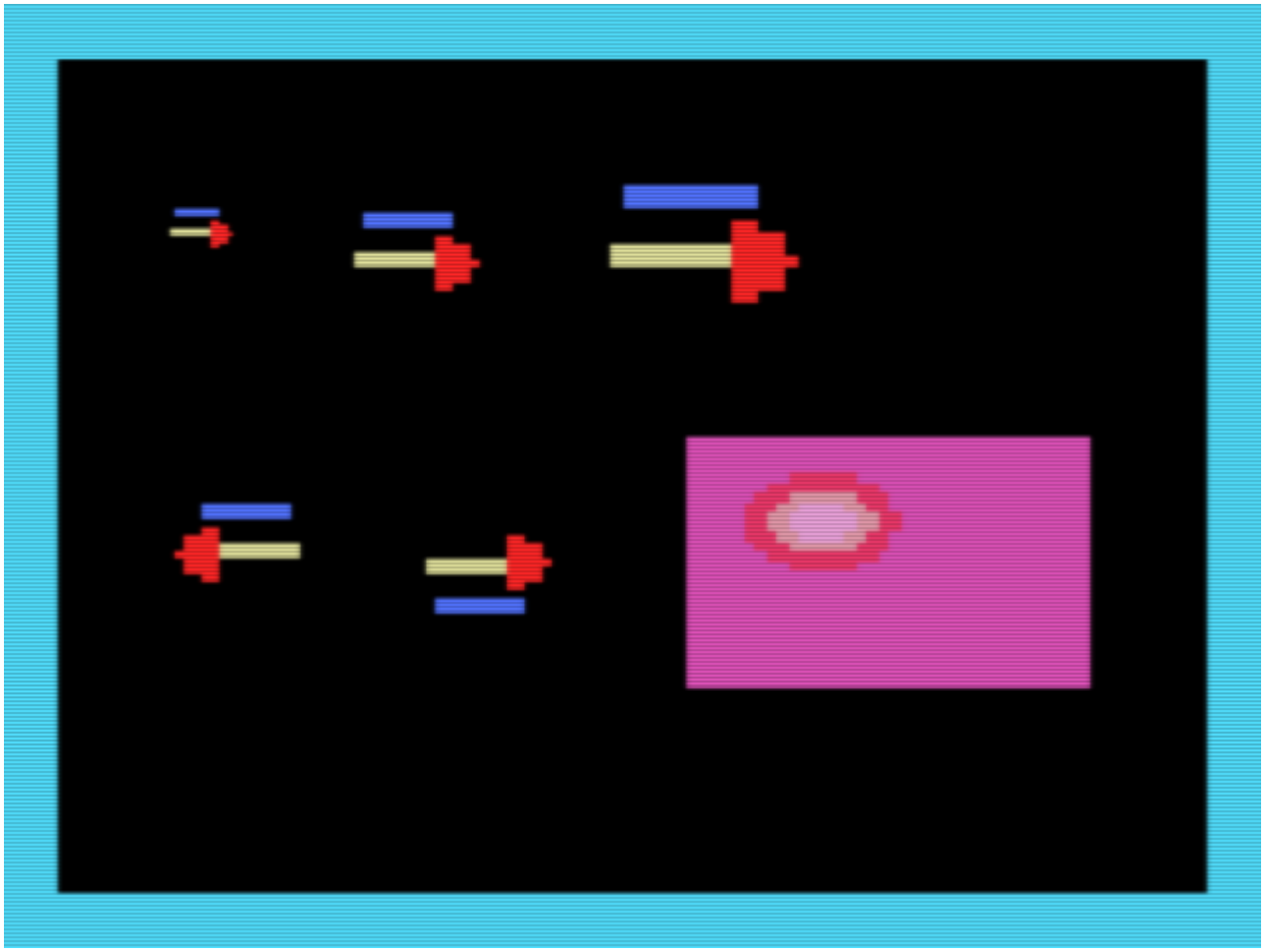


Figure 6.2 Execution result of spr3_demo.asm (Sprite Mode 3)

The attribute of Mode 3 is 8 bytes per plane, and carries, for each plane, the Y coordinate, the

magnification (MGY, MGX), the flip / transparency / palette set, the X coordinate, and the pattern position (→ Section 6.5.1). The attribute table of this sample is as follows. The end of the arrangement is terminated with an entry with Y = 216.

```
attribute_table:
        ; plane 0: arrow x1 at (24, 36)
        db      36, 0x00, 16, 0x00, 24, 0x00, 16, 0x00
        ; plane 1: arrow x2 at (64, 36)
        db      36, 0x00, 32, 0x00, 64, 0x00, 32, 0x00
        ; plane 2: arrow x3 at (120, 28)
        db      28, 0x00, 48, 0x00, 120, 0x00, 48, 0x00
        ; plane 3: arrow x2, RVX (points left) at (24, 110)
        db      110, 0x00, 32, 0x10, 24, 0x00, 32, 0x00
        ; plane 4: arrow x2, RVY (stripe at bottom) at (80, 110)
        db      110, 0x00, 32, 0x20, 80, 0x00, 32, 0x00
        ; plane 5: ball 40x40, TP = 2 (50%), at (150, 104)
        db      104, 0x00, 40, 0x80, 150, 0x00, 40, 0x01
        ; plane 6: terminator
        db      216, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
```

The arrow is given an asymmetrical picture, with a blue band at the top, a yellow shaft, and a red arrowhead, so it can be clearly seen that with RVX the arrowhead points to the left, and with RVY the blue band comes to the bottom. Because the magnification stretches the whole pattern to be displayed in the MGX×MGY rectangle, in the ×3 arrow the original dots have become 3×3 blocks (→ Section 6.5.3).

Because the pattern is placed in a sheet in the same format as Graphic 4 (128 bytes per line), each row of a tile is written at intervals of 128 bytes (→ Section 6.5.2). Note that in this sample, after enabling Sprite Mode 3, the base addresses of the attribute table and the pattern sheet are set explicitly with R#5, R#11, and R#6 (because the initial values differ depending on the BIOS implementation).

## 6.6 Collision Detection and Notification of the Display Limit

### 6.6.1 Sprite-on-Sprite Collision

"Collision" means that dots of sprites that are not transparent occupy the same coordinate. When a collision occurs, the C bit (bit5) of S#0 becomes 1.

- Mode 1: Only the occurrence of a collision can be known. The coordinates cannot be obtained.
- Mode 2: A dot with CC = 1, and a dot on a line with IC = 1, do not generate a collision. Also, a dot with color code 0 is normally transparent and does not collide, but when the TP bit of R#8 is 1 (when color 0 is treated as opaque) it is subject to collision.
- Mode 3: If dots whose color code is not 0 overlap, they collide. There is no mechanism corresponding to CC / IC.

In Mode 2 and Mode 3, the coordinates at which a collision first occurred can be read from S#3–S#6. However, the values obtained from the registers are not the actual coordinates themselves, and the following conversion is necessary (the same as on earlier chips).

```
Actual X coordinate of the collision = (the value represented by S#3, S#4) − 12
Actual Y coordinate of the collision = (the value represented by S#5, S#6) − 8
```

The collision coordinates are reset when S#5 is read out. Also, reading out S#0 clears each flag, including the C bit. For details, see Chapter 4.

### 6.6.2 Display-Limit Notification (5S and 5TH)

When planes are arranged exceeding the limit that can be displayed on one line, the 5S bit (bit6) of S#0 becomes 1, and the number of "the first plane that could not be displayed" is entered in the lower 5 bits (5TH) of S#0.

| Mode | Condition for 5S to be set | Number entered in 5TH |
|---|---|---|
| Mode 1 (S16=0) | 5 or more planes are arranged on one line | The plane number of the 5th plane |
| Mode 2 (S16=0) | 9 or more planes are arranged on one line | The plane number of the 9th plane |
| Mode 3, or Mode 1 / 2 with S16=1 | 17 or more planes are arranged on one line | The plane number of the 17th plane (however, because 5TH has only 5 bits, if the plane number is 32 or higher, 31 is entered) |

5S / 5TH are also cleared by reading out S#0.

## 6.7 Other Extension Bits

The SPS bit of R#25 is a sprite extension bit newly established in the V9968. SPS can be written only when in the extended mode (V58=0), and setting it to 1 causes the display priority of the sprites to change cyclically (rotation of the display priority). This operation can be used in all of Sprite Mode 1 / 2 / 3. Note that SP2 of R#25 is a page-number selection bit for horizontal scrolling, and is unrelated to sprites (see the section on R#25 in Chapter 3).

The S16 bit of R#20 is a sprite extension bit newly established in the V9968 (reset value 0). It extends the upper limit of the number of sprites that can be displayed on one line; in Sprite Mode 3 up to 16 planes are always displayed per line, so S16 has no effect. In Sprite Mode 2, up to 8 planes can be displayed when S16=0, for compatibility with earlier chips, and up to 16 planes when S16=1. In Sprite Mode 1, up to 4 planes can be displayed when S16=0, for compatibility with earlier chips, and up to 16 planes when S16=1. The notification when the display limit is exceeded (the number of planes at which 5S / 5TH are set) also changes in accordance with this limit (see Section 6.6.2).

---

The descriptions of the earlier modes (Mode 1 / 2) in this chapter were checked against Chapter 5 of the MSX2 Technical Handbook (TechHan wiki version). The Mode 3 and V9968 extension parts are based on the RTL (vdp_sprite_select_visible_planes.v, vdp_sprite_info_collect.v, vdp_sprite_makeup_pixel.v, vdp_timing_control_sprite.v, vdp_sprite_divide_table.v) and the verified register notes.

# Chapter 7 Color Palette and Color Representation

In most screen modes, the colors on the V9968 screen are determined indirectly, in the form "color code → color palette → RGB". This chapter describes the structure of the palette RAM and how to write to it, its state at reset, and how colors are interpreted in each screen mode.

For the bit definitions of the registers themselves, see Chapter 3; for the VRAM data format of each screen mode, see Chapter 5.

## 7.1 Structure of the Palette RAM

The color palette is held in the palette RAM inside the V9968.

- The number of entries is 256.
- Each entry is a set of 5 bits each for R, G, and B (0–31 each), so a color can be chosen from a total of 32,768 colors.
- The color information held by screen data and sprites is, in principle, interpreted as an 8-bit palette number consisting of a "palette set (upper 4 bits) + color code (lower 4 bits)", and the RGB of the corresponding entry becomes the displayed color.

On earlier chips (V9938/V9958), the palette had only 16 entries, and the palette number was the 4-bit color code itself. On the V9968, the palette has been expanded to 256 entries, and when the extended palette mode (EPAL, R#20 bit4) or sprite mode 3 is used, all 256 colors, including the upper 4 bits (palette set), can be referenced. In normal use with EPAL = 0, the palette set is fixed at 0, and the palette operates as a 16-color palette, as before.

The palette RAM is write-only and cannot be read back by the CPU.

## 7.2 Writing to the Palette

To write to the palette, specify the entry number in R#16 (color palette pointer) and write data to Port#2. The number of bytes to write and their format differ depending on the value of EPAL. In either format, rewriting R#16 returns the write phase (which byte is next) to the beginning, and each time the write of one entry is completed, the pointer is automatically incremented.

### 7.2.1 EPAL = 0 (Normal Format, V9938/V9958 Compatible)

- Specify the entry number (0–15) in R#16, and write 2 bytes to Port#2.
- The automatic increment of the pointer takes place within the range of the lower 4 bits (after 15 it returns to 0).

| Byte | Bit layout |
|---|---|
| 1st byte | [0][R2][R1][R0][0][B2][B1][B0] |
| 2nd byte | [0][0][0][0][0][G2][G1][G0] |

Each color is specified with 3 bits (0–7). Internally, it is expanded to 5 bits and stored. The expansion places the 3-bit value in the upper bits and replicates its upper 2 bits into the lower bits as well (5-bit value = {v[2:0], v[2:1]}); for example, 7 becomes 31, 6 becomes 27, and 3 becomes 13.

### 7.2.2 EPAL = 1 (Extended Format)

- Specify the entry number (0–255) in R#16, and write 3 bytes to Port#2.
- Each byte carries a 5-bit value for one color in the form [0][0][0][value4][value3][value2][value1][value0], and R, G, and B are written in that order.
- The automatic increment of the pointer takes place over the full 8-bit range (after 255 it returns to 0).

With EPAL = 1, you can either rewrite only selected entries that you want to initialize, or stream all 256 entries in order starting from entry 0.

## 7.3 The Palette at Reset

The state of the palette immediately after reset is as follows.

- **Only entries 0–15 are initialized, and they contain the V9958-compatible initial colors.**
- **Entries 16–255 are undefined.** If these entries are used with EPAL or sprite mode 3, be sure to initialize them in the program before use.

The initial colors (entries 0–15), expressed with 3 bits for each of R, G, and B, are as follows. The expansion to 5 bits follows the method of Section 7.2.1.

| Number | Color | R | G | B |
|---|---|---|---|---|
| 0 | Transparent | 0 | 0 | 0 |
| 1 | Black | 0 | 0 | 0 |
| 2 | Green | 1 | 6 | 1 |
| 3 | Bright green | 3 | 7 | 3 |
| 4 | Dark blue | 1 | 1 | 7 |
| 5 | Bright blue | 2 | 3 | 7 |
| 6 | Dark red | 5 | 1 | 1 |
| 7 | Light blue (cyan) | 2 | 6 | 7 |
| 8 | Red | 7 | 1 | 1 |
| 9 | Bright red | 7 | 3 | 3 |
| 10 | Dark yellow | 6 | 6 | 1 |
| 11 | Yellow | 6 | 6 | 3 |
| 12 | Dark green | 1 | 4 | 1 |
| 13 | Purple (magenta) | 6 | 2 | 5 |
| 14 | Gray | 5 | 5 | 5 |
| 15 | White | 7 | 7 | 7 |

## 7.4 Color Representation by Screen Mode

### 7.4.1 Palette-Referenced Modes (Text, Graphic1–4, Graphic6, Multicolor)

In these modes, the color code (4 bits) of the screen data becomes the lower 4 bits of the palette number as-is, and the palette set is 0. That is, the colors of entries 0–15 are displayed.

Color code 0 is special: it is normally treated as "transparent", and the border color (backdrop color, R#7) shows through at that dot. Setting the TP bit of R#8 to 1 removes this treatment, and color code 0 is also displayed with the color of entry 0. The same rule applies to sprites (see Chapter 6).

The border color itself, in the palette-referenced modes, points to one of entries 0–15 with the lower 4 bits of R#7.

### 7.4.2 Graphic5 (2-Bit Color)

In Graphic5, the color code is 2 bits, and entries 0–3 are used (the palette set is 0). For sprites, the 4-bit color code of mode 2 is interpreted as two pairs, "left half 2 bits / right half 2 bits", and separate colors are assigned to each pair of 2 horizontally adjacent dots. This is the same treatment as before, corresponding to the fact that 1 horizontal dot of Graphic5 corresponds to a half dot of a sprite.

### 7.4.3 Graphic7 RGB Mode (EPAL = 0, YJK = 0)

In Graphic7, 1 byte of screen data directly represents the color of 1 dot, and the palette is not used. The bit assignment is the same as before (SCREEN 8 of the V9938).

| Bit | Content |
|---|---|
| bit7-5 | G (3 bits) |
| bit4-2 | R (3 bits) |
| bit1-0 | B (2 bits) |

Internally, each color is expanded to 5 bits. R and G are expanded from the 3-bit value as {v[2:0], v[2:1]}, and B is expanded from the 2-bit value in the form {v[1:0], v[1]・v[0], v[1:0]}. The border color (R#7) is also interpreted as an 8-bit direct color in this mode.

Note that sprite colors also go through the palette in this mode (the 4-bit sprite color code points to entries 0–15).

This is a specification of the V9968 and differs from the earlier V9938. On the V9938, sprites in Graphic7 used a fixed set of 16 colors that did not use the palette, but on the V9968, sprite colors are also determined from the same 256-color palette as the background. The 256 colors are divided into 16 groups (palette sets) of 16 colors each, and a sprite selects one group and is displayed with the 16 colors within that group. However, because the first color of each group (color code 0) is treated as transparent, only 15 colors can actually be used. For how the palette set is determined, see Section 7.4.7 and Chapter 6.

### 7.4.4 Graphic7 Extended Palette Mode (EPAL = 1)

In Graphic7 with EPAL = 1, 1 byte of screen data is interpreted as a palette number (0–255) as-is, and all 256 colors can be used at the same time. In this case, the border color (R#7) is also interpreted as an 8-bit palette number.

### 7.4.5 YJK Mode (YJK = 1 in R#25)

The YJK mode is a mode in which the Graphic7 screen data is interpreted as luminance Y and color differences J and K (equivalent to the earlier SCREEN 10–12). Groups of 4 dots form one set, and J and K are shared by the dots within a set. The Y of each dot is 5 bits, and J and K are signed 6-bit values (-32–31). The conversion to RGB is performed with the following formulas, and each color is clamped to the range 0–31.

```
R = Y + J
G = Y + K
B = (5×Y − (2×J + K) + 2) ÷ 4
```

For the byte layout of the screen data (which byte holds Y, J, or K), see Chapter 5 (Graphic7-YJK). In YJK mode as well, the border color and sprite colors are determined by the palette.

### 7.4.6 YAE Mode (YAE = 1 in R#25)

YAE is a variant of the YJK mode in which, for each dot, the display can be switched between "the color converted from YJK" and "the color of the palette" (equivalent to the earlier SCREEN 11). Which color is

used is determined by a flag on the screen data side. For details of the data format, see Chapter 5 (Graphic7-YAE).

### 7.4.7 Sprite Colors

A sprite color code (4 bits) is combined with a palette set to form an 8-bit palette number.

| Sprite mode | How the palette set is determined |
|---|---|
| Mode 1 | Always 0 (entries 0–15) |
| Mode 2 (EPAL = 0) | Always 0 (entries 0–15) |
| Mode 2 (EPAL = 1) | Lower 4 bits of the 4th attribute byte (see Section 6.4.4) |
| Mode 3 | The palette set field of the attribute (see Section 6.5.1) |

In mode 3, in addition to this, per-plane translucent blending (opacity 0/25/50/75%) is applied to the displayed color determined by the palette (see Section 6.5.1).

## 7.5 Sample Program: Rewriting the Palette and Color Bars

The sample program `pal_demo.asm` is an example that rewrites all 16 entries of the palette with an original sequence of colors (a ramp) and draws 16 vertical bars on the screen, displaying the color of each entry as-is. The execution result is shown in Figure 7.1. Because the flow of colors is completely different from the initial palette, it can be seen that the palette rewrite is taking effect.



Figure 7.1 Execution result of pal_demo.asm (color bars using the rewritten palette)

To write to the palette, simply set the starting entry number in R#16 and write 2 bytes at a time to the palette port (Port#2). The entry number is automatically increased each time one entry is written (→ Section 7.2.1).

```
        ; ---- rewrite palette entries 0..15
        ld      a, 0            ; R#16 = 0 (start entry)
        ld      b, 16
        call        vdp_reg
        ld      hl, palette_table
        ld      b, 32           ; 16 entries x 2 bytes
palette_loop:
        ld      a, [hl]
        out     [0x9a], a       ; palette port (auto-increments)
        inc     hl
        djnz        palette_loop
```

The palette data is in the 2-byte format (EPAL = 0) in which the 1st byte holds R and B and the 2nd byte

holds G. In this sample, the ramp goes from black through blue, purple, red, yellow, and green to white.

```
        db      0x00, 0x00      ;  0: R=0 G=0 B=0 (black)
        db      0x13, 0x00      ;  1: R=1 G=0 B=3
        db      0x24, 0x00      ;  2: R=2 G=0 B=4
        db      0x35, 0x01      ;  3: R=3 G=1 B=5
        ; (omitted)
        db      0x77, 0x07      ; 15: R=7 G=7 B=7 (white)
```

HMMV is used to draw the bars, filling bar i with color i (see the source file for the bar-drawing code). Color 0 is treated as the transparent color in graphic modes, and the background color (palette entry 0 = black) is visible as-is, so bar 0 at the left edge blends into the background.

---

The palette write formats and RAM structure in this chapter are based on the verified register notes and the RTL (vdp_cpu_interface.v, vdp_color_palette.v, vdp_color_palette_ram.v). The table of initial colors was checked against the initial palette table of the MSX2 Technical Handbook (Technical Handbook wiki edition). The initialization range at reset (only the first 16 entries; entries 16–255 are undefined) is based on items confirmed by the author.

# Chapter 8 VDP Commands

The VDP commands are a set of functions that execute rectangle transfers, fills, straight-line drawing, color searches, and similar operations in VRAM in hardware. The CPU only has to set the parameters in the registers and start a command; much of the drawing work can be left to the VDP. This chapter first describes the execution model common to all commands, and then explains each command individually.

The bit layouts of the command-related registers (R#32–R#58) themselves were described in Chapter 3. This chapter concentrates on "how to use the commands." The descriptions of the operation of the conventional commands in this chapter (the 13 commands present since the V9938/V9958) are based on the *MSX2 Technical Handbook* (Technical Handbook wiki edition), checked against the V9968 implementation (RTL).

## 8.1 Command Execution Model

### 8.1.1 Command List

The V9968 can execute the following 16 commands. In R#46 (the command register), the upper 4 bits are the command code and the lower 4 bits specify the logical operation.

| Command | Category | Source | Destination | Unit | Upper 4 bits of R#46 |
|---|---|---|---|---|---|
| HMMC | High-speed transfer | CPU | VRAM | Byte | 1111 |
| YMMM | High-speed transfer | VRAM | VRAM | Byte | 1110 |
| HMMM | High-speed transfer | VRAM | VRAM | Byte | 1101 |
| HMMV | High-speed transfer | VDP | VRAM | Byt | 1100 |

| Command | Category | Source | Destination | Unit | Upper 4 bits of R#46 |
|---|---|---|---|---|---|
| | | | | e | |
| LMMC | Logical transfer | CPU | VRAM | Dot | 1011 |
| LMCM | Logical transfer | VRAM | CPU | Dot | 1010 |
| LMMM | Logical transfer | VRAM | VRAM | Dot | 1001 |
| LMMV | Logical transfer | VDP | VRAM | Dot | 1000 |
| LINE | Drawing | VDP | VRAM | Dot | 0111 |
| SRCH | Search | VDP | VRAM | Dot | 0110 |
| PSET | Drawing | VDP | VRAM | Dot | 0101 |
| POINT | Read | VRAM | VDP | Dot | 0100 |
| LRMM | Logical transfer [New in V9968] | VRAM | VRAM | Dot | 0011 |
| LFMC | Font drawing [New in V9968] | CPU | VRAM | Dot | 0010 |
| LFMM | Font drawing [New in V9968] | VRAM | VRAM | Dot | 0001 |
| STOP | Stop | — | — | — | 0000 |

LRMM, LFMC, and LFMM are commands newly added in the V9968. They are **valid only in extended mode (when the V58 bit of R#21 is 0)**; in V9958 compatible mode (V58=1) they are treated as the STOP command (→ Chapter 3). Note that the "ECOM bit of R#20" found in earlier documentation was a bit that had once been provided to enable the new commands, but in the current design it has been integrated into V58, and no independent ECOM bit exists.

### 8.1.2 Parameter Registers

The command parameters are set in R#32–R#58 before the command is started. The roles of the

principal parameters are as follows (see Chapter 3 for details of the bit layouts).

| Register | Name | Description |
|---|---|---|
| R#32, R#33 | SX | Source X coordinate (9 bits) |
| R#34, R#35 | SY | Source Y coordinate (10 bits) |
| R#36, R#37 | DX | Destination X coordinate (9 bits) |
| R#38, R#39 | DY | Destination Y coordinate (10 bits) |
| R#40, R#41 | NX | Number of dots in the X direction (10 bits; 0 is treated as 1024) |
| R#42, R#43 | NY | Number of dots in the Y direction (10 bits; 0 is treated as 1024) |
| R#44 | CLR | Color code / data transferred to and from the CPU |
| R#45 | ARG | Argument register (MAJ, DIX, EQ, DIY, MXS, MXD, XHR/FG4) |
| R#46 | CMR | Command code (writing starts execution) |
| R#47, R#48 | VX | Rotation vector X (for LRMM; signed 16 bits, 8.8 fixed point) |
| R#49, R#50 | VY | Rotation vector Y (for LRMM; signed 16 bits, 8.8 fixed point) |
| R#51, R#52 | WSX | Window start X coordinate (for LRMM) |
| R#53, R#54 | WSY | Window start Y coordinate (for LRMM) |
| R#55, R#56 | WEX | Window end X coordinate (for LRMM) |
| R#57, R#58 | WEY | Window end Y coordinate (for LRMM) |

The foreground color used by LFMC/LFMM is set in R#44, and the background color in R#12 (→ Sections 8.15 and 8.16). There are no color registers dedicated to font drawing.

Coordinates do not distinguish pages; the whole of VRAM is treated as a single large coordinate system.

The X coordinate ranges from 0 to 511, and the Y coordinate from 0 to 1023. In commands that specify a rectangle, (SX, SY) or (DX, DY) is the base point, and NX and NY specify the number of dots along each side.

Each bit of the argument register R#45 has the following meaning.

| Bit | Name | Meaning |
|---|---|---|
| 0 | MAJ | Long-axis direction for LINE (0: the X axis is the long axis, 1: the Y axis is the long axis) |
| 2 | DIX | Transfer direction in the X direction (0: to the right, 1: to the left). For SRCH, the search direction |
| 1 | EQ | Match condition for SRCH (0: search for a dot equal to the specified color, 1: search for a dot different from the specified color) |
| 3 | DIY | Transfer direction in the Y direction (0: downward, 1: upward) |
| 4 | MXS | Source memory select (in the V9968 the expansion RAM has been abolished, so the only memory that can be selected is VRAM) |
| 5 | MXD | Destination memory select (same as above) |
| 7 | FG4 | Valid only in extended mode. When 1, the VDP commands operate as Graphic4 (SCREEN 5) regardless of the screen mode specification (see Section 8.1.3) |
| 6 | XHR | Valid only in extended mode. In LRMM, treats the pixels as vertically elongated pixels, so that a rotated display in Graphic5 and Graphic6 can be viewed with the correct aspect ratio (see Section 8.14). For MXC in compatible mode, see Chapter 3 |

When FG4 (R#45 bit 7, valid only in extended mode) is set to 1, the VDP commands operate as Graphic4 (SCREEN 5) regardless of the current screen mode specification. Even in display modes that are not normally targets of the commands, such as SCREEN 1–4, the command side is processed in the Graphic4 coordinate and color formats. Its principal use is drawing into the sprite patterns of SpriteMode3: because the pattern address calculation is in Graphic4 format, lines and the like can be drawn into the

pattern data while the display remains in SCREEN 6, 7, or 8 (→ Section 6.5.2). If the display itself is Graphic4 (SCREEN 5), the commands already operate as Graphic4, so the presence or absence of FG4 makes no difference to the operation.

Selection of expansion RAM by MXS/MXD is a function present in the V9938/V9958 specifications, but in the V9968 the expansion RAM itself has been abolished, and the only memory that can be selected is VRAM.

### 8.1.3 Starting a Command and Checking for Completion

The procedure for executing a command is as follows.

1. Set the required parameters in R#32–R#45 (and, for LRMM, in R#47–R#58)
2. Writing the command code and the logical operation to R#46 starts execution
3. During execution, the CE bit (bit 0) of status register S#2 is 1
4. When execution ends, the CE bit returns to 0

The CPU polls the CE bit and waits for completion. Another command must not be started while a command is executing. The end of a command can also be detected without polling by using the command completion interrupt (enabled by the CEIE bit of R#20) (→ Chapter 10).

To stop a command that is executing partway through, execute the STOP command (write 00h to R#46).

In commands that exchange data with the CPU, such as HMMC, LMMC, and LMCM, the TR bit (bit 7) of S#2 indicates transfer ready. The CPU accesses R#44 (write) or S#7 (read) only when TR is 1. The procedure is described in the section for each command.

Command operation is guaranteed in the bitmap screen modes (equivalent to GRAPHIC 4 to GRAPHIC 7). However, when FG4=1 in extended mode, the commands operate as Graphic4 even if the display is in another screen mode (including SCREEN 1–4), so, for example, the sprite patterns of SpriteMode3 (Graphic4 format) can be drawn into with VDP commands while the display remains in SCREEN 6, 7, or 8.

Also, when FIL=1 (flat interlace, R#20 bit 5; see Chapter 3), the screen formed by joining page0 and page1 vertically is treated as a single high-resolution screen, and the VDP commands also operate with this joined screen as their target.

### 8.1.4 Logical Operations

In commands that write in dot units, the lower 4 bits of R#46 can specify an operation between the source color SC and the destination color DC.

| Code | Name | Operation |
|---|---|---|
| 0000 | IMP | DC = SC |
| 0001 | AND | DC = SC AND DC |
| 0010 | OR | DC = SC OR DC |
| 0011 | EOR | DC = SC XOR DC |
| 0100 | NOT | DC = NOT SC |
| 0101–0111 | (Reserved) | Same as IMP (writes SC as-is) |
| 1000 | TIMP | If SC=0, DC is left unchanged; otherwise DC = SC |
| 1001 | TAND | If SC=0, DC is left unchanged; otherwise DC = SC AND DC |
| 1010 | TOR | If SC=0, DC is left unchanged; otherwise DC = SC OR DC |
| 1011 | TEOR | If SC=0, DC is left unchanged; otherwise DC = SC XOR DC |
| 1100 | TNOT | If SC=0, DC is left unchanged; otherwise DC = NOT SC |
| 1101–1111 | (Reserved) | Same as TIMP (operates as IMP with transparency) |

In the operations marked with "T", dots whose source color is 0 are not subject to rewriting, and the destination is left as-is. These are used to overlay only the colored parts of a figure. The handling of the reserved codes (0101–0111 are equivalent to IMP, and 1101–1111 are equivalent to TIMP) has been confirmed in the V9968 implementation.

In the V9968 implementation, the transparency decision for the T-series operations is made according to whether the entire source color code is 0. When specifying a color in Graphic 6/7 (4 bits/dot), put the

color code in the lower nibble (implementation note from the documentation of the alexmoncks fork).

### 8.1.5 Notes on Byte-Unit Commands

The high-speed transfer commands HMMC, YMMM, HMMM, and HMMV handle data in byte units. For this reason, in screen modes in which multiple dots are packed into one byte, the lower bits of the X coordinate and of NX are not referenced (the lowest 1 bit in GRAPHIC 4 and 6, and the lowest 2 bits in GRAPHIC 5). If alignment in dot units is required, use the logical transfer commands.

### 8.1.6 Register State at Command Completion

As a command executes, the parameter registers such as SX, SY, DX, DY, NX, and NY are updated in step with the internal progress. The values at the time of completion reflect how far processing has proceeded, and the values that were set do not necessarily remain as-is (for example, NY is the number of remaining lines, and SY/DY point to the coordinate following the final line). For the next command, set the required parameters again each time. For the detailed update rules, see Table 4.7 in Chapter 6 of the *MSX2 Technical Handbook*.

## 8.2 HMMC (High-Speed CPU → VRAM Transfer)

| Item | Description |
|---|---|
| Command code | F0h |
| Transfer unit | Byte |
| Logical operation | Cannot be specified |
| Parameters used | DX, DY, NX, NY, CLR, ARG (DIX, DIY) |

### HMMC

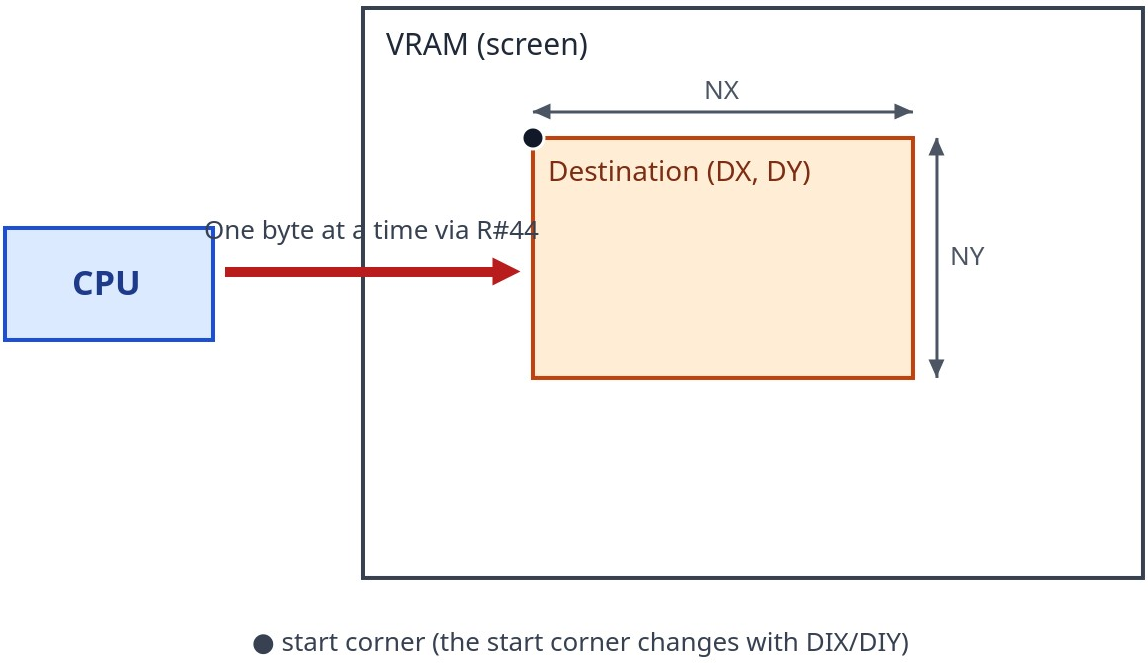



Figure 8.1 Coordinate relationship of HMMC

Transfers data from the CPU to a specified area of VRAM in byte units.

The procedure is as follows. First set the parameters, and, **after writing only the first byte of the data to be transferred to R#44**, write F0h to R#46. The VDP receives the data in R#44, writes it to VRAM, and thereafter waits for data from the CPU. Each time the TR bit of S#2 becomes 1, the CPU writes the next byte to R#44. When all the data has been sent and the CE bit becomes 0, the transfer is complete.

## 8.3 YMMM (High-Speed VRAM → VRAM Transfer in the Y Direction)

| Item | Description |
|---|---|
| Command code | E0h |
| Transfer unit | Byte |
| Logical operation | Cannot be specified |
| Parameters used | SY, DX, DY, NY, ARG (DIX, DIY) |

### YMMM

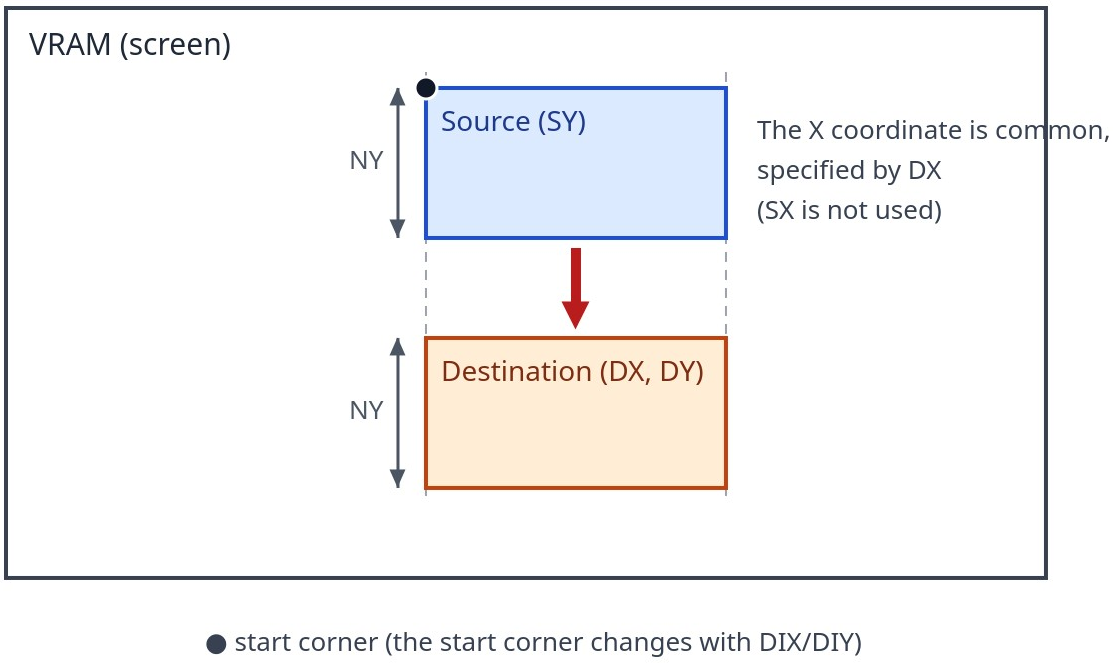



Figure 8.2 Coordinate relationship of YMMM

Transfers data within VRAM in the Y direction in byte units, keeping the same X coordinate. The X coordinate is common to the source and the destination, and is specified by DX (SX is not used). This is used for vertical scrolling and for copying within the same column. The transfer is limited to the Y direction only, and NX is not used. DIY selects the transfer direction (up or down).

## 8.4 HMMM (High-Speed VRAM → VRAM Transfer)

| Item | Description |
|---|---|
| Command code | D0h |
| Transfer unit | Byte |
| Logical operation | Cannot be specified |
| Parameters used | SX, SY, DX, DY, NX, NY, ARG |

### HMMM

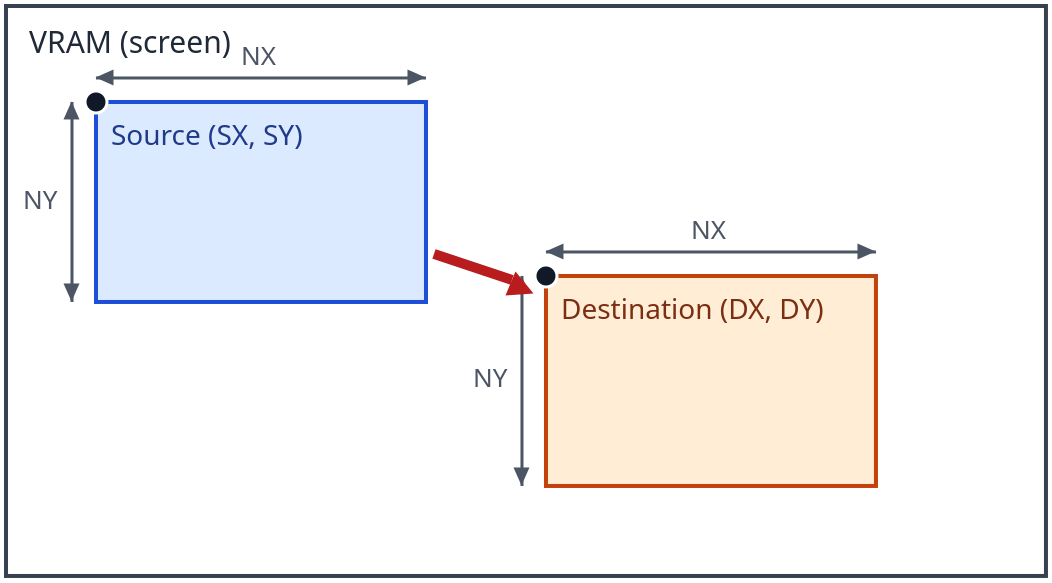



Figure 8.3 Coordinate relationship of HMMM

Transfers the data in a specified area of VRAM to another area of VRAM in byte units. Because the transfer is in byte units, the lower bits of the X coordinates are not referenced, as with HMMC (→ Section 8.1.5).

## 8.5 HMMV (High-Speed Rectangular Fill)

| Item | Description |
|---|---|
| Command code | C0h |
| Transfer unit | Byte |
| Logical operation | Cannot be specified |
| Parameters used | DX, DY, NX, NY, CLR, ARG |

### HMMV

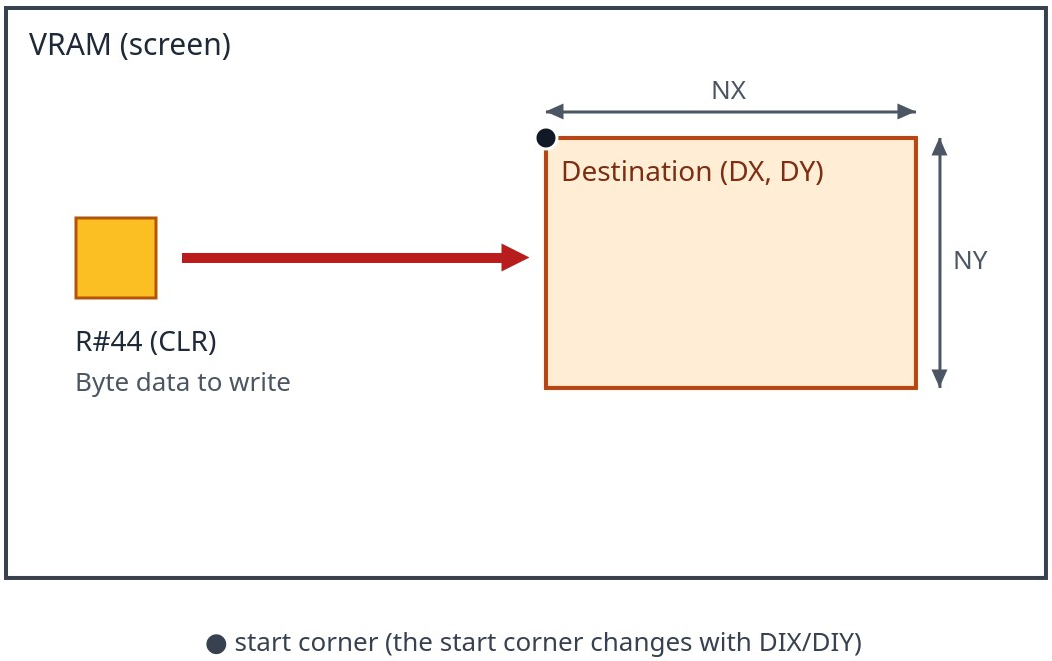



Figure 8.4 Coordinate relationship of HMMV

Fills a specified area of VRAM with the byte data set in R#44 (CLR). CLR is not a color code, but the byte itself that is written to VRAM.

## 8.6 LMMC (Logical CPU → VRAM Transfer)

| Item | Description |
|---|---|
| Command code | B0h + logical operation |
| Transfer unit | Dot |
| Parameters used | DX, DY, NX, NY, CLR, ARG |

### LMMC

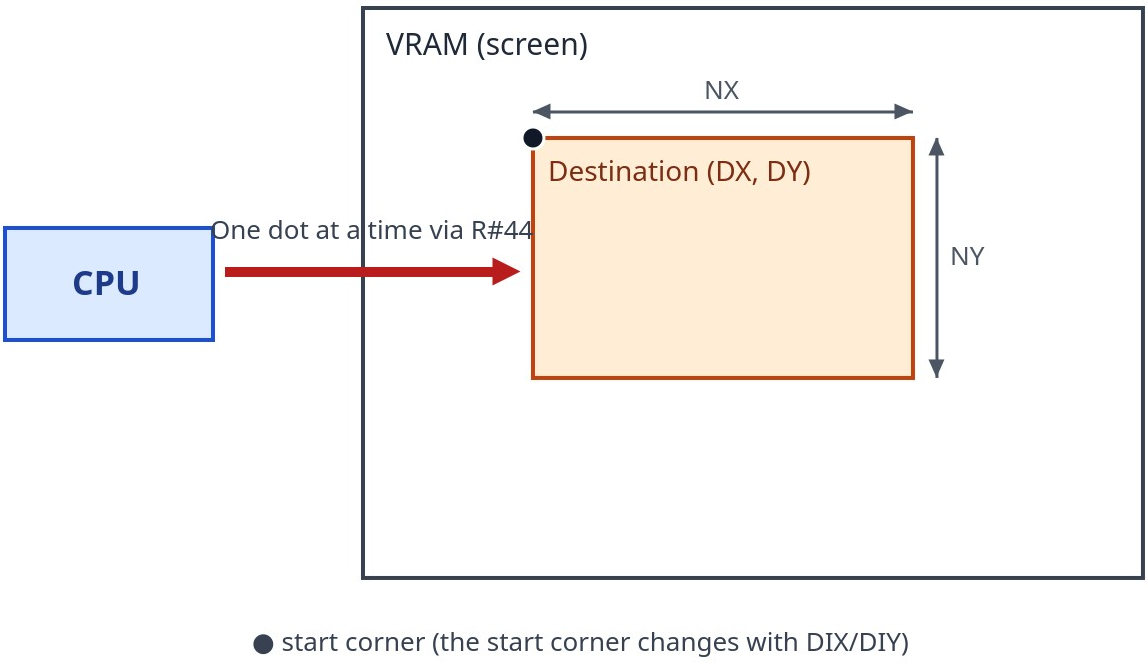



Figure 8.5 Coordinate relationship of LMMC

Transfers data from the CPU to a specified area of VRAM in dot units. In the logical transfer commands, the CPU supplies a color code of "1 dot = 1 byte" in all screen modes.

The procedure is the same as for HMMC: write the first byte to R#44 and then write the command to R#46, and thereafter send the data one dot at a time to R#44 while watching the TR bit.

## 8.7 LMCM (Logical VRAM → CPU Transfer)

| Item | Description |
|---|---|
| Command code | A0h |
| Transfer unit | Dot |
| Parameters used | SX, SY, NX, NY, ARG |

### LMCM

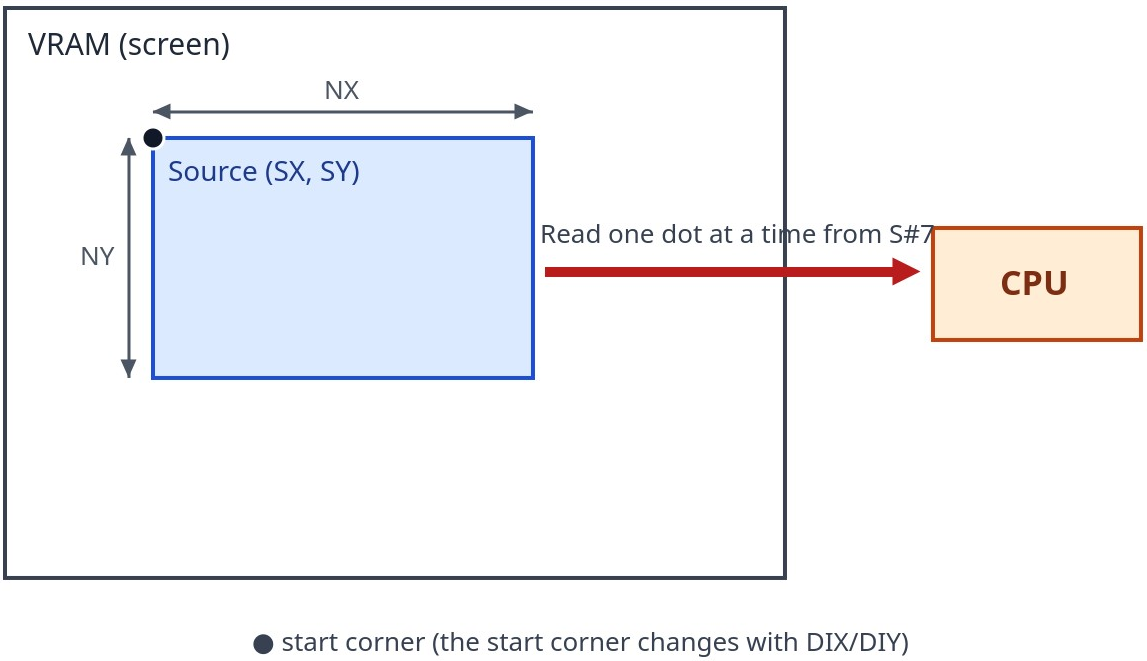



Figure 8.6 Coordinate relationship of LMCM

Transfers the data in a specified area of VRAM to the CPU in dot units. When the command is started, the VDP begins to prepare the data. When the TR bit of S#2 is 1, the CPU reads the color code for one dot from **S#7**. When the CE bit becomes 0, the transfer of all the data is finished.

Note that the conventional practice is to read S#7 once before starting the reads, to initialize the TR bit.

## 8.8 LMMM (Logical VRAM → VRAM Transfer)

| Item | Description |
|---|---|
| Command code | 90h + logical operation |
| Transfer unit | Dot |
| Parameters used | SX, SY, DX, DY, NX, NY, ARG |

### LMMM

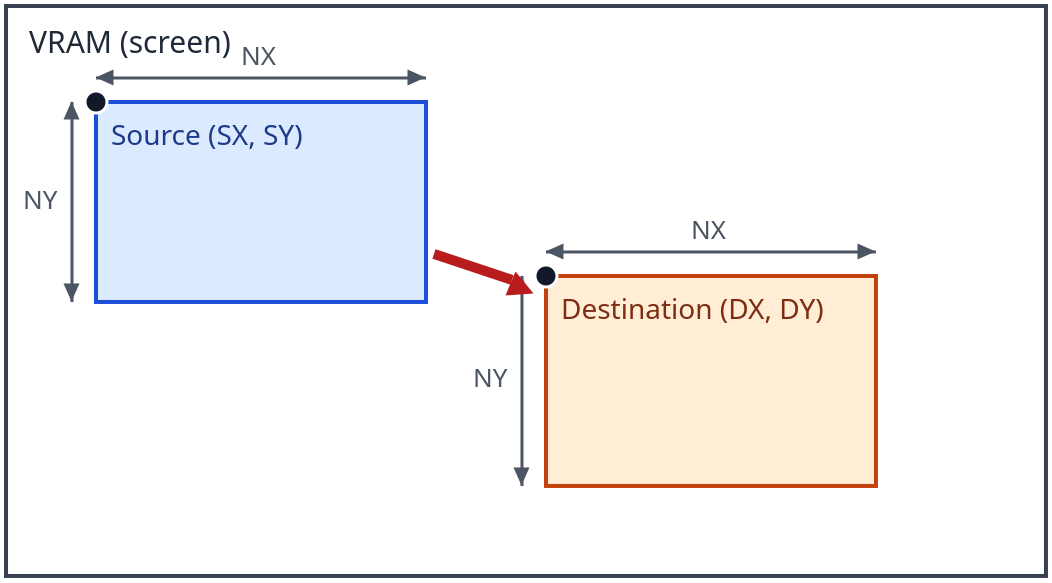



Figure 8.7 Coordinate relationship of LMMM

Transfers the data in a specified area of VRAM to another area of VRAM in dot units. Combined with a transparency operation such as TIMP, figures can be overlaid with color 0 as the transparent color.

## 8.9 LMMV (Logical Rectangular Fill)

| Item | Description |
|---|---|
| Command code | 80h + logical operation |
| Transfer unit | Dot |
| Parameters used | DX, DY, NX, NY, CLR, ARG |

### LMMV

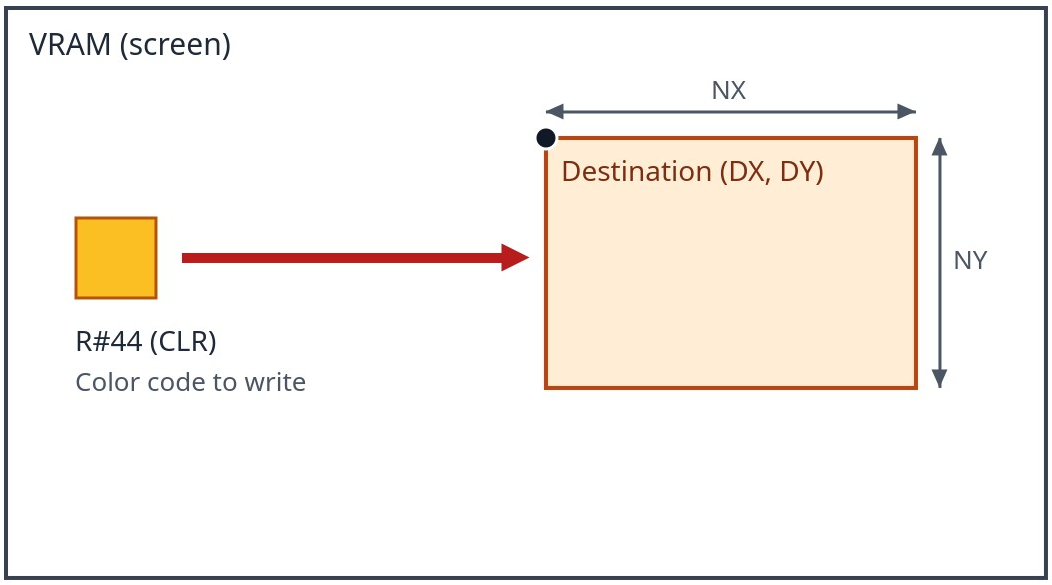


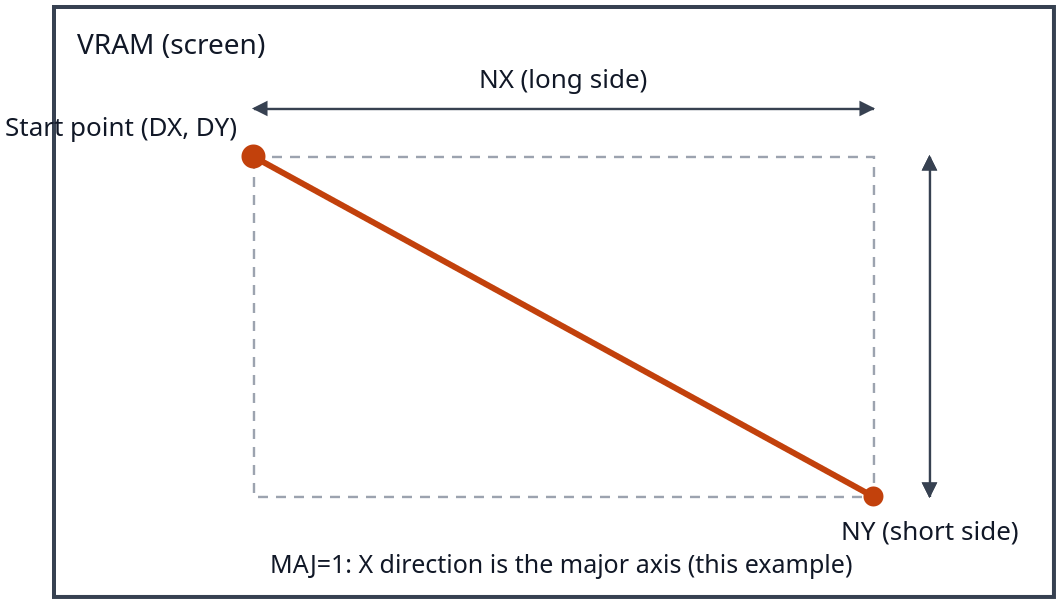

Figure 8.8 Coordinate relationship of LMMV

Fills a specified area of VRAM with the color code in R#44 (CLR) in dot units. A logical operation with the existing VRAM data can also be specified.

## 8.10 LINE (Line Drawing)

| Item | Description |
|---|---|
| Command code | 70h + logical operation |
| Transfer unit | Dot |
| Parameters used | DX, DY, NX (long side), NY (short side), CLR, ARG (MAJ, DIX, DIY) |

## LINE




Draw color is R#44 (CLR). DIX/DIY select the direction the line extends

Figure 8.9 Coordinate relationship of LINE

Draws a straight line starting at (DX, DY). The straight line is specified by the long side and the short side of a rectangle of which it is the diagonal. **Set the number of dots of the long side in NX and the number of dots of the short side in NY, and indicate the direction of the long axis with the MAJ bit of ARG** (if MAJ=0, the X-axis direction is the long axis; if MAJ=1, the Y-axis direction is the long axis). Before setting the parameters, compare the two sides and be sure to put the longer one in NX (the long side). DIX and DIY specify the direction in which the line extends (left/right and up/down). The color drawn is CLR.

## 8.11 SRCH (Color Search)

| Item | Description |
|---|---|
| Command code | 60h |
| Transfer unit | Dot |
| Parameters used | SX, SY, CLR, ARG (DIX, EQ) |
| Result | BD bit of S#2, S#8, S#9 |

| Item | Description |
| --- | --- |
| | |

### SRCH

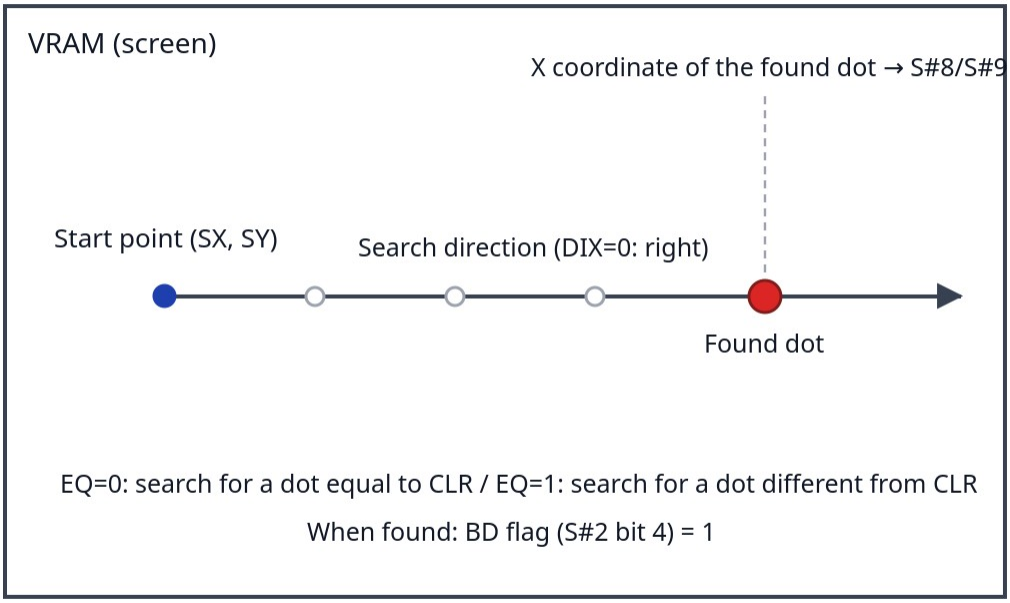



Figure 8.10 Coordinate relationship of SRCH

Scans from (SX, SY) to the right (DIX=0) or to the left (DIX=1) and searches for a dot of a color that meets the condition. If EQ=0, a dot equal to CLR is a "hit"; if EQ=1, a dot different from CLR is a "hit." The command ends when the target dot is found, or when the search reaches the edge of the screen without finding one.

After the command ends, if the BD bit (bit 4) of S#2 is 1, it indicates that a dot was found, and the X coordinate of the dot found is stored in S#8 (the lower 8 bits) and S#9 (the most significant bit). This is used for purposes such as searching for the boundary in a paint routine.

## 8.12 PSET (Plot a Dot)

| Item | Description |
| --- | --- |
| Command code | 50h + logical operation |

| Item | Description |
|---|---|
| Transfer unit | Dot |
| Parameters used | DX, DY, CLR, ARG |

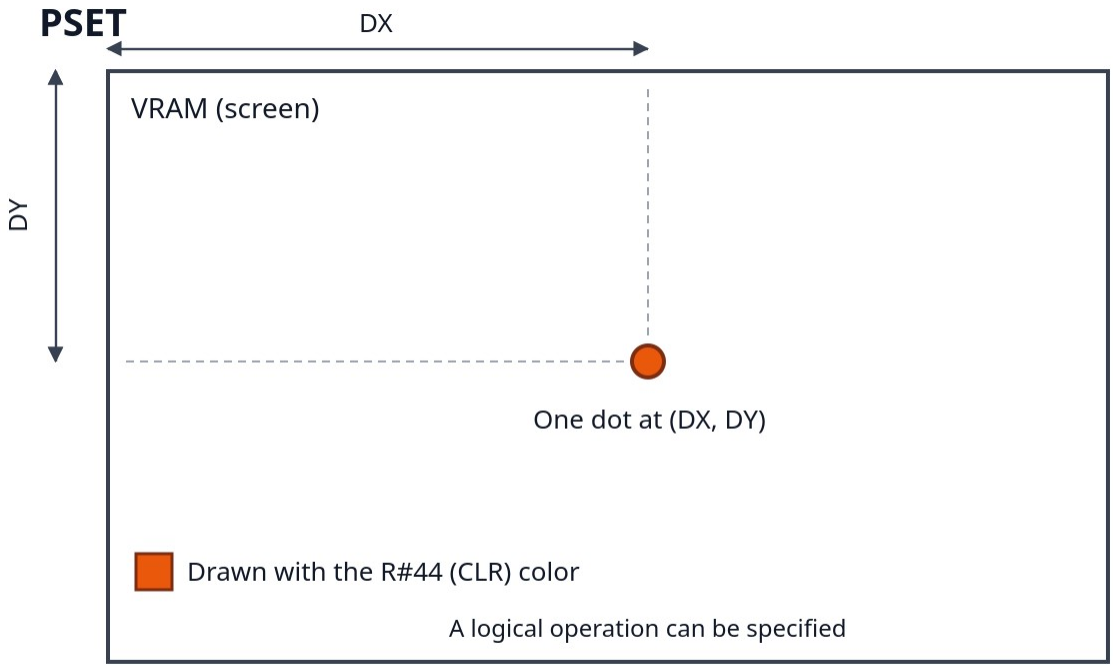



Figure 8.11 Coordinate relationship of PSET

Draws the single dot at (DX, DY) in the color of CLR.

## 8.13 POINT (Read a Color)

| Item | Description |
|---|---|
| Command code | 40h |
| Transfer unit | Dot |
| Parameters used | SX, SY |
| Result | S#7 |

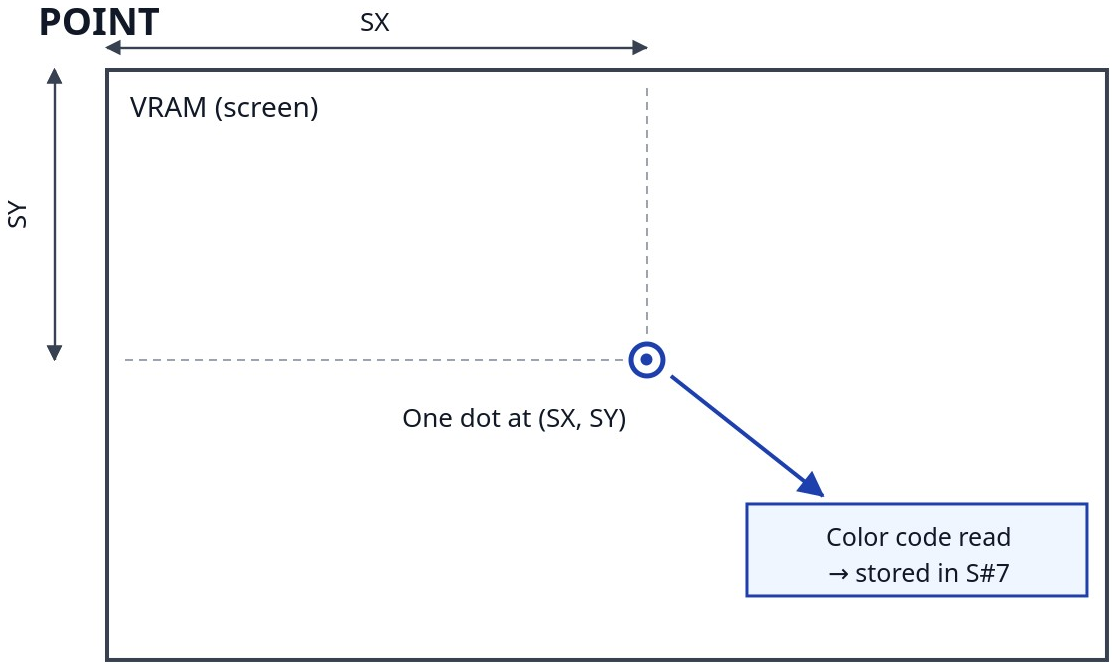



Figure 8.12 Coordinate relationship of POINT

Reads the color code of the single dot at (SX, SY). After the command ends, the color code is stored in S#7.

## 8.14 LRMM (Logical Transfer with Rotation and Scaling)

| Item | Description |
|---|---|
| Command code | 30h + logical operation |
| Transfer unit | Dot |
| Parameters used | SX, SY, DX, DY, NX, NY, CLR, ARG, VX (R#47, R#48), VY (R#49, R#50), WSX (R#51, R#52), WSY (R#53, R#54), WEX (R#55, R#56), WEY (R#57, R#58) |
| Conditions of use | Valid only in extended mode (V58=0) |

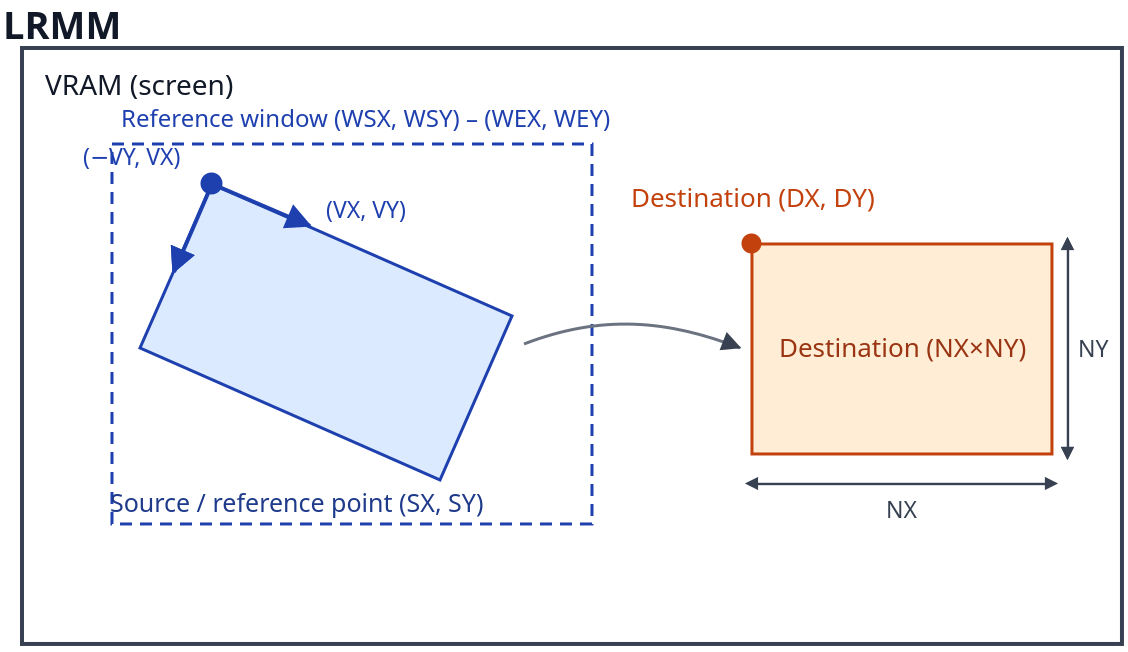



Figure 8.13 Coordinate relationship of LRMM

This is a command newly added in the V9968 that transfers from VRAM to VRAM in dot units while applying rotation and enlargement/reduction (scaling). For each dot of the destination, it reads the dot at the "reference point" on the source side and draws it through a logical operation.

(SX, SY) is the starting point of the reference. The first dot of the destination, (DX, DY), corresponds to the dot at (SX, SY). Because (SX, SY) is not itself the center of rotation, the starting point must be calculated and set if the figure is to be rotated about its center.

As drawing progresses, the reference point moves by the rotation vector (VX, VY). Each time the destination advances by one dot in the X direction, the reference point advances by (VX, VY); when one line has been completed and drawing moves to the next line, it advances by (−VY, VX). VX and VY are signed fixed-point numbers (8 bits of fraction), set as VX = INT(256×cosθ) and VY = INT(256×sinθ) for a rotation angle θ. 256 corresponds to 1.0; if the magnitude of the vector is greater than 1.0, the source is traversed more quickly and the figure is reduced, and if it is smaller, the figure is enlarged.

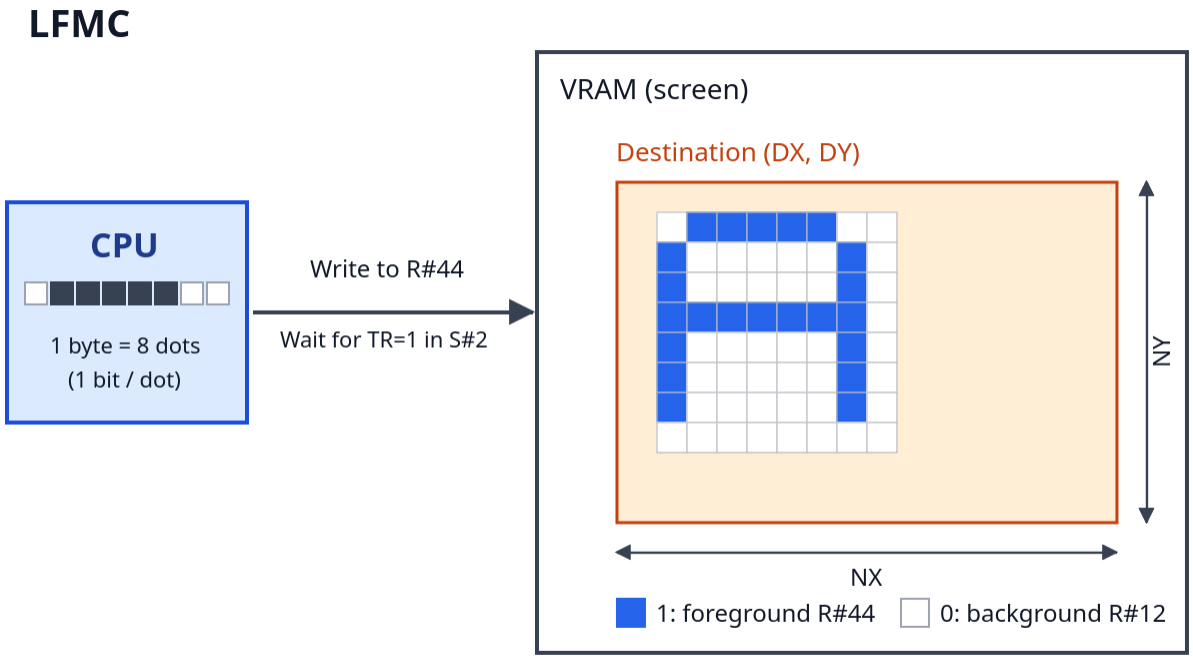
A reference window, (WSX, WSY)–(WEX, WEY), can be specified for the source. The window is a range that includes both its starting point and its end point, and dots whose reference point lies inside it are read from the source. If the reference point lies outside the window, the color of R#44 (CLR) is used in place of the source pixel for that dot, and the result of applying the logical operation to it is drawn to the

destination. The window registers (R#51–R#58) are effective for LRMM only, and have no effect on the other commands.

When bit 6 of the argument register R#45 (XHR, in extended mode) is set to 1, LRMM treats the pixels as vertically elongated pixels. The pixels of Graphic5 and Graphic6 (SCREEN 6, 7) are vertically elongated, so rotating them as-is would break the aspect ratio of the display. XHR is a mechanism for allowing a rotated display to be viewed with the correct aspect ratio in such screen modes.

## 8.15 LFMC (CPU → VRAM Font Drawing)

| Item | Description |
|---|---|
| Command code | 20h + logical operation |
| Transfer unit | Dot |
| Parameters used | DX, DY, NX, NY, ARG, foreground color (R#44), background color (R#12) |
| Conditions of use | Valid only in extended mode (V58=0) |




Input is 1 bit/dot, expanded to the screen mode's bits/dot when drawn

Figure 8.14 Coordinate relationship of LFMC

This is a command newly added in the V9968 that draws a font pattern (bitmap) sent from the CPU into VRAM while expanding it. The input data is 1 bit/dot, and is expanded to the bits/dot of the screen mode when drawn. A dot whose bit is 1 is drawn in the foreground color, and a dot whose bit is 0 in the background color. Whereas LMMC sends a color code for each dot, LFMC packs 8 dots into 1 byte, which reduces the processing load, but only the two values of the foreground color and the background color can be drawn.

The procedure is as follows.

1. Before execution, set the foreground color in R#44 and the background color in R#12
2. Set the parameters (DX, DY, NX, NY, ARG) and write the command code to R#46 to start
3. Thereafter, wait until the TR bit of S#2 becomes 1, and then write the pattern data to R#44 one byte at a time. One byte is 8 dots horizontally, consumed from the MSB (bit 7) side
4. When the CE bit becomes 0, drawing is complete

If NX is not a multiple of 8, the fractional dots also consume one byte from the MSB side, and the remaining bits are discarded. DIX and DIY of ARG are also effective: with DIX=1 the image is flipped horizontally, and with DIY=1 it is flipped vertically (as with LMMC).

## 8.16 LFMM (VRAM → VRAM Font Drawing)

| Item | Description |
|---|---|
| Command code | 10h + logical operation |
| Transfer unit | Dot |
| Parameters used | Font address (R#32–R#34), DX, DY, NX (number of characters), NY, ARG, foreground color (R#44), background color (R#12) |
| Conditions of use | Valid only in extended mode (V58=0) |

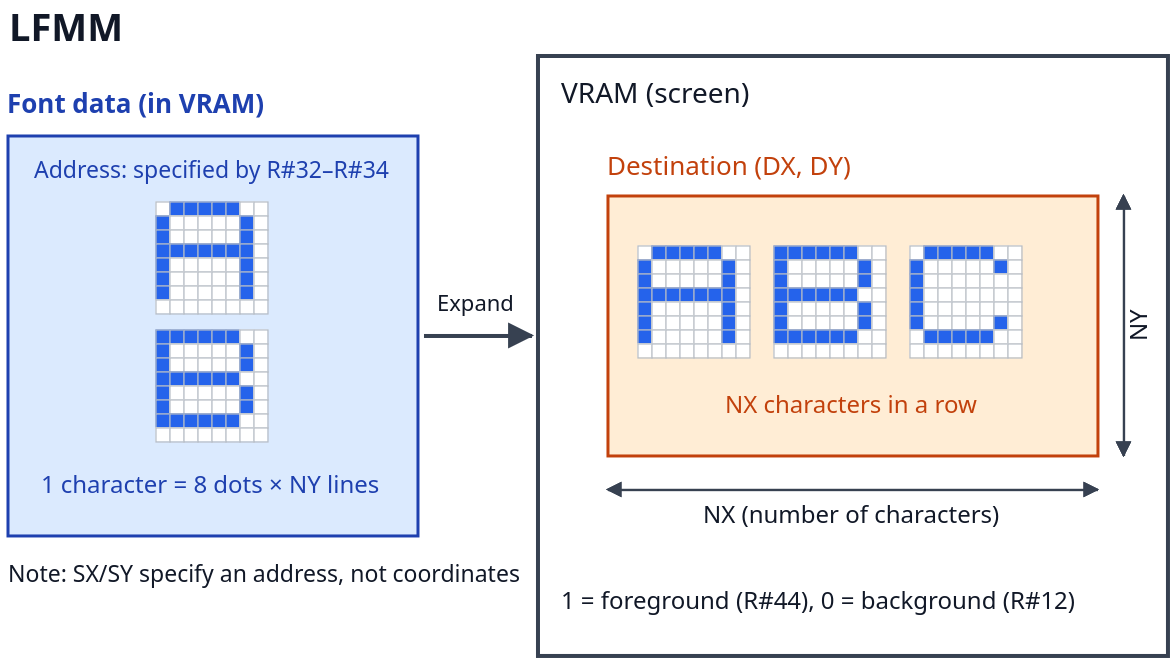



Figure 8.15 Coordinate relationship of LFMM

This command reads font data present in VRAM, expands it, and draws it into another area of VRAM. As with LFMC, the input data is 1 bit/dot, is expanded to the bits/dot of the screen mode when drawn, and a dot whose bit is 1 is drawn in the foreground color (R#44) and a dot whose bit is 0 in the background color (R#12).

In LFMM, SX and SY are used not as coordinates but to specify the address of the font data. Set A7–A0 in R#32, A15–A8 in R#33, and A17–A16 in R#34, and specify the starting address of the font data within the 256KB space as a 3-byte little-endian value. Because R#35 is not used, write 0 to it.

The font data consists of a pattern of 8 dots wide × NY lines per character, with one line being 1 byte (8 dots, consumed from the MSB side). NX specifies not a number of dots but a **number of characters**. Drawing proceeds character by character: after the NY lines of one character have been drawn, it moves on to the next character. At the destination, NX characters of 8×NY dots are laid out. To draw the full-width characters of a kanji ROM, set NY=16 and NX=2, and execute the command once for each character, specifying the address each time. DIX and DIY of ARG are also effective.

This is suited to uses such as displaying character strings, in which a font in VRAM is drawn repeatedly.

## 8.17 STOP (Stop Command)

| Item | Description |
|---|---|
| Command code | 00h |

Interrupts the command that is executing. No parameters are required; it operates simply by writing 00h to R#46. The drawing results up to the point of interruption remain in VRAM.

In extended mode, the codes 0011, 0010, and 0001, which were conventionally treated as STOP, are assigned to LRMM, LFMC, and LFMM respectively (→ Section 8.1.1). In compatible mode, these are also treated as STOP.

## 8.18 Sample Program: Drawing and Copying Figures

The sample program `cmd_demo.asm` is an example that draws figures using only VDP commands in SCREEN 5 (Graphic 4). The flow of processing is: (1) fill the whole screen and a rectangle with HMMV, (2) draw a frame and diagonals on the rectangle with LINE, and (3) copy the completed rectangle to another location with LMMM. The execution result is shown in Figure 8.16. The rectangle duplicated at the lower right directly shows the operation of LMMM.

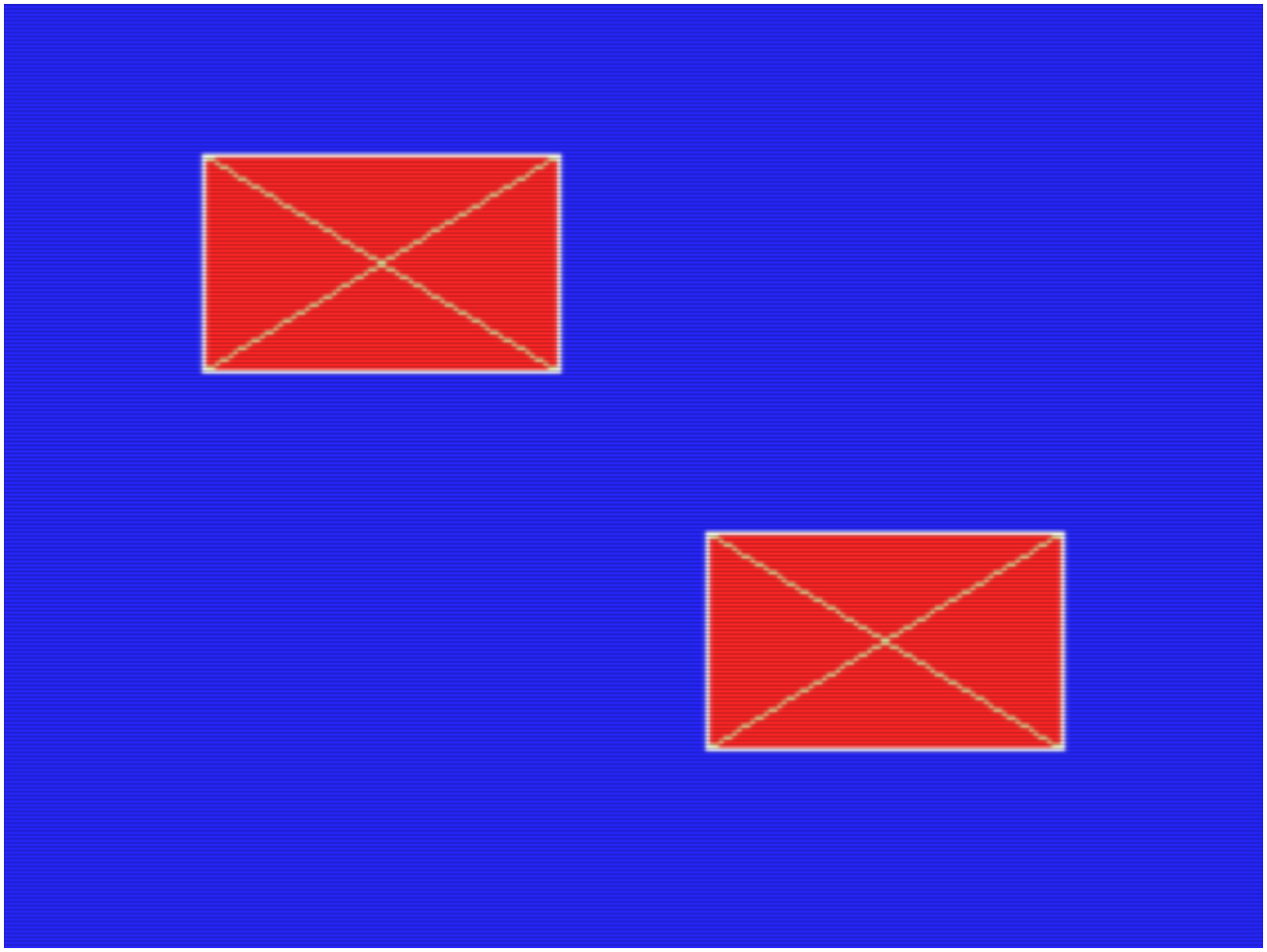


Figure 8.16 Execution result of cmd_demo.asm (HMMV, LINE, LMMM)

The part that fills the rectangle (HMMV) is as follows. Set the parameters in R#36–R#45, and finally write the command code C0h to R#46, and execution begins. Because CLR specifies not a color code but the byte itself that is written, to fill with color 8, set 88h, with 8 in both nibbles (→ Section 8.5).

```
; ---- 2. HMMV: fill a box (32,24)-(111,79) with color 8 (red)
ld      hl, 32          ; DX = 32
ld      b, 36
call        vdp_reg16
ld      hl, 24          ; DY = 24
ld      b, 38
call        vdp_reg16
ld      hl, 80          ; NX = 80
ld      b, 40
call        vdp_reg16
ld      hl, 56          ; NY = 56
ld      b, 42
```

```
call        vdp_reg16
ld      a, 0x88         ; CLR: color 8 in both nibbles
ld      b, 44
call        vdp_reg
ld      a, 0xc0         ; HMMV (ARG is still 0)
ld      b, 46
call        vdp_reg
call        vdp_wait
```

For the diagonals (LINE), the key point is to **put the number of dots of the long side in NX and that of the short side in NY**. Diagonal 1 is a line that advances 79 dots in the X direction and 55 dots in the Y direction from the starting point (32, 24), so the long side is the 79 on the X side. In ARG, specify the direction of the long axis (MAJ) and the direction in which the line extends (DIX, DIY) (→ Section 8.10).

```
; ---- 3. LINE: cross (X) inside the box, color 11
; diagonal 1: (32,24) -> (111,79)
ld      hl, 32          ; DX = 32
ld      b, 36
call        vdp_reg16
ld      hl, 24          ; DY = 24
ld      b, 38
call        vdp_reg16
ld      hl, 79          ; NX = 79 (long side)
ld      b, 40
call        vdp_reg16
ld      hl, 55          ; NY = 55 (short side)
ld      b, 42
call        vdp_reg16
ld      a, 11           ; CLR = 11
ld      b, 44
call        vdp_reg
ld      a, 0x00         ; ARG: MAJ=0 (X is the long axis)
ld      b, 45
call        vdp_reg
ld      a, 0x70         ; LINE, LOP = IMP
ld      b, 46
call        vdp_reg
```

```
call        vdp_wait
```

Diagonal 2 (extending to the left with DIX = 1) and the four sides of the frame are drawn with LINE in the same way (see the source file).

Finally, the rectangle drawn is copied to (144, 120) with LMMM. Set the coordinates and the size of both the source and the destination, and write 90h (LOP = IMP) to R#46 (→ Section 8.8).

```
; ---- 5. LMMM: copy the box (32,24,80x56) to (144,120)
ld      hl, 32          ; SX = 32
ld      b, 32
call        vdp_reg16
ld      hl, 24          ; SY = 24
ld      b, 34
call        vdp_reg16
ld      hl, 144         ; DX = 144
ld      b, 36
call        vdp_reg16
ld      hl, 120         ; DY = 120
ld      b, 38
call        vdp_reg16
ld      hl, 80          ; NX = 80
ld      b, 40
call        vdp_reg16
ld      hl, 56          ; NY = 56
ld      b, 42
call        vdp_reg16
ld      a, 0            ; ARG = 0
ld      b, 45
call        vdp_reg
ld      a, 0x90         ; LMMM, LOP = IMP
ld      b, 46
call        vdp_reg
call        vdp_wait
```

After each command, the CE bit of S#2 is polled to wait for completion (`vdp_wait`, → Section 8.1.3). Because the parameter registers are updated by each command, the required parameters are set again

for each command (→ Section 8.1.6).

## 8.19 Sample Program: Rotated Copy with LRMM

The sample program `lrmm_demo.asm` is an example that draws a 64×64-dot figure in SCREEN 5 (Graphic 4) and copies it to the right, rotated by 30 degrees with LRMM. The execution result is shown in Figure 8.17. On the left is the source figure, at the upper right is the rotated copy, and below is an ordinary LMMM copy for comparison. From the positions of the yellow block at the upper left of the figure and the green block at the lower right, it can be seen that the figure has been rotated 30 degrees clockwise.

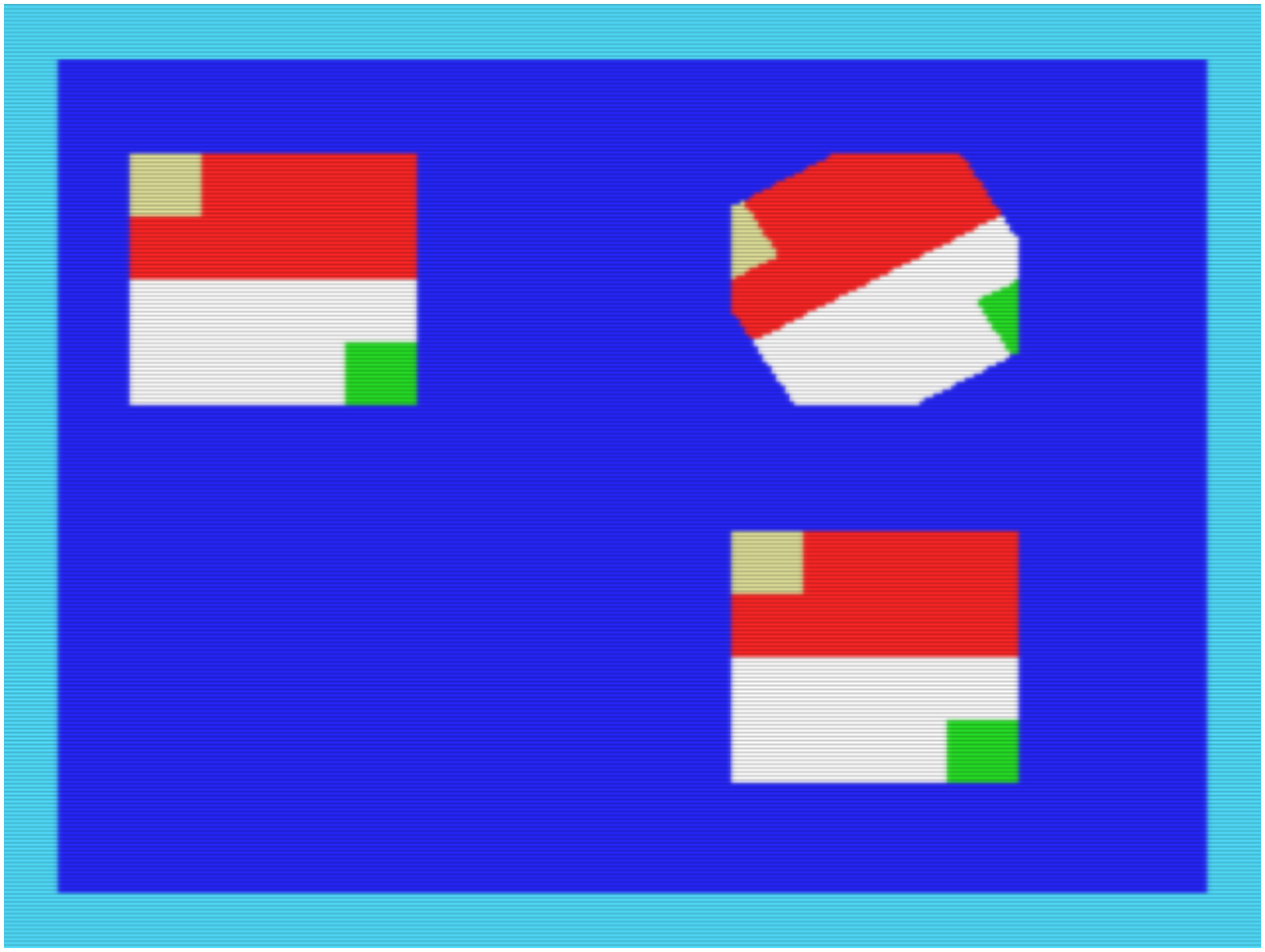


Figure 8.17 Execution result of lrmm_demo.asm (30-degree rotation with LRMM)

Because LRMM is an extended command, switch to extended mode (V58 = 0) before executing it. R#20 and R#21 are locked at reset (LockExtReg = 1), so first write bit7 = 0 to Port#4 to release the lock (→ Section 2.7).

The source figure is drawn by a combination of HMMV rectangle fills (see the source file). The LRMM parameter settings are as follows. **(SX, SY) is not the center of rotation but the reference starting**

**point**, and indicates which position in the source the upper-left dot of the destination refers to. In this example, so that the rotated figure fits in roughly the same position as the source, the value was back-calculated to (36, 13) so that the center of the read parallelogram coincides with the center of the source rectangle, (47.5, 55.5) (→ Section 8.14).

```
; ---- LRMM: copy the pattern to (150,24), rotated by 30 degrees
ld      hl, 36          ; SX = 36 (reference start point)
ld      b, 32
call        vdp_reg16
ld      hl, 13          ; SY = 13
ld      b, 34
call        vdp_reg16
ld      hl, 150         ; DX = 150
ld      b, 36
call        vdp_reg16
ld      hl, 24          ; DY = 24
ld      b, 38
call        vdp_reg16
ld      hl, 64          ; NX = 64
ld      b, 40
call        vdp_reg16
ld      hl, 64          ; NY = 64
ld      b, 42
call        vdp_reg16
ld      a, 4            ; CLR = 4: color used outside the window
ld      b, 44
call        vdp_reg
ld      a, 0            ; ARG = 0
ld      b, 45
call        vdp_reg
```

The rotation vector and window settings are the parts specific to LRMM. For θ = 30 degrees, VX = INT(256 × cosθ) = 221 and VY = INT(256 × sinθ) = 128 (8.8 fixed point; 256 is 1.0). The window is set to the source rectangle itself, (16, 24)–(79, 87) (both ends inclusive), and the four corners of the destination, where the rotation takes the read position outside the rectangle, are drawn as if background color 4, specified by CLR, had been read. For this reason, the figure at the upper right of Figure 8.17 has a shape whose four

corners are cut off in the background color.

```
        ld      hl, 221         ; VX = INT(256 * cos 30 deg) = 221
        ld      b, 47
        call        vdp_reg16
        ld      hl, 128         ; VY = INT(256 * sin 30 deg) = 128
        ld      b, 49
        call        vdp_reg16
        ld      hl, 16          ; WSX = 16
        ld      b, 51
        call        vdp_reg16
        ld      hl, 24          ; WSY = 24
        ld      b, 53
        call        vdp_reg16
        ld      hl, 79          ; WEX = 79
        ld      b, 55
        call        vdp_reg16
        ld      hl, 87          ; WEY = 87
        ld      b, 57
        call        vdp_reg16
        ld      a, 0x30         ; LRMM, LOP = IMP
        ld      b, 46
        call        vdp_reg
        call        vdp_wait
```

## 8.20 Sample Program: Drawing a Character String with LFMM

The sample program `lfmm_demo.asm` is an example that draws the character string "V9968" with LFMM from 8×16-dot font data placed in VRAM, in SCREEN 5 (Graphic 4). The execution result is shown in Figure 8.18. The lower row is the same character string drawn with DIY = 1, flipped vertically.

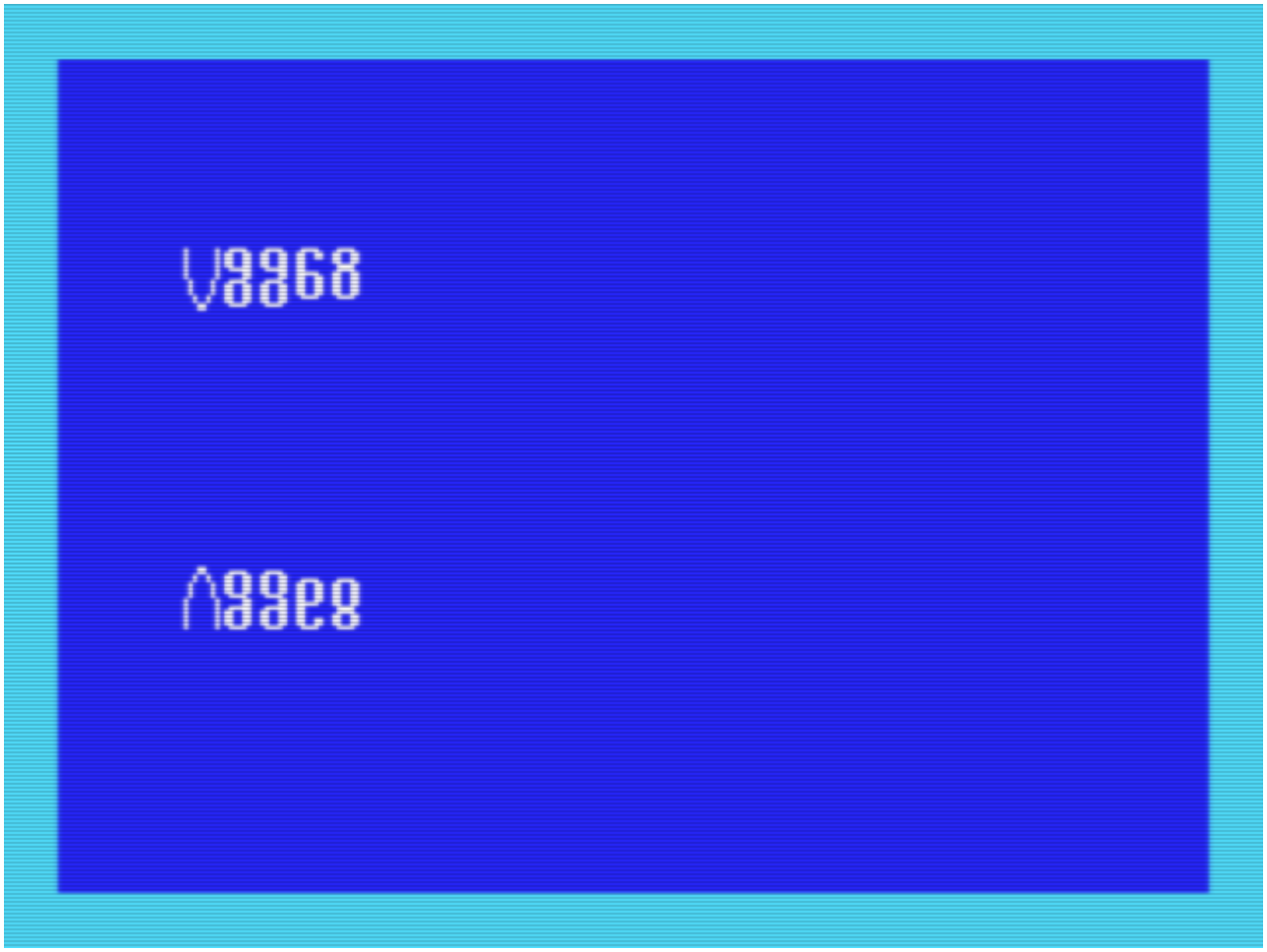



Figure 8.18 Execution result of lfmm_demo.asm (font drawing and vertical flip with LFMM)

The font data is placed at 8000h in VRAM (outside the display page). One character is 1 byte per line (the MSB is the left end) × 16 lines, laid out consecutively in the order of the character string. The first character, "V", is data such as the following (see the source file for the remaining characters).

```
; "V"
db      0x81, 0x81, 0x81, 0x81, 0x81, 0x81, 0x81, 0x81
db      0x42, 0x42, 0x42, 0x42, 0x24, 0x24, 0x18, 0x18
```

The LFMM parameter settings are as follows. **R#32–R#34 are not coordinates but the address of the font data** (3 bytes, from the lowest byte upward), and 0 is written to R#35 because it is unused. **NX is not a number of dots but a number of characters.** Also, set the foreground color in R#44 and the background color in R#12 before executing the command (→ Section 8.16).

```
; ---- LFMM #1: draw "V9968" at (28, 48), upright
ld      hl, 0x8000      ; font address -> R#32 (low), R#33
ld      b, 32
call        vdp_reg16
```

```
ld      hl, 0           ; R#34 = address bits 17..16 = 0
ld      b, 34           ; (R#35 = 0 as well, it is unused)
call        vdp_reg16
ld      hl, 28          ; DX = 28
ld      b, 36
call        vdp_reg16
ld      hl, 48          ; DY = 48
ld      b, 38
call        vdp_reg16
ld      hl, 5           ; NX = 5 (characters!)
ld      b, 40
call        vdp_reg16
ld      hl, 16          ; NY = 16 (font height)
ld      b, 42
call        vdp_reg16
ld      a, 15           ; foreground color -> R#44
ld      b, 44
call        vdp_reg
ld      a, 4            ; background color -> R#12
ld      b, 12
call        vdp_reg
ld      a, 0            ; ARG = 0
ld      b, 45
call        vdp_reg
ld      a, 0x10         ; LFMM, LOP = IMP
ld      b, 46
call        vdp_reg
call        vdp_wait
```

The second time, DIY of ARG is set to 1 and the string is drawn at (28, 144). With DIY = 1, drawing proceeds upward from DY, so DY is the lower end of the flipped character string (the point that the parameters are set again for each command is the same as for the other commands).

# Chapter 9 Geo3D

Geo3D is a 3D graphics support processor implemented in the FPGA of the V9968 cartridge. It executes the perspective projection computation of 3D coordinates and the drawing itself of wireframe / filled-face figures in hardware, so a program on the Z80 side can perform 3D drawing simply by "giving the model data and a camera".

This chapter explains the specification of **Geo3D v2.0**, based on the Geo3D specification document (the documentation of the fork by alexmoncks) and the RTL. The formal version of Geo3D supplied with the V9968 is this v2.0. The differences from the previous version, v1.0.0, are summarized in Section 9.7.

## 9.1 Overview

Geo3D has two operating modes.

- **Immediate mode**: as vertices are sent one at a time, the 2D coordinates obtained by rotation, translation, and perspective projection are returned. Drawing itself is not performed. It is used when you want to leave only the vertex computation of a wireframe to it.
- **Phase-2 engine**: when the data of vertices, edges, and faces is passed together, it executes everything from projection through drawing in one batch. It drives the VDP command engine of the V9968 (Chapter 8) from inside, and performs wireframe drawing by LINE, face fill, and texture application.

In both modes, the coordinate system and the drawing destination are the VRAM coordinate system of the V9968 (the coordinate system of Chapter 8) itself. Consider Geo3D not as an independent display device, but as something that takes over the drawing of the V9968 on its behalf.

## 9.2 Ports and Register Access

The registers of Geo3D are accessed by an index / data method. With the DIP switch of the cartridge, the base of the ports is selected from the 88h family and the 98h family (to avoid I/O conflicts with other hardware).

| Base | Index / status port | Data port |
|---|---|---|
| 88h family | 8Dh | 8Fh |
| 98h family | 9Dh | 9Fh |

The access procedure is as follows.

- Write: write the register number to the index port, and write the value to the data port. For a register with a 16-bit value, write to consecutive indices in the order of the lower byte, then the upper byte.
- Read: write the register number to the index port, and the value can be read from the data port.

In this chapter, registers are written in the form "index 0x40".

## 9.3 Immediate Mode

### 9.3.1 Registers

| Index | Name | Content |
|---|---|---|
| 0x00–0x11 | M00–M22 | Rotation matrix 3×3 (each 16 bits, Q2.14 fixed point, little-endian) |
| 0x12–0x17 | TX, TY, TZ | Translation (each 16-bit signed) |
| 0x18–0x19 | F | Focal length (16-bit unsigned) |
| 0x1A–0x1B | CX | Screen center X coordinate |
| 0x1C–0x1D | CY | Screen center Y coordinate |
| 0x1E–0x1F | ZNEAR | Z value of the near clipping plane |
| 0x20–0x21 | W | Screen width (for overflow determination) |

| Index | Name | Content |
|---|---|---|
| 0x22–0x23 | H | Screen height (for overflow determination) |
| 0x24–0x29 | VX, VY, VZ | Vertex input stream (each 16-bit signed). Writing to 0x29 starts the computation |
| 0x30–0x31 | SX | X coordinate of the projection result (read) |
| 0x32–0x33 | SY | Y coordinate of the projection result (read) |
| 0x34–0x35 | Z | Z value in the camera coordinate system (read) |
| 0x36 | FLAGS | Flags (read) |

### 9.3.2 Conversion and Projection

A vertex (VX, VY, VZ) is first converted into the camera coordinate system by the rotation matrix M and the translation T.

```
X = M00 · VX + M01 · VY + M02 · VZ + TX
Y = M10 · VX + M11 · VY + M12 · VZ + TY
Z = M20 · VX + M21 · VY + M22 · VZ + TZ
```

It is then converted into screen coordinates by perspective projection.

```
SX = CX + X · F / Z
SY = CY − Y · F / Z
```

When 6 bytes are written from index 0x24 in the order VX, VY, VZ, the computation starts, triggered by the writing of the last byte (0x29). Only one vertex can be processed at a time. The result is read out from 0x30. **Reading 0x30 first latches the result**, so read the 7 bytes 0x30–0x36 consecutively (on the Z80, the INIR instruction can be used. On the input side, 6 bytes are sent with OTIR).

For a vertex whose Z is less than ZNEAR (or is 0), the NEAR bit of FLAGS is set. In this case SX / SY are also computed, but the values cannot be trusted, so skip drawing that uses that vertex.

The bit layout of FLAGS is as follows.

| Bit | Name | Meaning |
|---|---|---|
| 0 | NEAR | Z < ZNEAR (too close to the camera) |
| 1 | OVFX | SX is outside the range of the screen width |
| 2 | OVFY | SY is outside the range of the screen height |
| 3 | BUSY | Computation in progress |
| 4 | SX<0 | SX is negative |
| 5 | SX>=W | SX is at least the screen width |
| 6 | SY<0 | SY is negative |
| 7 | SY>=H | SY is at least the height |

The time taken by the computation is a maximum of 49 clocks per vertex (about 2.3 µs when operating at 21.477MHz). In theory, up to about 420,000 vertices per second can be processed, but in an actual program the transfer time of the OTIR / INIR on the Z80 side becomes dominant.

## 9.4 Phase 2 Engine

The phase-2 engine receives the data of the entire scene via the registers, and executes projection and drawing in one batch.

### 9.4.1 Registers

| Index | Name | Content |
|---|---|---|
| 0x40 | VADDR | Storage destination of the vertex data (address in the internal RAM) |
| 0x41 | EADDR | Storage destination of the edge data |
| 0x42 | NVERT | Number of vertices |

| Index | Name | Content |
|---|---|---|
| 0x43 | NEDGE | Number of edges |
| 0x44 | COLOR | Color of the wireframe |
| 0x45 | LOP | Logical operation at drawing time (the same specification as in Chapter 8) |
| 0x46–0x47 | YPAGE | Y offset of the drawing destination page (16 bits) |
| 0x48 | CTRL/STATUS | Write: bit0 = RUN start, bit1 = face fill enable, bit2 = texture enable. Read: bit0 = busy |
| 0x49 | ID | Version identification. Returns 0xFF in v1.0.0, 0x20 in v2.0 |
| 0x4A–0x4B | SKIPPED | Number of edges skipped (16 bits, read) |
| 0x4C–0x4D | DRAWN | Number of edges drawn (16 bits, read) |
| 0x4E–0x4F | CULLED | Number of faces culled (16 bits, read) |
| 0x50 | Vertex stream | Write-only (9.4.2) |
| 0x51 | Edge stream | Write-only (9.4.2) |
| 0x52 | Face stream | Write-only (9.5) |
| 0x53 | Texture coordinate stream | Write-only (9.6) |
| 0x58 | FADDR | Storage destination of the face data |
| 0x59 | NFACE | Number of faces |

| Index | Name | Content |
|---|---|---|
| 0x5A–<br>0x5F | LX, LY, LZ | Light source direction vector (each 16-bit signed) |

The registers added in v2.0 (VFIRST and so on) are summarized in Section 9.7.

### 9.4.2 Loading the Data

A scene is loaded in three kinds of streams. In each case, first decide the storage destination with VADDR / EADDR / FADDR, and then write consecutively to the corresponding stream port. Each write automatically advances the internal address.

- **Vertex stream (0x50)**: 5 bytes per vertex. X and Y are 16 bits (little-endian), and Z is the upper byte only (the lower 8 bits of Z are treated as 0)
- **Edge stream (0x51)**: 2 bytes per edge. The vertex indices of the start point and the end point

The numbers of vertices and edges are set in NVERT and NEDGE (in v1.0.0, up to 255 each. For the extension in v2.0, see Section 9.7).

A scene with only vertices and edges is drawn as a wireframe. The drawing color is specified by the COLOR register, and the drawing destination page is selected by YPAGE.

### 9.4.3 Execution and Waiting for Completion

Writing the RUN bit (bit0) to CTRL (0x48) starts execution. During execution, busy (bit0 on reading) is 1.

**Care is required when waiting for completion.** Geo3D draws using the VDP command engine, but between commands there are moments when the CE bit of S#2 becomes 0. "CE = 0" alone does not mean that the processing of Geo3D is complete. Always confirm **that Geo3D's busy is 0, and that CE is 0**.

Also, **while busy is 1, you must not write from the Z80 side to the VDP command registers (R#32–R#58) or to Geo3D's setting registers**. Geo3D itself is using them, so they would conflict.

The breakdown of the processing result can be checked with the counters SKIPPED (edges that were not drawn, for example because they were outside the screen), DRAWN (edges drawn), and CULLED (faces

culled as back-facing).

### 9.4.4 Relationship with the VDP Command Engine

Geo3D draws by issuing commands directly to the parameter registers of the VDP commands (the equivalent of R#36–R#46). Each edge of the wireframe is drawn by a LINE command, and texture application is drawn by an LRMM command (→ Section 8.14). The engine watches for a timing at which the VDP command is free (CE = 0) and feeds in the next command. Because of this mechanism, while Geo3D is executing, the Z80 also cannot use VDP commands directly (→ Section 9.4.3).

## 9.5 Face Fill and Shading

Setting bit1 of CTRL enables face fill drawing using face data.

11 bytes per face are written to the face stream (0x52).

| Byte | Content |
|---|---|
| 0–2 | Vertex indices A, B, C |
| 3 | BASE color (bit7 is the flag of a texture face, 9.6) |
| 4–9 | Face normal vector (NX, NY, NZ, each 16-bit signed) |
| 10 | strict flag |

The shading is determined from the dot product of the face normal N and the light source direction L (the LX, LY, LZ registers). The brightness stage, level, is 7 stages from 0 to 6, and the drawing color becomes BASE + level. The way to use it is to prepare in advance, on the palette, a gradient (ramp) of 7 colors starting from BASE. A face whose dot product is negative (a face that the light does not strike) becomes the darkest stage.

The drawing order is the painter's method. Faces are drawn in order from those with the larger average Z value (those further back), so of overlapping faces the one in front is drawn on top. The edges of the wireframe have no such reordering, and are drawn in the order in which they were registered in the scene.

## 9.6 Textures

Setting bit2 of CTRL enables texture application to faces. A face for which bit7 of the BASE color in the face stream is set is treated as a texture face.

Texture coordinates are loaded into the texture coordinate stream (0x53), in accordance with the order of the faces, at 2 bytes per corner (U, V). The arrangement of the texture image is specified by the following registers.

| Index | Name | Content |
|---|---|---|
| 0x60–0x61 | TEXX | X coordinate of the texture origin |
| 0x62–0x63 | TEXY | Y coordinate of the texture origin |
| 0x64 | TSTRIDE | Number of bytes per row of the texture |
| 0x65 | TADDR | Base of the texture (page offset. It is added to the Y coordinate of the texture) |

The texture is applied by an affine transformation. The engine decomposes a face into strips row by row, and draws each strip by transferring it from VRAM to VRAM with an LRMM command.

## 9.7 Extensions in v2.0

In v2.0, the scale of the scenes that can be handled and the way scenes are reused have been extended. From a program, the ID register (0x49) is read, and if 0x20 is returned it can be judged to be v2.0 (in v1.0.0, 0xFF is returned).

### 9.7.1 1024-Entry Support and Extended Mode

Setting bit0 of the XCTRL register (0x6F) selects the extended mode, in which up to a maximum of **1023** vertices, edges, and faces can be handled (in the v1.0.0-compatible format, up to 255). Registers for the upper bits have been added to the registers that count the numbers.

| Index | Name | Content |
|---|---|---|
| 0x54–0x55 | VFIRST | Start position of the vertex ring (16 bits) |

| Index | Name | Content |
|---|---|---|
| | | |
| 0x56–0x57 | FFIRST | Start position of the face ring (16 bits) |
| 0x66–0x67 | EFIRST | Start position of the edge ring (16 bits) |
| 0x68 | VADDR_H | Upper bits of VADDR |
| 0x69 | EADDR_H | Upper bits of EADDR |
| 0x6A | NVERT_H | Upper bits of NVERT |
| 0x6B | NEDGE_H | Upper bits of NEDGE |
| 0x6C | FADDR_H | Upper bits of FADDR |
| 0x6D | NFACE_H | Upper bits of NFACE |
| 0x6E | TADDR_H | Upper bits of TADDR |
| 0x6F | XCTRL | bit0: extended mode. Bits 7–4: address width (AW). 10 for 1024 entries |

In extended mode, the stream formats also change.

- Vertex stream: 6 bytes per vertex (Z can also be specified with the full 16 bits)
- Edge stream: 3 bytes per edge (A lower, a byte packing the upper bits of A / B, B lower)
- Face stream: 12 bytes per face (to the conventional 11 bytes, a byte packing the upper bits of the index and the BASE color is added)

### 9.7.2 Resident Scenes and the Ring Buffer

VFIRST, FFIRST, and EFIRST specify the start position of the data on the internal RAM. Each data sequence is treated as a ring, and when it reaches the end of the RAM it wraps around to the beginning. By this means, drawing can be done with the data kept resident in the internal RAM, without sending the scene again each time.

Furthermore, the projection results are retained across RUNs. If it is only a matter of drawing the same scene again without recalculating the vertices, executing RUN with NVERT = 0 allows redrawing (repainting) using the projection result of the previous time. When moving only the camera, the vertices must be loaded again.

## 9.8 BASIC API (G3BASIC)

For Geo3D, an extension library for use from BASIC, "G3BASIC", is provided. It is a group of extended instructions called by CALL statements of MSX-BASIC, and without directly hitting the registers, the loop of model definition, placement, camera, and drawing can be written as a BASIC program. The main instructions are as follows.

| Category | Instruction | Role |
|---|---|---|
| Session | G3INIT, G3END | Initialization and termination of Geo3D |
| Data | G3DATA | Loading of model data (from DATA statements) |
| Data | G3PAL, G3RAMP | Palette setting, generation of a color ramp for shading |
| Placement | G3OBJ | Assignment of an object to a slot |
| Placement | G3POS | Setting of the position |
| Placement | G3ROT, G3SPIN | Setting of the rotation angle, continuous rotation |
| Placement | G3SIZE | Enlargement / reduction (scaling) |
| Placement | G3COLOR, G3STYLE | Color and drawing style (wireframe / face fill, etc.) |
| Camera | G3CAM | Setting of the camera position and orientation |
| Drawing | G3FRAME | Execution of drawing for one frame |

In addition to these, there are extended instructions such as center-of-gravity calculation (G3CENT), inertia (G3INERT), looking at a target (G3LOOK / G3AIM), the light source (G3LIGHT), fog (G3FOG), texture specification (G3TEX), and display control (G3SHOW / G3HIDE, and others).

As models, built-in models such as polyhedra and alphabetic characters (numbers 0–15) are provided, and by giving the data of vertices, edges, and faces in DATA statements, models of your own making (numbers 16–31) can also be defined.

As a guide to performance, on a 3.58MHz Z80 an object of 255 faces can be drawn at about 20 fps (according to the G3BASIC documentation).

## 9.9 Sources for This Chapter

This chapter was written by collating the Geo3D documents by alexmoncks (the README, the BASIC API specification, and the v2.0 specification notes) with the RTL on the V9968 cartridge side. If there is a point where the documents and the RTL disagree, the RTL is taken as correct. The formal version of Geo3D supplied with the V9968 is v2.0, and this chapter describes v2.0 as the formal specification.

## 9.10 Sample Program: The Official Demo ROM

The official demo ROM of Geo3D is contained in `geo3d/rom/` of alexmoncks's V9968_Cartridge fork (branch `geo3d-v2.0`). It is a ROM that plays six kinds of demos in a loop, and Figure 9.1 and Figure 9.2 show the results of generating the version that places the V9968 at port 98h with the build script (`build_rom.py --base 0x98`) (GEO3D_98.ROM) and running it on an openMSX with Geo3D emulation (machine C-BIOS_V9968_JP, extension geo3d). Selecting "1" (English) in the language menu immediately after starting starts the demo, and the space key advances to the next demo.

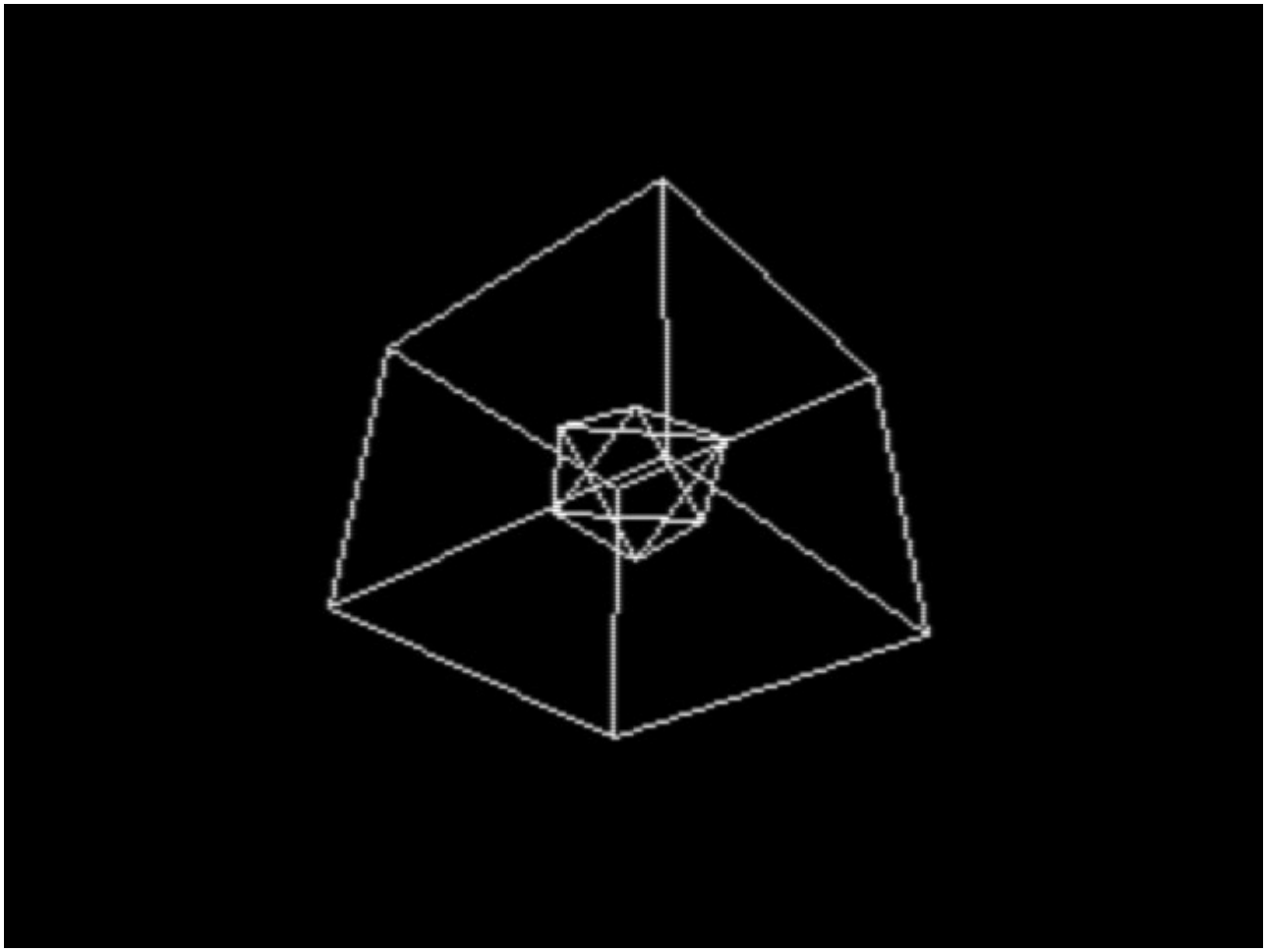


Figure 9.1 Execution result of the official demo ROM (wireframe: cube and octahedron)

Figure 9.1 is the wireframe demo, in which a cube and an octahedron (together, 14 vertices and 24 edges) rotate. The work on the host (Z80) side is only the loading of the model, once at the beginning; after that it only sends about 30 bytes per frame. Erasure of the back page (HMMV) and switching of the display page (R#2) are normal VDP operations, and to Geo3D it sends the drawing destination page (YPAGE), a precomputed 24-byte matrix, and RUN. The frame loop of the MSX-DOS program from which this demo originates (`geo3d/z80/geo3d_demo.asm`; the demo ROM contains the port output stream obtained by executing this group of programs) is as follows (the port numbers are for the case of the cartridge's DIP switch set to 88h; in the 98h version, 10h is added to each port).

```
; ---- matrix for this frame + RUN ----
xor a
out (GEO_IDX), a            ; index 0x00 = M00
ld bc, 24 * 256 + GEO_DAT   ; B = 24 bytes, C = port
otir
ld a, 0x48
out (GEO_IDX), a
```

```
        ld a, 1
        out (GEO_DAT), a            ; RUN

geo_wait:
        in a, (GEO_IDX)
        rrca
        jr c, geo_wait              ; bit0 = RUN busy
```

The matrix is prepared as a table for one revolution, and the Z80 performs no coordinate calculation at all. The conversion of the vertices, the clipping of the edges, and the generation of the LINE command sequence are all done by Geo3D. Without Geo3D, the Z80 would have to write about 312 VDP command register values per frame, in addition to the vertex calculation (according to the demo documentation).

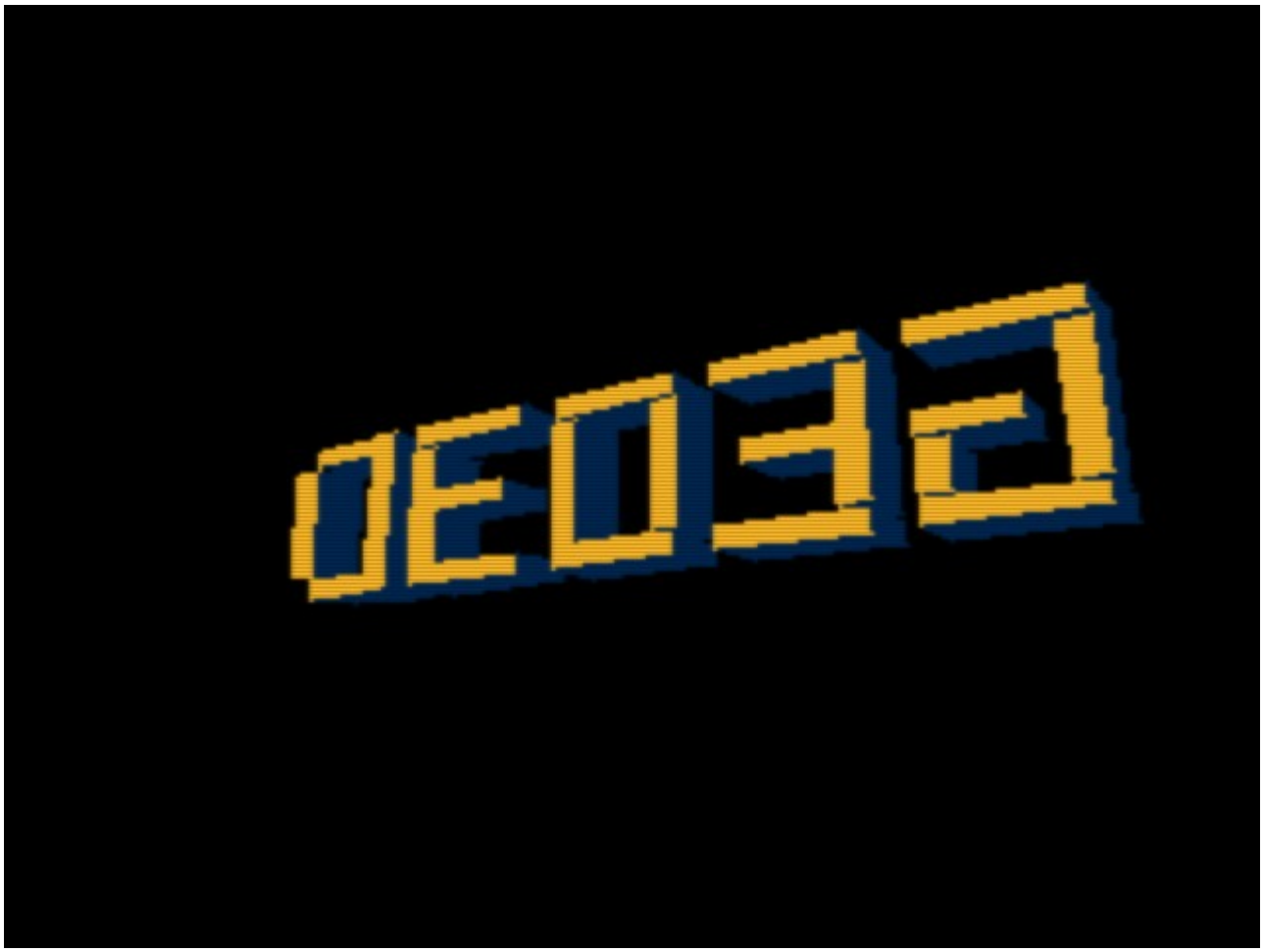


Figure 9.2 Execution result of the official demo ROM (face fill: the three-dimensional letters "GEO3D")

Figure 9.2 is the face-fill demo. Removal of back faces, shading by the light source direction, sorting of the drawing order from the far faces, and filling of the faces by horizontal LINEs are handled by Geo3D, and the amount transmitted from the host side is unchanged from the wireframe case, at about 30 bytes

per frame. After this, demos of face fill with textures, pan / zoom, and a fly-in follow.

# Chapter 10 Interrupts

The V9968 has an interrupt facility for informing the CPU that screen display and VDP command processing are proceeding in parallel with program execution. This chapter describes the interrupt sources, how to enable and clear each source, and how to use the interrupt control port (Port#4).

The bit configurations of the registers and flags related to interrupts were described in Chapter 3 and Chapter 4. This chapter focuses on "how to use interrupts".

## 10.1 Overview of the Interrupt Sources

The V9968 has the following three interrupt sources.

| Source | Occurrence condition | Flag | Enable bit | Indication on Port#4 |
|---|---|---|---|---|
| Frame interrupt | At the timing of vertical synchronization (once per frame) | F in S#0 | IE0 in R#1 | FI |
| Line interrupt | When the display line reaches the line specified by R#19 | FH in S#1 | IE1 in R#0 | LI |
| Command completion interrupt | When execution of a VDP command is completed | (CI on Port#4) | CEIE in R#20 | CI |

The interrupt output (INT) is the OR, in negative logic, of the condition "the flag is set and the corresponding enable bit is 1" for these three sources. While any one of the conditions holds, the interrupt output is active. Therefore, within the interrupt handler, check which sources remain by reading Port#4, and clear the sources that have been handled. If a source is left remaining, an interrupt will occur again immediately after returning from the interrupt.

Note that sprite collision (C in S#0) and exceeding the maximum displayable number (5S in S#0) are **not interrupt sources**. These are detected by polling the status registers (see Chapter 4 and Chapter 6).

## 10.2 Frame Interrupt

The frame interrupt occurs once per frame at the timing of vertical synchronization. It is the most basic interrupt, used when screen updates are to be aligned with the blanking interval, or as the reference for processing at a fixed period.

- Enabling: Set IE0 (bit5) of R#1 to 1.
- Flag: F (bit7) of S#0. Even if the interrupt is not enabled, F is set at the timing of vertical synchronization.
- Clearing: Done in either of the following ways.
- Read S#0 (at this time, C and 5S are also cleared at the same time; see Chapter 4).
- Write FI (bit0) = 1 to Port#4.

F remains set until it is read out. Because the flag of a frame interrupt that occurred while IE0 = 0 also remains, setting IE0 to 1 afterwards may cause an interrupt at that point. It is recommended to clear the flag before enabling.

## 10.3 Line Interrupt

The line interrupt occurs at the timing when the display line reaches the specified line number. It is used for so-called raster processing, such as switching the palette or changing the scroll settings in the middle of the screen.

- Setting the line number: Set the display line number at which the interrupt is to occur in R#19 (Interrupt Line).
- Enabling: Set IE1 (bit4) of R#0 to 1.
- Flag: FH (bit0) of S#1. Even if the interrupt is not enabled, FH is set when the specified line is reached.
- Clearing: Done in any of the following ways.
- Read S#1.
- Write LI (bit1) = 1 to Port#4.
- Write to R#0 or R#19 (FH is cleared as a side effect).

Because writing R#19 clears FH, when multiple line interrupts are to be used within one frame, writing "the next line number to R#19" within the handler clears the flag and sets up the next occurrence at the

same time.

Also note the relationship between the line number and vertical scrolling. Normally, the comparison of the line number of R#19 is affected by the offset of vertical scrolling (R#23). Setting ILNS (bit2) of R#20 to 1 makes the line number unaffected by R#23, and it is interpreted as an absolute position with the top of the screen as 0 (R#19 = 0 is the first line of the screen). When an interrupt is wanted at a fixed position on the screen while moving the vertical scroll, use ILNS.

## 10.4 Command Completion Interrupt

The command completion interrupt occurs at the timing when execution of a VDP command is completed. It is used when the end of a command is to be awaited by interrupt instead of polling the CE bit of S#2 (for the command execution model, see Chapter 8).

- Enabling: Set CEIE (bit6) of R#20 to 1.
- Checking the source: CI (bit2) of Port#4 indicates that the command completion source is occurring.
- Clearing: Write CI (bit2) = 1 to Port#4.

There are two points to note.

1. **CEIE is in the extended register R#20.** While LockExtReg (bit7 of Port#4) is 1, writes to R#20 are ignored, so the lock must be released on Port#4 before setting CEIE (for the procedure, see Section 10.5 and Chapter 2). Note that the use of CEIE itself is possible even when not in extended mode (V58 = 0). Releasing the lock and switching the mode are separate operations.
2. **CE in S#2 is not a flag but an in-progress indication.** CE is a bit that is 1 while a command is executing, and reading it does not clear it. Clearing of the completion interrupt source must always be done by writing to Port#4.

## 10.5 Checking and Clearing Sources with Port#4

Port#4 is the port that handles the checking and clearing of interrupt source states and the locking of the extended registers (LockExtReg) (see Chapter 2).

## Reading

| bit | Name | Meaning |
|---|---|---|
| 7 | LockExtReg | Lock state of the extended registers (1 at reset) |
| 6–3 | — | 0 |
| 2 | CI | 1 if the command completion interrupt source is occurring |
| 1 | LI | 1 if the line interrupt source is occurring |
| 0 | FI | 1 if the frame interrupt source is occurring |

When reading, the occurrence state of the sources themselves is obtained regardless of the state of the enable bits (IE0 / IE1 / CEIE). In addition to identifying sources in an interrupt handler, it can also be used by programs that do not use interrupts, for example to detect frame synchronization.

## Writing

| bit | Meaning |
|---|---|
| 7 | Write to LockExtReg (0 releases the lock, 1 locks) |
| 2 | Writing 1 clears the command completion interrupt flag |
| 1 | Writing 1 clears the line interrupt flag |
| 0 | Writing 1 clears the frame interrupt flag |

Clearing of flags applies only to "the bits to which 1 was written"; writing 0 has no effect. Multiple sources can also be cleared at the same time. However, bit7 is a write to LockExtReg itself. If only source clearing is to be performed, write the same value as the current lock state to bit7 (the lock state can be checked by reading). Carelessly changing bit7 will change the protection state of the extended registers.

## Procedure for Unlocking the Extended Registers

To set R#20 / R#21, LockExtReg must be released. The procedure is as follows (see also Chapter 2).

1. Write a value with bit7 = 0 to Port#4 (LockExtReg becomes 0 and the lock is released).
2. Write the necessary settings to R#20 / R#21 (when switching to extended mode, set V58 of R#21 to 0).
3. As necessary, write a value with bit7 = 1 to Port#4 to lock again.

The lock is a mechanism for protecting the mode settings from unintended writes immediately after reset. It is recommended to lock again after finishing the use of the extended functions.

## 10.6 Flow of Using Interrupts

The overall flow is shown with an example that uses the frame interrupt and the command completion interrupt together.

1. **Initialization**: Read Port#4, and if any source flags remain, clear them by writing to Port#4. When using the command completion interrupt, release LockExtReg, set CEIE of R#20 to 1, and lock again.
2. **Enabling**: Set IE0 of R#1 to 1 to enable the frame interrupt (when using the line interrupt, set R#19 and then set IE1 of R#0 to 1).
3. **Handler**: When an interrupt occurs, read Port#4 and identify the sources that are set. Perform the processing for each source, and clear each handled source by writing to Port#4 (or by reading the corresponding status register).
4. **Waiting**: When the main processing side wants to wait for the completion of a command, it starts the command and then enters an interrupt wait. When waiting by polling, confirm that CE of S#2 becomes 0 (see Chapter 8). With either method, after checking, clear the command completion interrupt source (CI) so that it does not remain set.

A design that does not use interrupts is also possible. Frame synchronization can be replaced by polling F of S#0, display period detection by the VR / HR bits of S#2, and command completion by polling CE of S#2 (see Chapter 4).

## 10.7 Sample Program: Screen Split Using a Line Interrupt

The sample program `line_demo.asm` is an example that polls the line interrupt flag (FH) and switches the palette between the upper and lower parts of the screen. In SCREEN 5 (Graphic 4), the 16 colors of the palette are drawn as vertical bands, and R#19 is set to line 106, at the center of the screen. The execution

result is shown in Figure 10.1. It can be seen that the color of each band differs between the upper half and the lower half (because the border color also goes through the palette, its color also differs between the top and bottom).

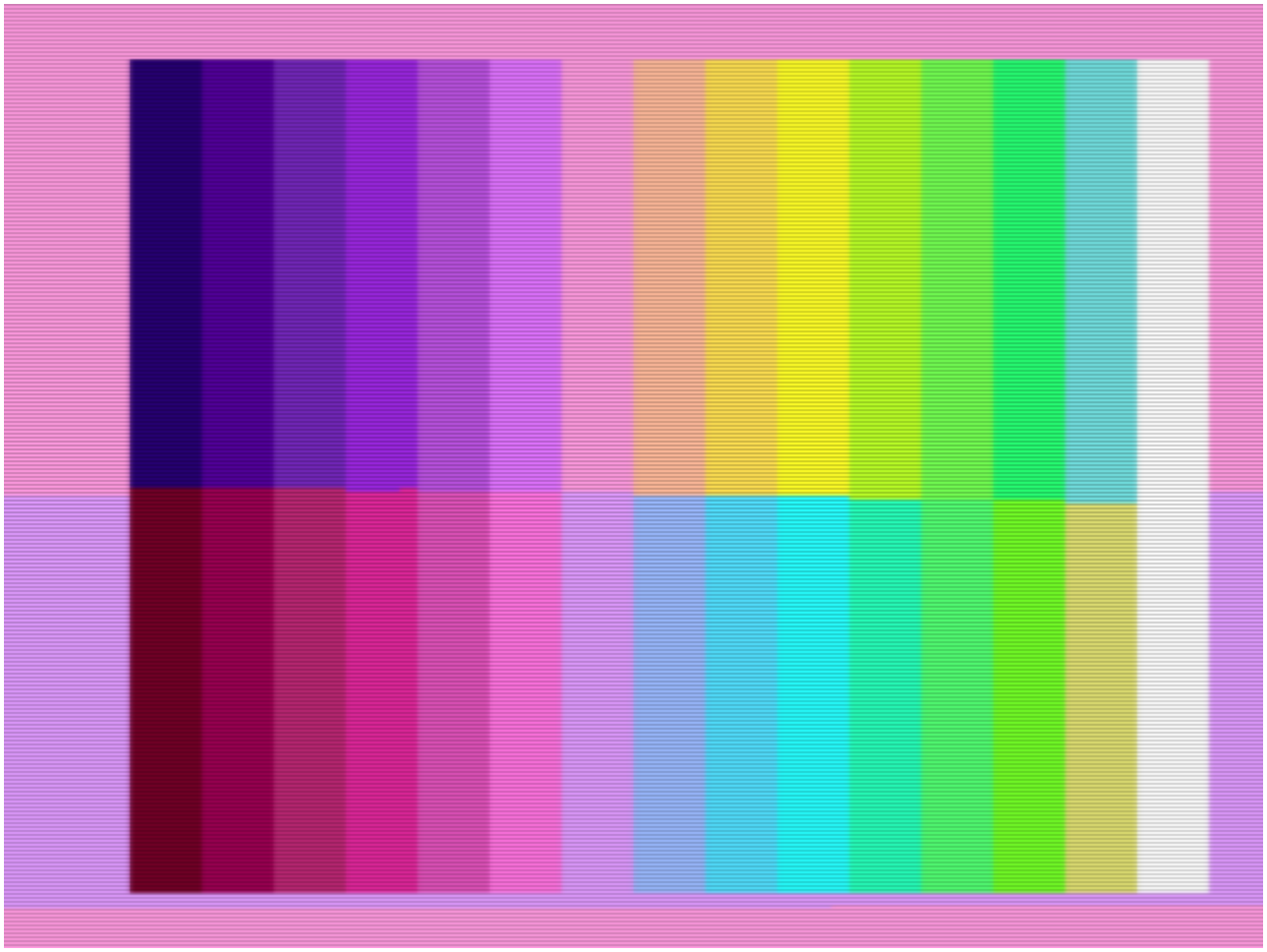


Figure 10.1 Execution result of line_demo.asm (palette switching at line 106)

The main loop is as follows. It waits for the start of a frame with the F bit of S#0 and writes palette A. Next, it switches the status selection to S#1, waits for the FH bit to be set (the raster reaches the line of R#19), and writes palette B (palette A with the R and B components exchanged).

```
split_loop::
        ; wait for the V-blank flag F (S#0 bit7)
wait_f:
        in      a, [0x99]
        bit     7, a
        jr      z, wait_f
        ; top half: palette A
        ld      hl, palette_a
```

```
        call        apply_palette
        ; select S#1 and wait for the line-interrupt flag FH (bit0)
        ld      a, 1
        ld      b, 15
        call        vdp_reg
wait_fh:
        in      a, [0x99]
        bit     0, a
        jr      z, wait_fh
        ; bottom half: palette B
        ld      hl, palette_b
        call        apply_palette
        ; restore the S#0 selection
        xor     a, a
        ld      b, 15
        call        vdp_reg
        jr      split_loop
```

The FH flag is set at the point when the raster reaches the specified line, even if the line interrupt is not enabled with IE1 (→ Section 10.3). Therefore, screen splitting can be realized by polling alone, without preparing an interrupt handler. With a cartridge ROM, page 0, which contains the vector at 0038h, is in the BIOS area and cannot be replaced, which is another reason for choosing the polling method.

Note that, because it takes several lines from FH being set until the writing of all 16 palette entries is finished, the actual switching position is several lines below the R#19 specification. For the palette data and the band drawing, see the source file.

## 10.8 Notes for Environments with Two VDPs

The I/O base address and the way the vertical synchronization interrupt is received differ between the case where the V9968 is built into the main unit and the case where it is connected as a second VDP (2nd VDP), for example on a cartridge.

| Configuration | I/O base address | Hook that receives the vertical synchronization interrupt |
|---|---|---|
| Built-in VDP | 98h | H.TIMI |

| Configuration | I/O base address | Hook that receives the vertical synchronization interrupt |
|---|---|---|
| | | |
| 2nd VDP | 88h | Must also be handled in H.KEYI |

For the built-in VDP, the base address is 98h, and 98h is also stored in 0006h / 0007h of the BIOS work area (see Chapter 2). The vertical synchronization interrupt can be received as usual via the H.TIMI hook through the BIOS interrupt processing.

For the 2nd VDP, the base address is 88h. Because 0006h / 0007h of the work area remain at the value of the built-in VDP (98h), a program that accesses the 2nd VDP should specify 88h directly. Also, because the vertical synchronization interrupt of the 2nd VDP cannot be received in H.TIMI, it must also be handled in the H.KEYI hook. Checking and clearing of sources should be done with the 2nd VDP's own Port#4 (88h + 4 = 8Ch) and status registers.

> **Note:** The vertical synchronization of the built-in VDP and the 2nd VDP does not necessarily coincide. The timing at which H.TIMI is called is aligned with the vertical synchronization of the built-in VDP, and there is no guarantee that the 2nd VDP has entered its vertical blanking interval at that point. If screen updates on the 2nd VDP side are to be aligned with the blanking interval, do not use H.TIMI as a guide; instead, handle the 2nd VDP's own frame interrupt in H.KEYI. The interrupt source must be cleared either by using Port#4 or by reading the status registers.
>
> The same applies to line interrupts: the built-in VDP and the 2nd VDP do not necessarily display the same line, so even if the line interrupt of the built-in VDP is used to control the 2nd VDP, or the other way around, note that there is no guarantee that the control will take place at the specified line.

# Chapter 11 VRAM Access and Timing

This chapter brings together, from the viewpoint of timing, the material on the structure of VRAM and on the model and restrictions that apply when the CPU accesses VRAM, which was distributed between Chapter 2 (access procedures) and Chapter 8 (VDP commands).

What this chapter describes is limited to the access model visible to the programmer and to the restrictions that have been established in the implementation. Detailed timing specifications, such as the CPU access window for each screen mode (at what timing during the display period access is guaranteed), are listed as items to be confirmed at the end of this chapter.

## 11.1 VRAM Organization

The VRAM address space of the V9968 is 18 bits (A17–A0), for a maximum of 256KB. The range that can actually be used changes with the operating mode.

| Mode | Usable VRAM | Treatment of addresses |
|---|---|---|
| V9958 compatible mode (V58 = 1) | Equivalent to 128KB | A17 is masked in various places and treated as 0 |
| Extended mode (V58 = 0) | 256KB | The full range of A17–A0 is valid |

The places where A17 is masked in compatible mode are as follows (see Chapter 3).

- An A17 written to the table base address registers R#2 / R#4 / R#6 / R#10 / R#11 is retained as 0.
- The A17 of R#14, which specifies the upper bits of the address for CPU access, is also masked.
- In addition, the moment a 1 is written to R#21 to return to compatible mode, bit17 of all five table base addresses is cleared to 0. If a program that was using the upper pages in extended mode returns to compatible mode, set up the table layout again.

The same applies in connection with sprites and commands: in compatible mode, the upper bits of coordinates and addresses are treated on the assumption of 128KB (for example, the number of valid bits of DY / NY in R#39 / R#43. See Chapter 3).

### Physical Layout of VRAM (DRAM Mapping)

Physically, VRAM consists of two DRAMs (DRAM0 and DRAM1, 128KB each), and which DRAM a logical address (the VRAM address seen by the program) is placed in switches according to the screen mode.

| Mapping | Conditions of application | Logical address → DRAM |
|---|---|---|
| Flat | SCREEN 0–6 (and, in SCREEN 7 and above, when in sprite mode 3) | 0000h–FFFFh → DRAM0, 10000h–1FFFFh → DRAM1 |
| Interleave | SCREEN 7 and above (when not in sprite mode 3) | Even addresses (0000h, 0002h, 0004h, ⋯, 1FFFEh) → DRAM0, odd addresses (0001h, 0003h, 0005h, ⋯, 1FFFFh) → DRAM1 |

In SCREEN 7 and above, the interleave is used both to preserve compatibility with earlier chips and to distribute accesses between the two DRAMs in order to secure bandwidth.

In sprite mode 3 (SP3, R#20 bit3), on the other hand, the pattern data must be read out as a contiguous address range, so the layout returns to flat even in SCREEN 7 and above.

> **Note:** In a screen mode of SCREEN 7 or above, switching SP3 changes the physical arrangement of the VRAM data. The logical addresses handled by the program do not themselves change, but data written under one mapping before the switch appears in a different arrangement after the switch. For example, if you switch to sprite mode 3 after placing a bitmap or patterns in SCREEN 8 (or do the reverse), place the required data again to suit the layout after the switch. This behavior is the same in the FPGA version.

## 11.2 Address Model and Wraparound

The address register for CPU access is 18 bits, shared between read and write. The lower 14 bits (A13–A0) are set by the Port#1 two-byte pair protocol, and the upper bits A17–A14 are set by R#14 (see Chapter 2).

The range over which the address wraps around when incremented (the carry range) changes according to the operating conditions.

| Condition | Wrap range |
|---|---|
| The family in which the screen mode {M5, M4} is 00 (Graphic1 / Text1 / Multicolor / Graphic2), or when VR of R#8 is 0 | 14 bits (only A13–A0 advance) |
| Other than the above, in compatible mode (V58 = 1) | 17 bits |
| Other than the above, in extended mode (V58 = 0) | 18 bits |

The family that wraps at 14 bits is a specification deliberately provided for compatibility dating back to the TMS9918 era. In this family, A17–A14 can be specified with R#14 to access the area from 4000h onward as well, but because the carry from bit13 to bit14 is blocked, in sequential access 3FFFh is followed by 0000h, and 7FFFh is followed by 4000h. For a transfer that crosses a page boundary, set the address again at each boundary.

Also, the A17 set in R#14 is treated as fixed at 0 when the VR bit of R#8 is 0.

## 11.3 Notes on the CPU Access Procedure

The relationship between address setting and the first access follows the same rules as in earlier VDPs (see Chapter 2).

- When an address has been set for writing, the first access immediately after the address setting is not incremented (the data is written to the set address itself).
- When an address has been set for reading, the first read from Port#0 returns the data at the set address. The read address is incremented at the time of the response from VRAM.
- A Port#1 two-byte pair is discarded if, partway through, Port#0 is accessed or any port is read. Take care that sequences of address setting and register writes do not conflict with interrupt handlers and the like (if the VDP is accessed inside a handler, the two-byte pair of the main processing may be interrupted).

## 11.4 Access While a VDP Command Is Executing

While a VDP command is executing, VRAM is also being used by the command engine. The matters that have been established regarding the relationship between commands and CPU access are as follows.

- While a command is executing, another command must not be started (see Chapter 8). Completion is checked by the CE bit of S#2, or by the command completion interrupt (see Chapter 10).
- While the MXC bit of R#45 is 1, VRAM access from the CPU is masked. During this time, writes to VRAM are not executed, and reads return FFh. MXC is a bit by which software explicitly blocks access, so that CPU access does not conflict while a command is executing (see Chapter 8).

[TBC] For the case in which the CPU accesses VRAM while a command is executing with MXC = 0, the arbitration (which side is given priority, and how the access is treated) has not been established in the current materials. It is safest either to avoid CPU access while a command is executing, or to block it with MXC.

## 11.5 Access During the Display Period

In earlier VDPs, CPU access during the display period was subject to restrictions depending on the screen mode, and considerations such as using the blanking interval were required for fast sequential access. In the V9968 as well, the VR bit (during the vertical blanking interval) and the HR bit (during the horizontal blanking interval) of S#2 are provided as clues for knowing the display state, and software can poll these to detect gaps in the display (see Chapter 4).

[TBC] Timing specifications for each screen mode, such as "at what timing CPU access during the display period is guaranteed" and "the minimum interval for sequential access", have not been established in the current materials. Whether software for earlier machines operates with the same timing on the V9968 must await the establishment of these specifications.

## 11.6 About High-Speed Mode

Setting the HS bit (bit0) of R#20 to 1 selects the high-speed mode (see Chapter 3). HS is ORed with an input signal on the board side (force_highspeed), so even if the register value is 0, the high-speed mode may be selected depending on the conditions of the board on which the chip is mounted.

[TBC] The concrete effect of the high-speed mode on command execution times and on the timing of CPU access (which operations become how much faster) has not been established in the current materials.

# Chapter 12 Compatibility and Migration Notes

The V9968 inherits the lineage of the VDPs that developed from the TMS9918 through the V9938 to the V9958, and it is designed on the basic policy that existing software operates as it is. This chapter summarizes the differences in operation from earlier chips, the scope of the "9958 Compatible mode" (V9958 compatible mode), and the points to note when running existing software on the V9968 and when extending software for the V9968.

For the details of each item, refer to the relevant chapter. This chapter is limited to a list of the differences.

## 12.1 Compatible Mode and Extended Mode

The operating mode of the V9968 is switched by the V58 bit (bit0) of R#21 (see Chapter 3).

- **Immediately after a reset, V58 = 1 (compatible mode).** Right after power-on, the V9968 behaves as the equivalent of a V9958, and existing software operates as it is.
- Switching to extended mode (V58 = 0) requires an explicit procedure: release the LockExtReg lock, then write to R#21 (see Chapter 10). Existing software never enters extended mode by mistake.
- The elements that become valid in extended mode are mainly the following.
- The 256KB VRAM address space (Chapter 11)
- The new VDP commands LRMM / LFMC / LFMM (Chapter 8)
- Bits that are stored only in extended mode, such as FG4 of R#45 (Chapter 3)
- Writing of SPS (R#25 bit7) (see Section 12.3)

The ID of S#1 can be used to distinguish the mode. The ID returns `00010` in compatible mode (the same value as the V9958) and `00011` in extended mode (see Chapter 4). Software that distinguishes the generation of the chip by its ID can treat a V9968 in compatible mode as a V9958.

As a side effect of returning to compatible mode, the moment a 1 is written to R#21, bit17 of all five table base addresses (pattern name / pattern generator / sprite pattern generator / color / sprite attribute) is cleared to 0 (see Chapter 3).

## 12.2 Behavior Formalized as "9958 Compatible"

Among the behaviors that were undefined in earlier materials, or for which only representative values were shown, those that have been found to be actually usable on the V9968 are explained in this manual as formal features, annotated "9958 Compatible". Representative examples are as follows.

- **Don't-cares in screen mode selection**: In Text1 and Multicolor, M3 is a don't-care, and the combinations with M3 = 1 (Text1 = 00101, Multicolor = 00110) also operate as the same modes (see Chapter 5).
- **Treatment of the reserved logical operations**: In the logical operations of the VDP commands, the codes 0101–0111, which were formerly reserved, operate as the equivalent of IMP, and 1101–1111 operate as the equivalent of TIMP (see Chapter 8).

## 12.3 Known Differences in Operation from Earlier Chips

The items listed next are points where, even in compatible mode, the operation differs from earlier chips (or differs from the descriptions in earlier materials). Take care when porting existing software and when verifying operation.

### Differences Relating to Display and Color

- **Sprite colors in Graphic7 (difference from the V9938)**: On the V9938, sprites in Graphic7 used a fixed 16 colors that did not use the palette, but on the V9968 the sprite colors are also determined from the same 256-color palette as the background. The 256 colors are divided into 16 groups (palette sets) of 16 colors each, and sprites are displayed with the 16 colors within a group. However, the first color of each group is treated as transparent, so the number of colors that can actually be used is 15 (see Chapter 7).
- **Reset initial values of the palette**: Only entries 0–15 are initialized at a reset, and they receive initial colors compatible with the V9958. **Entries 16–255 are undefined.** Existing programs use only 16 colors and are therefore not affected, but when using the palette on the extended side, such as EPAL or Sprite mode3, be sure to initialize it from the program before use (see Chapter 7).

### Differences Relating to Register Read Values

- **Bit3 / bit2 of S#2 are fixed at 1.** These bits are not described in old materials; do not make a

judgment on the premise that unused bits are returned as 0 (see Chapter 4).

- **Bit7 / bit6 of S#1 are fixed at 0**, and the unused bits of S#4 / S#6 / S#9 are **fixed at 1**. Judge the values of the status registers by masking only the bits required (see Chapter 4).
- **S#10–S#15 return FFh.** The read value when a number that is not implemented is specified is established as FFh (see Chapter 4).
- **Reads from Port#2 / Port#3 return FFh.** These ports are write-only (see Chapter 2).

## Treatment of Not-Implemented Registers

- R#22, R#24, R#28–R#31, and the command registers R#59–R#63 are not implemented, and writes to them are ignored (see Chapter 3). Initialization code written on the premise of writing to undefined registers has no effect on the V9968.

## Protection of the Extended Bits

- **SPS (R#25 bit7) is a bit newly provided in the V9968, and can be written only in extended mode (V58 = 0).** In compatible mode, writes are ignored, so existing software that writes a value including bit7 of R#25 cannot unintentionally enable the sprite priority circulation (see Chapter 6).
- Extended settings of R#20 / R#21, such as S16 (R#20 bit7) and CEIE (R#20 bit6), are protected by the LockExtReg lock (see Chapter 10).

## Compatibility Specifications Relating to the Address Space

- In the family in which the screen mode {M5, M4} is 00 (Graphic1 / Text1 / Multicolor / Graphic2), the address of CPU access wraps at 14 bits. This is a specification intentionally retained for compatibility with machines equipped with the TMS9918, and is not a defect (see Chapter 11).
- In compatible mode, A17 is masked and VRAM is treated as the equivalent of 128KB. The earlier address layout can be used as it is (see Chapter 11).
- In SCREEN 7 and above, the physical layout of VRAM becomes the interleave compatible with earlier chips (even addresses in DRAM0, odd addresses in DRAM1). However, enabling sprite mode 3 returns the layout to flat, and the physical arrangement of the data changes with the switching (see Section 11.1).

### Differences Relating to VDP Commands

- The operation of the 13 earlier commands has been checked against the earlier specification (the "MSX2 Technical Handbook") and confirmed to be identical (see Chapter 8).
- The codes of the new commands LRMM / LFMC / LFMM (0001–0011) are assigned to command codes that were unused on the V9958. If they are started in compatible mode, they are treated as STOP. Existing software normally never issues an unused code, but be aware that the code space is shared (see Chapter 8).

## 12.4 Summary of Notes for Migration and Extension

When extending or porting existing software for the V9968, check the following points.

1. **Follow the procedure for migrating to extended mode**: perform it in the order release of the LockExtReg lock → V58 = 0 by R#21 → the extended settings required, and lock it again when you have finished using it (see Chapter 10).
2. **Initialize palette entries 16–255**: they are undefined immediately after a reset (Section 12.3).
3. **Judge the status registers after masking**: on the premise that fixed bits (fixed at 1 and fixed at 0) exist, do not compare by a match of all bits (see Chapter 4).
4. **Set the table bases again when returning to compatible mode**: there is the side effect that bit17 is cleared (Section 12.1).
5. **Use the ID of S#1 to identify the chip**: in compatible mode, the same value as the V9958 (00010) is returned, so if you want to detect the very existence of the V9968, a procedure such as switching to extended mode and then checking the ID is necessary.
6. **If it is used as a second VDP (2nd VDP), note the differences in the base address and the interrupt hook**: the base address is 88h, and the vertical synchronization interrupt must also be handled in the H.KEYI hook. It is not necessarily coincident with the vertical synchronization of the built-in VDP (see Section 10.8).

The [TBC] items that remain in each chapter of this manual (FIL / ILNS / SVNS of R#20, FG4 / XHR of R#45, the scope of application of the window registers, the CPU access timing during the display period, and so on) will be reflected in a future edition as soon as the specifications are established. Do not create software that depends on items that have not yet been established.

# Appendix

This appendix collects quick references for the material described in Chapters 3 to 8. For details of the bit layouts and functional descriptions, see the respective chapters. If the contents of this appendix differ from those of the chapters, the descriptions in the chapters take precedence.

## Appendix A Control Register Summary

### A.1 Display and Access Registers (R#0–R#27)

All are write-only. Writes to R#20 / R#21 are ignored while LockExtReg = 1 (see Chapter 10). "V9968 extended" indicates registers (bits) added or extended in the V9968.

| R# | Name | Reset Value | Description |
|---|---|---|---|
| R#0 | Mode0 | IE1=0, M5M4M3=000 | bit4: IE1 (line interrupt enable) / bit3–1: M5, M4, M3 |
| R#1 | Mode1 | All 0 | bit6: BL (display enable) / bit5: IE0 (frame interrupt enable) / bit4: M1 / bit3: M2 / bit1: SI / bit0: MAG |
| R#2 | Pattern Name Table Address | 0 | A17–A10 (base address [17:10]) |
| R#3 | Color Table Address (L) | 0 | A13–A6 (color table base [13:6]) |
| R#4 | Pattern Generator Table Address | 0 | A17–A11 (base [17:11]) |
| R#5 | Sprite Attribute Table Address (L) | 0 | A14–A7 (base [14:7]). bit1 / bit0 also function as an AND mask with the plane number |

| R# | Name | Reset Value | Description |
|---|---|---|---|
| R#6 | Sprite Pattern Generator Table Address | 0 | A17–A11 (base [17:11]) |
| R#7 | Background Color | 0 | Border color (BD7–BD0, all bits stored) |
| R#8 | Mode2 | VR=1, others 0 | bit5: TP / bit3: VR / bit1: SPD |
| R#9 | Mode3 | All 0 | bit7: LN / bit3: IL / bit2: EO / bit1: NT |
| R#10 | Color Table Address (H) | 0 | A17–A14 (color table base [17:14]) |
| R#11 | Sprite Attribute Table Address (H) | 0 | A17–A15 (base [17:15]) |
| R#12 | Text Color / Back Color | 0 | Upper nibble: text color / lower nibble: back color (used in place of R#7 when blinking) |
| R#13 | Blinking Period | 0 | bit7–4: CN3–CN0 / bit3–0: CF3–CF0 |
| R#14 | VRAM Access Base Address | Address 0 | A17–A14 (upper bits of the CPU access address) |
| R#15 | Status Register Pointer | 0 | Number of the status register to read (S3–S0) |
| R#16 | Color Palette Pointer | Pointer 0 | Number of the palette write destination. 0–15 when EPAL=0, 0–255 when EPAL=1 |
| R#17 | Control Register Pointer | Pointer 0, AII=0 | bit7: AII (auto-increment) / bit5–0: register number |

| R# | Name | Reset Value | Description |
|---|---|---|---|
| R#18 | Display Adjust | 0 | bit7–4: V3–V0 (vertical position) / bit3–0: H3–H0 (horizontal position) |
| R#19 | Interrupt Line | 0 | Line number at which to generate a line interrupt (IL7–IL0) |
| R#20 | Mode5 | All 0 | bit7: S16 / bit6: CEIE / bit5: FIL / bit4: EPAL / bit3: SP3 / bit2: ILNS / bit1: SVNS (Note 1) / bit0: HS |
| R#21 | Mode6 | V58=1 | bit7: 0 (fixed) / bit6: 0 (fixed) / bit5: 1 (fixed) / bit4: 1 (fixed) / bit3: 1 (fixed) / bit2: 0 (fixed) / bit1: 1 (fixed) / bit0: V58 (1: compatible mode / 0: extended mode) |
| R#22 | — | — | Not implemented (writes are ignored) |
| R#23 | Display Offset | 0 | Vertical scroll offset (DO7–DO0) |
| R#24 | — | — | Not implemented (writes are ignored) |
| R#25 | Mode4 | All 0 | bit7: SPS (Note 2) / bit6: CMD / bit5: VDS (Note 3) / bit4: YAE / bit3: YJK / bit2: WTE (Note 3) / bit1: MSK / bit0: SP2 |
| R#26 | Horizontal Offset (By Character) | 0 | HO8–HO3 (horizontal scroll, in character units) |
| R#27 | Horizontal Offset (By Dot) | 0 | HO2–HO0 (horizontal scroll, in dot units) |
| R#28–R#3 | — | — | Not implemented (writes are ignored) |

| R# | Name | Reset Value | Description |
|---|---|---|---|
| 1 | | | |

- Note 1: SVNS is a bit that makes sprites unaffected by vertical scrolling (R#23); the sprites are displayed at fixed, screen-anchored positions, with Y=0 at the very top of the screen (see Chapters 3 and 6). ILNS is a bit that makes the line number for the line interrupt (R#19) unaffected by vertical scrolling (R#23), using an absolute position referenced to the top of the screen (see Chapters 3 and 10). FIL is flat interlace, a mode in which, during interlace, page 0 and page 1 are joined vertically in VRAM for display (see Chapter 3).
- Note 2: SPS is a bit newly added in the V9968. It can be written only in extended mode (V58 = 0).
- Note 3: VDS / WTE are the names in the V9958 layout; in the V9968 they are not stored.

## A.2 Command Registers (R#32–R#58)

All are write-only. Writing to R#46 starts a command.

| R# | Name | Reset Value | Description |
|---|---|---|---|
| R#32 | Source X (L) | 0 | SX[7:0] |
| R#33 | Source X (H) | 0 | SX[11:8] |
| R#34 | Source Y (L) | 0 | SY[7:0] |
| R#35 | Source Y (H) | 0 | SY[12:8] |
| R#36 | Destination X (L) | 0 | DX[7:0] |
| R#37 | Destination X (H) | 0 | DX[8] |
| R#38 | Destination Y (L) | 0 | DY[7:0] |

| R# | Name | Reset Value | Description |
|---|---|---|---|
| R#39 | Destination Y (H) | 0 | DY[10:8] (in 256K mode) / DY[9:8] (in 128K mode, DY[10] is fixed at 0) |
| R#40 | Number Of Dots X (L) | 0 | NX[7:0] |
| R#41 | Number Of Dots X (H) | 0 | NX[10:8] |
| R#42 | Number Of Dots Y (L) | 0 | NY[7:0] |
| R#43 | Number Of Dots Y (H) | 0 | NY[10:8] (same mode dependence as R#39) |
| R#44 | Color | 0 | Color code. In HMMC / LMMC / LFMC, also serves as the receiving port for CPU transfer data. In LFMC / LFMM, the foreground color |
| R#45 | Argument | All 0 | bit7: FG4 (Note 4) / bit6: MXC (compatible mode) / XHR (extended mode) / bit5: MXD / bit4: MXS (Note 4) / bit3: DIY / bit2: DIX / bit1: EQ / bit0: MAJ |
| R#46 | Command | STOP / logical operation 0 | bit7–4: command code (Appendix D) / bit3–0: logical operation |
| R#47 | Rotation Vector X (L) | 0 | VX[7:0] (for LRMM, signed 8.8 fixed-point) |
| R#48 | Rotation Vector X (H) | 0 | VX[15:8] |
| R#49 | Rotation Vector Y (L) | 0 | VY[7:0] |
| R#50 | Rotation | 0 | VY[15:8] |

| R# | Name | Reset Value | Description |
|---|---|---|---|
| | Vector Y (H) | | |
| R#51 | Window Start X (L) | 0 | WSX[7:0] (used only by LRMM) |
| R#52 | Window Start X (H) | 0 | WSX[8] |
| R#53 | Window Start Y (L) | 0 | WSY[7:0] |
| R#54 | Window Start Y (H) | 0 | WSY[10:8] |
| R#55 | Window End X (L) | 1FFh for WEX as a whole | WEX[7:0] |
| R#56 | Window End X (H) | (Same as above) | WEX[8] |
| R#57 | Window End Y (L) | 7FFh for WEY as a whole | WEY[7:0] |
| R#58 | Window End Y (H) | (Same as above) | WEY[10:8] |
| R#59 – R#63 | — | — | Not implemented (writes are ignored) |

- Note 4: FG4 is a bit which, when 1, makes VDP commands operate as Graphic4 (SCREEN 5) regardless of the specified screen mode (used, for example, for drawing into the pattern data of Sprite Mode 3. If the display itself is in Graphic4, the operation does not change whether FG4 is set or not. See Chapters 3 and 8). FG4 is masked to 0 when stored unless in extended mode. XHR is a bit used in extended mode for the pixel aspect correction of LRMM (treating pixels as

vertically elongated) (see Chapter 8). Also, because extended RAM has been abolished in the V9968, the only selection target of MXS / MXD is VRAM.

Note that writing to R#32 / R#33 / R#34 also simultaneously affects bits [7:0] / [15:8] / [17:16] of the font address (18 bits) for LFMC / LFMM (see Chapter 8).

## A.3 Bit Layout Diagrams by Register (R#0–R#27)

The bit layout of each register is shown in numerical order. The diagrams are read in the same way as in Chapter 3: the left side is the MSB and the right side is the LSB. The bracket below each field explains that field, and for flags it shows the meanings of 0 and 1. "•" denotes an unused bit, and a gray cell denotes a fixed value, or a bit that is not stored in the V9968. For the unimplemented R#22 and R#24, the diagrams show all bits as unused. R#28–R#31 have no individual diagrams, because they are summarized as a range in the table in A.1.

### R#0 Mode0

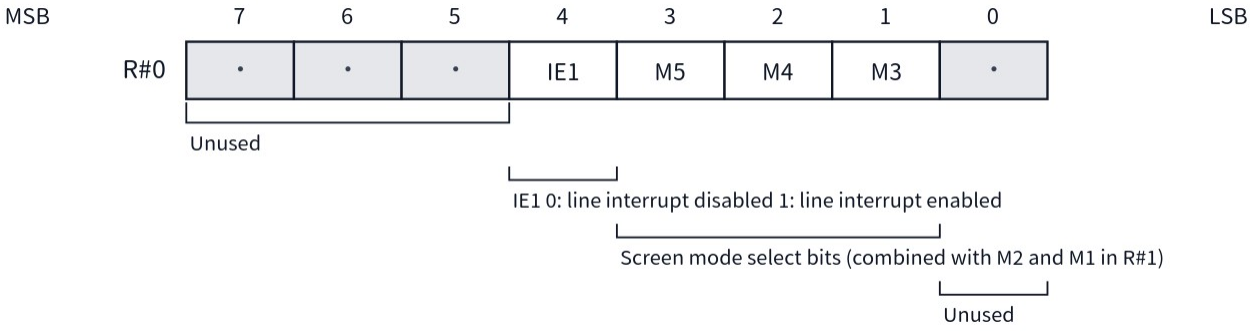



Figure A.1 Bit layout of R#0 Mode0

### R#1 Mode1

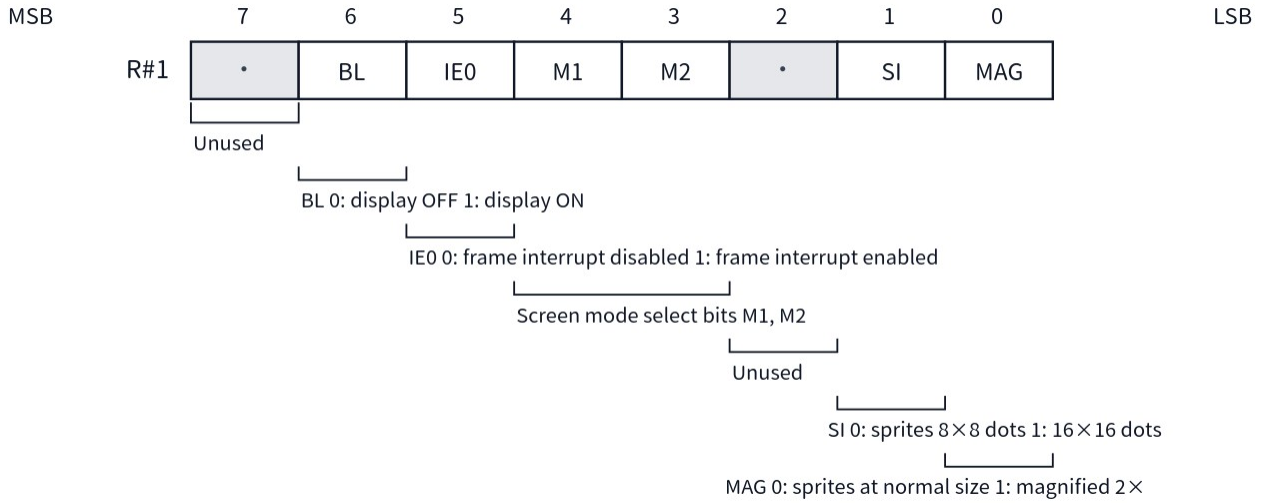



Figure A.2 Bit layout of R#1 Mode1

### R#2 Pattern Name Table Address

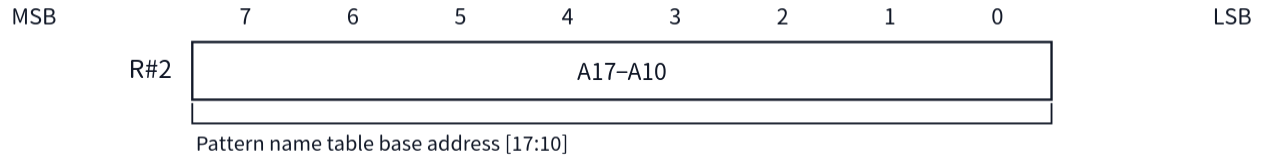



Figure A.3 Bit layout of R#2 Pattern Name Table Address

### R#3 Color Table Address (L)

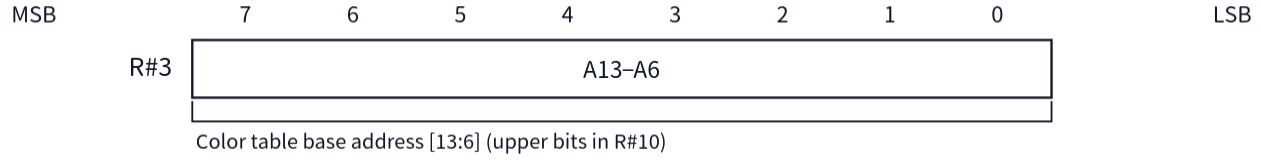



Figure A.4 Bit layout of R#3 Color Table Address (L)

### R#4 Pattern Generator Table Address

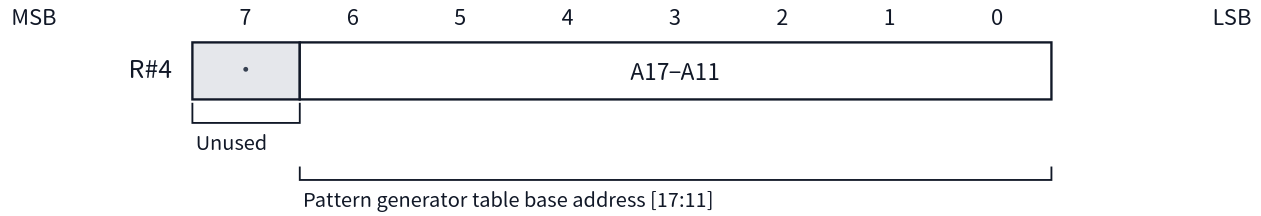


Figure A.5 Bit layout of R#4 Pattern Generator Table Address

### R#5 Sprite Attribute Table Address (L)

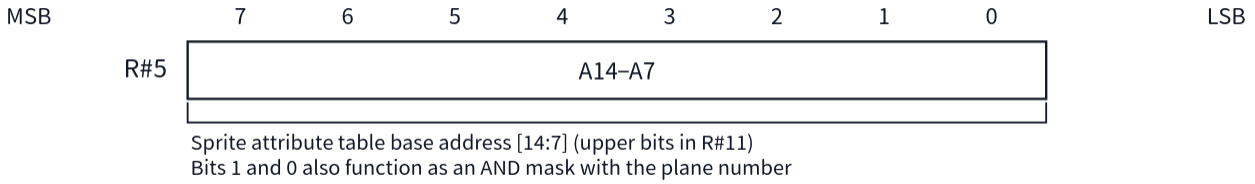



Figure A.6 Bit layout of R#5 Sprite Attribute Table Address (L)

### R#6 Sprite Pattern Generator Table Address

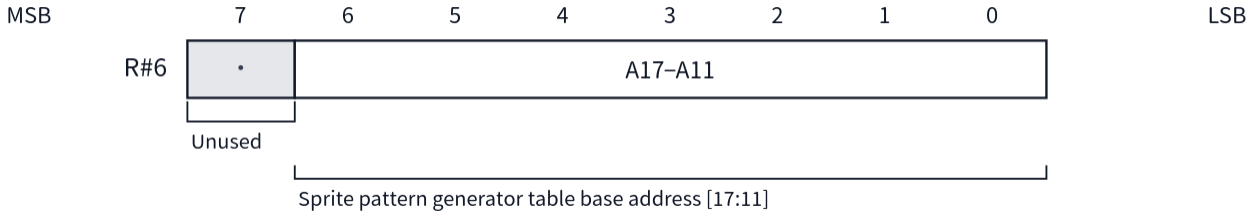



Figure A.7 Bit layout of R#6 Sprite Pattern Generator Table Address

### R#7 Background Color




Figure A.8 Bit layout of R#7 Background Color

### R#8 Mode2

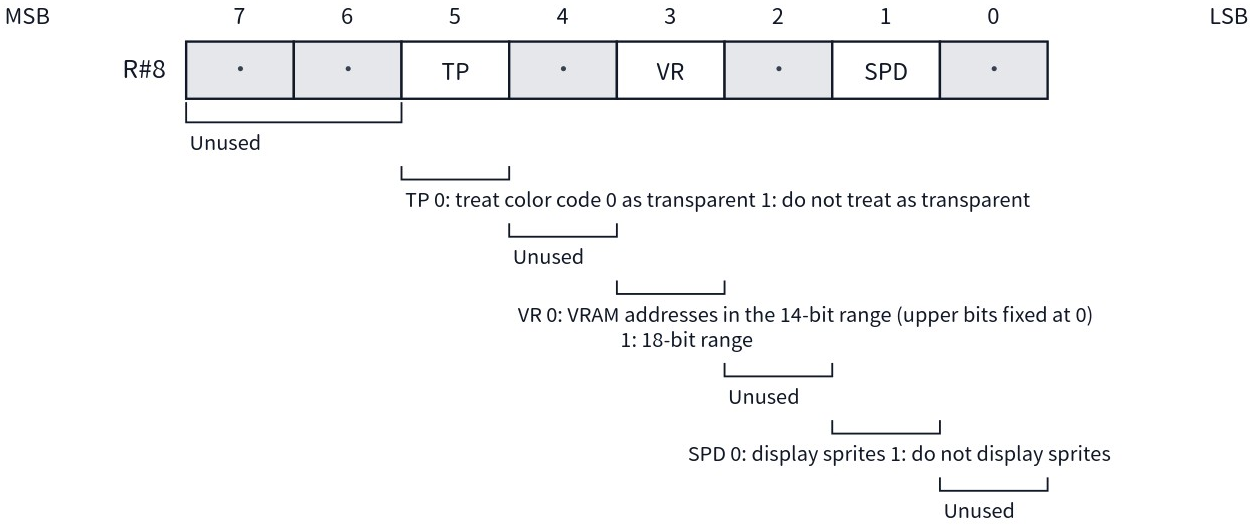



Figure A.9 Bit layout of R#8 Mode2

### R#9 Mode3

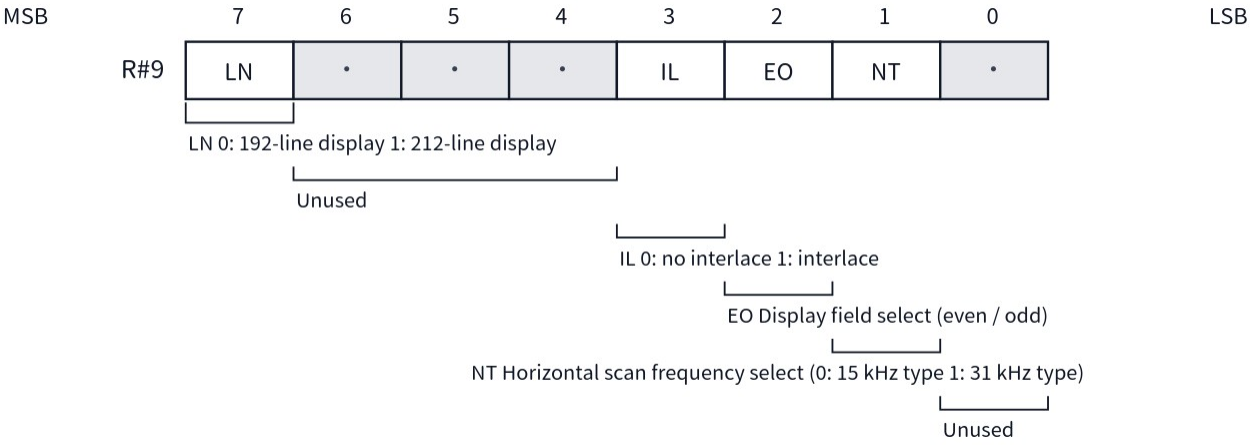



Figure A.10 Bit layout of R#9 Mode3

### R#10 Color Table Address (H)

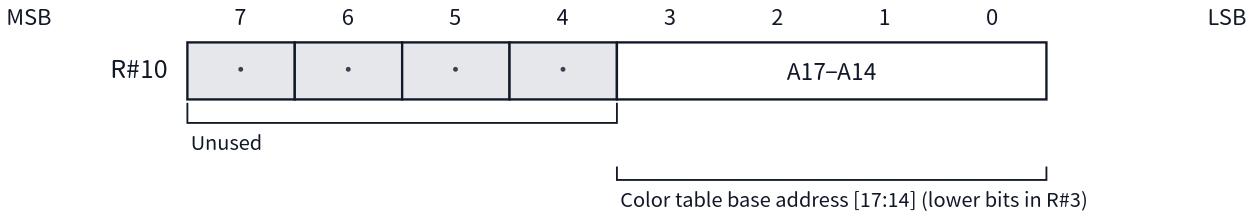



Figure A.11 Bit layout of R#10 Color Table Address (H)

### R#11 Sprite Attribute Table Address (H)

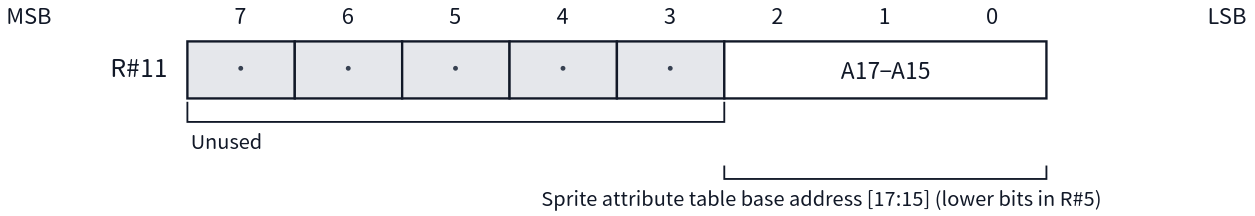



Figure A.12 Bit layout of R#11 Sprite Attribute Table Address (H)

### R#12 Text Color / Back Color

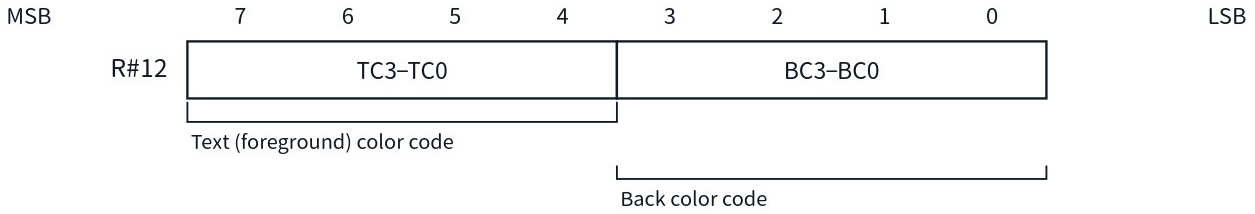



Figure A.13 Bit layout of R#12 Text Color / Back Color

### R#13 Blinking Period

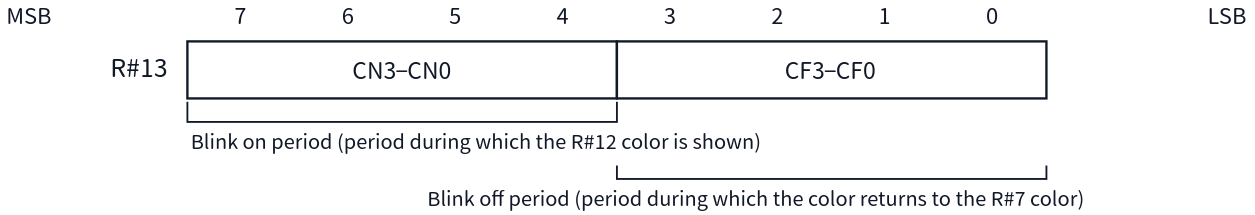



Figure A.14 Bit layout of R#13 Blinking Period

### R#14 VRAM Access Base Address

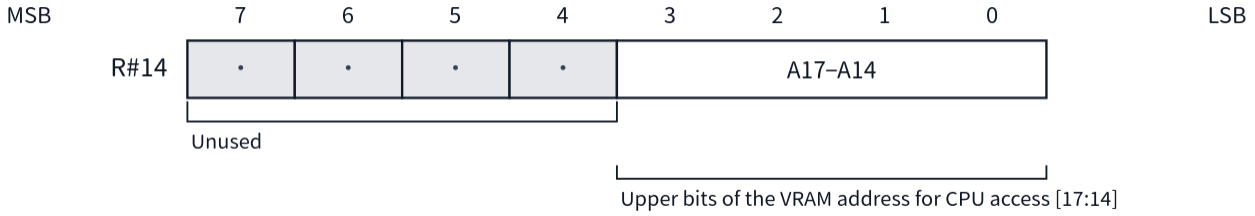



Figure A.15 Bit layout of R#14 VRAM Access Base Address

### R#15 Status Register Pointer

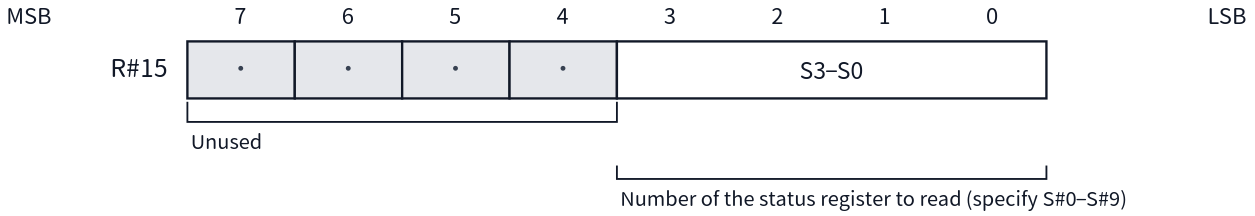



Figure A.16 Bit layout of R#15 Status Register Pointer

### R#16 Color Palette Pointer

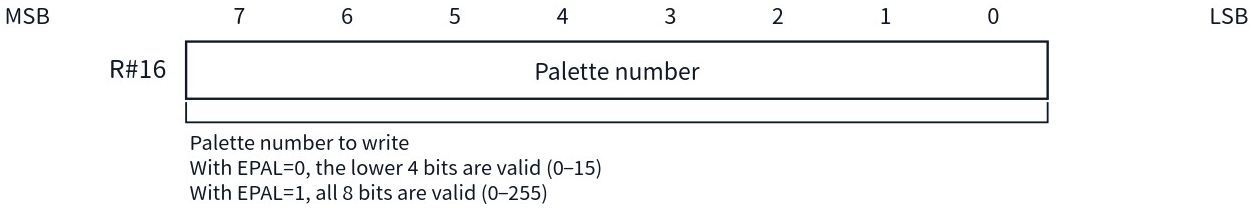



Figure A.17 Bit layout of R#16 Color Palette Pointer

### R#17 Control Register Pointer

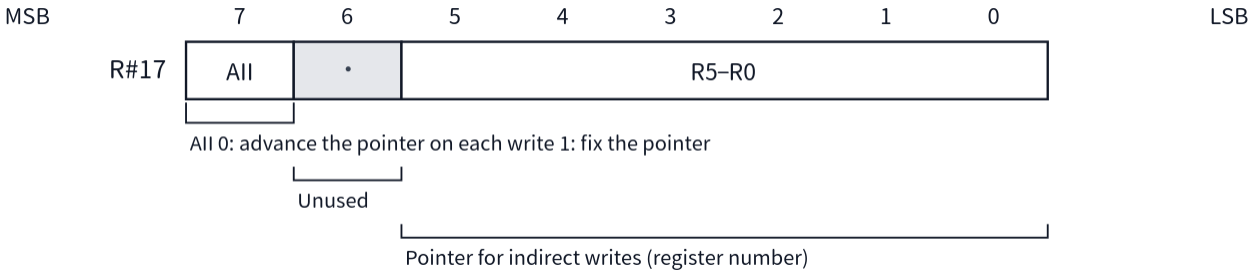



Figure A.18 Bit layout of R#17 Control Register Pointer

### R#18 Display Adjust

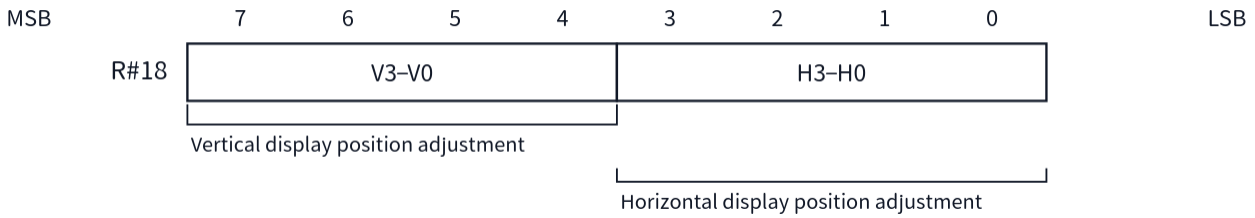



Figure A.19 Bit layout of R#18 Display Adjust

### R#19 Interrupt Line

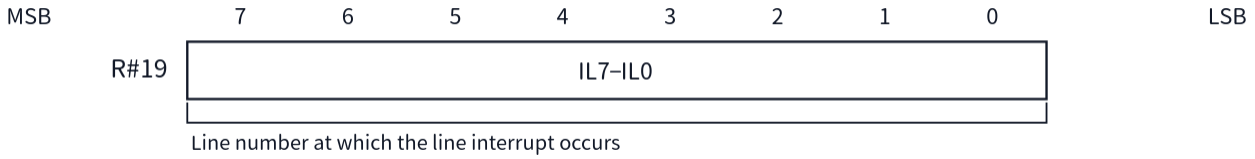



Figure A.20 Bit layout of R#19 Interrupt Line

### R#20 Mode5

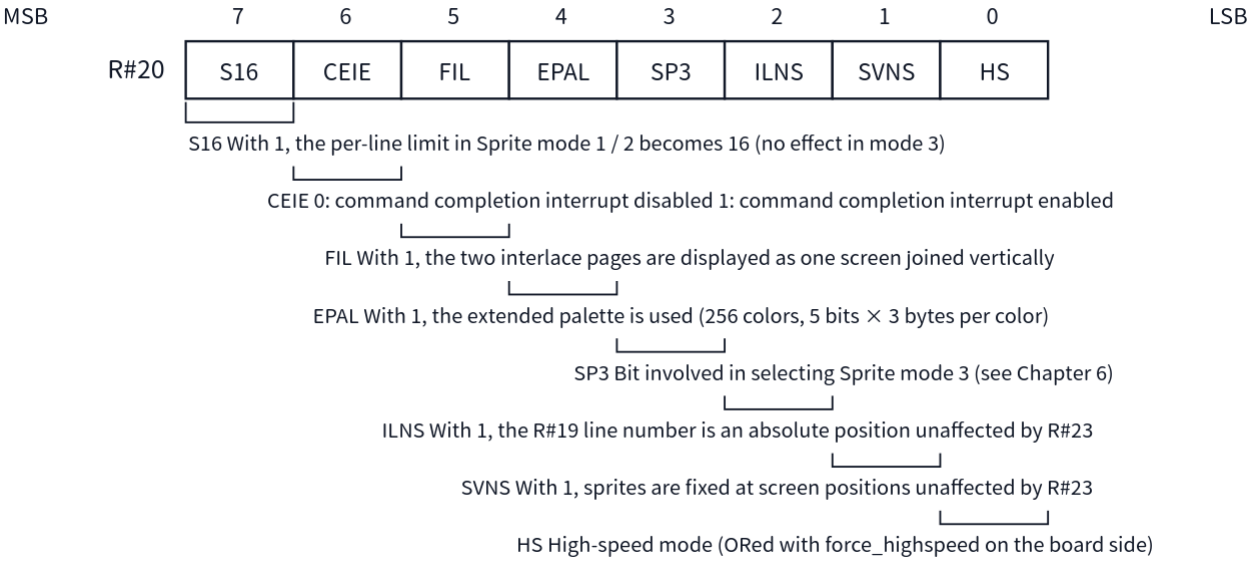



Figure A.21 Bit layout of R#20 Mode5

### R#21 Mode6

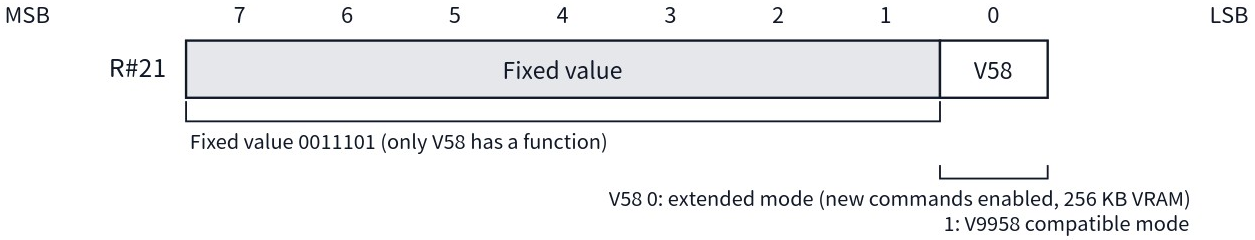



Figure A.22 Bit layout of R#21 Mode6

### R#22 (Not Implemented)

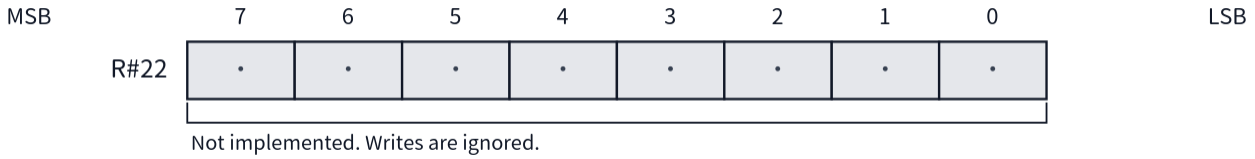



Figure A.23 Bit layout of R#22 (Not Implemented)

### R#23 Display Offset

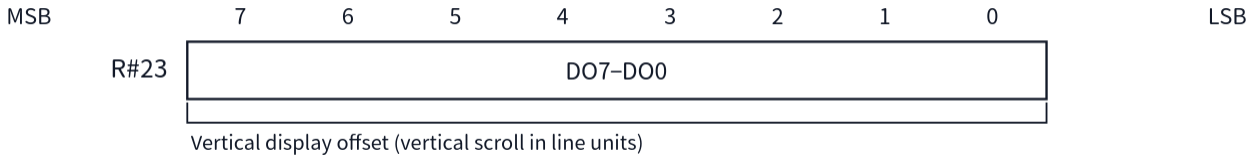



Figure A.24 Bit layout of R#23 Display Offset

### R#24 (Not Implemented)

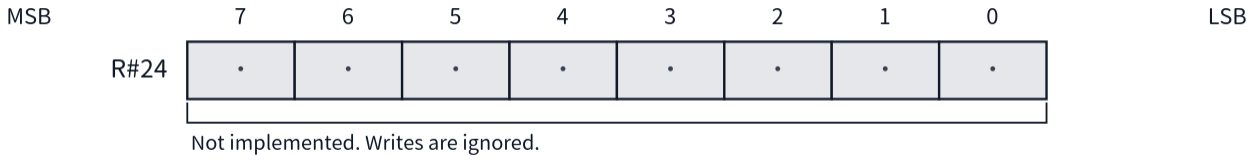



Figure A.25 Bit layout of R#24 (Not Implemented)

### R#25 Mode4

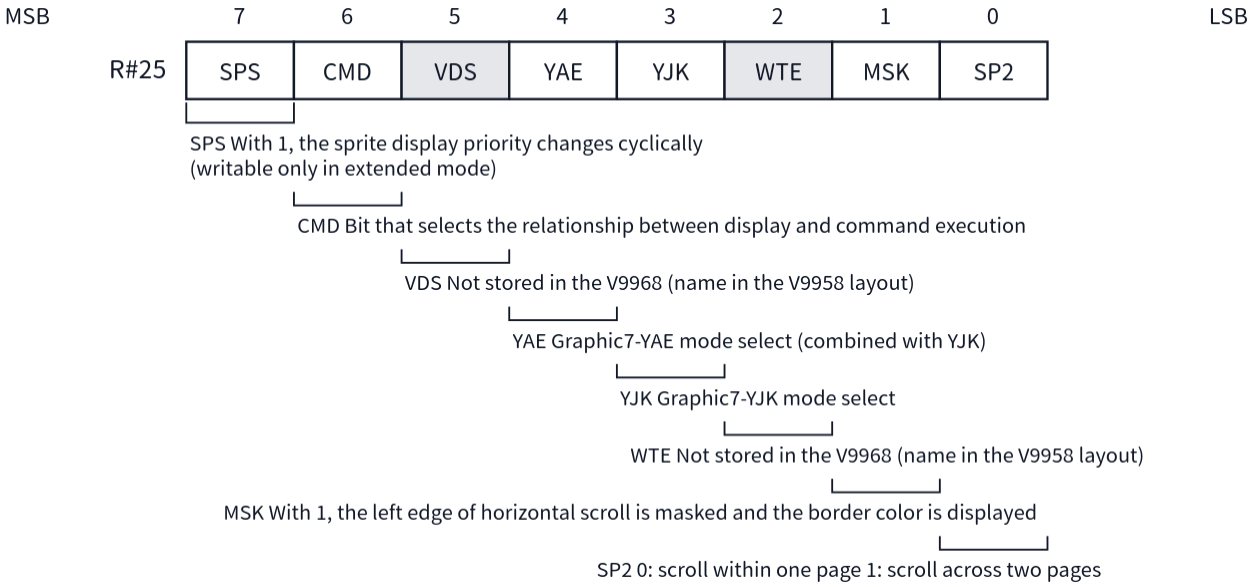


Figure A.26 Bit layout of R#25 Mode4

### R#26 Horizontal Offset (By Character)

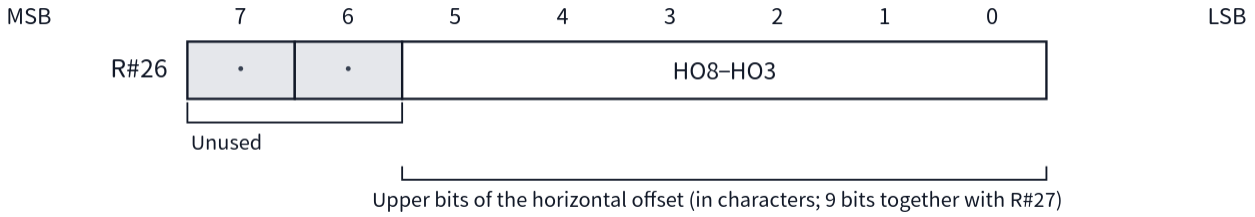



Figure A.27 Bit layout of R#26 Horizontal Offset (By Character)

### R#27 Horizontal Offset (By Dot)

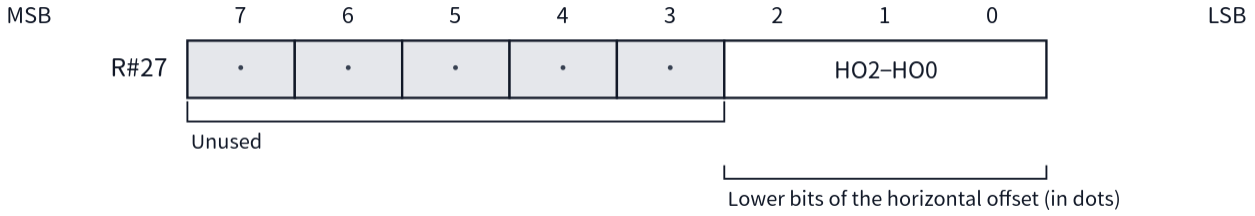



Figure A.28 Bit layout of R#27 Horizontal Offset (By Dot)

## A.4 Bit Layout Diagrams by Register (R#32–R#58)

The bit layouts of the command-related registers are shown in numerical order. The diagrams are read in the same way as in A.3. For details of their use in each command, see Chapter 8. R#59–R#63 are not implemented, so they have no diagrams.

### R#32 Source X (L)

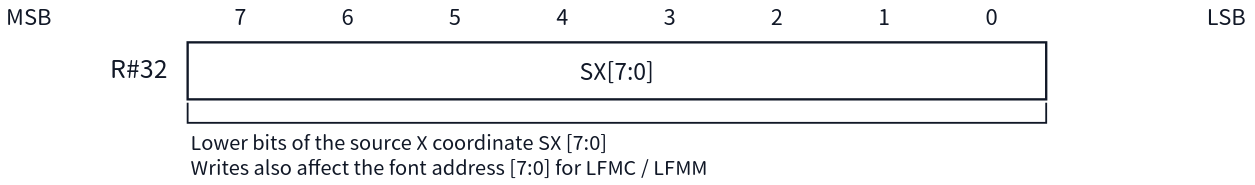



Figure A.29 Bit layout of R#32 Source X (L)

### R#33 Source X (H)

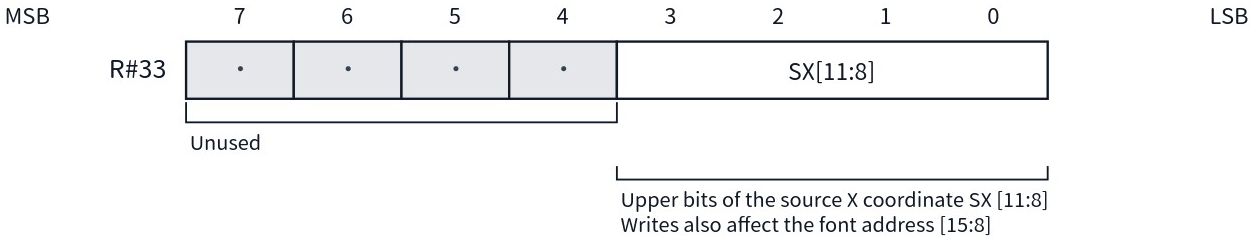



Figure A.30 Bit layout of R#33 Source X (H)

### R#34 Source Y (L)

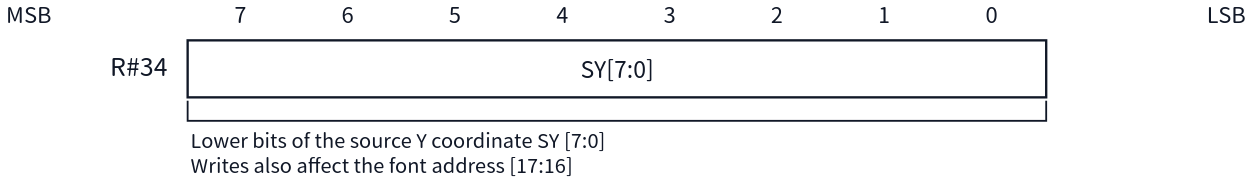



Figure A.31 Bit layout of R#34 Source Y (L)

### R#35 Source Y (H)

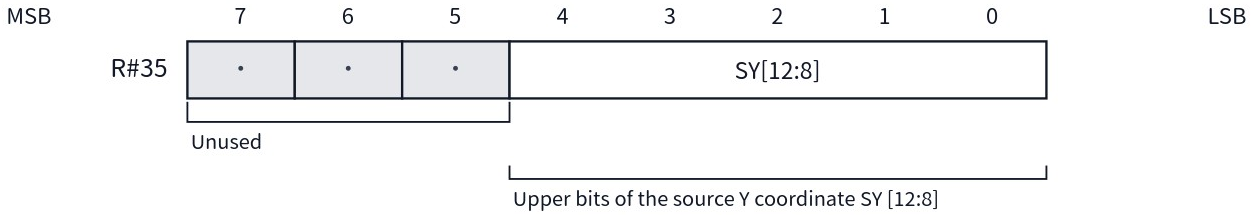



Figure A.32 Bit layout of R#35 Source Y (H)

### R#36 Destination X (L)

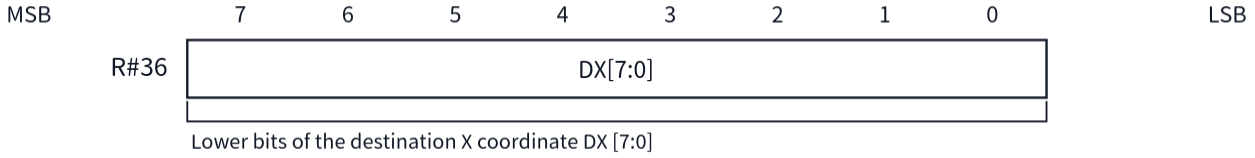



Figure A.33 Bit layout of R#36 Destination X (L)

### R#37 Destination X (H)

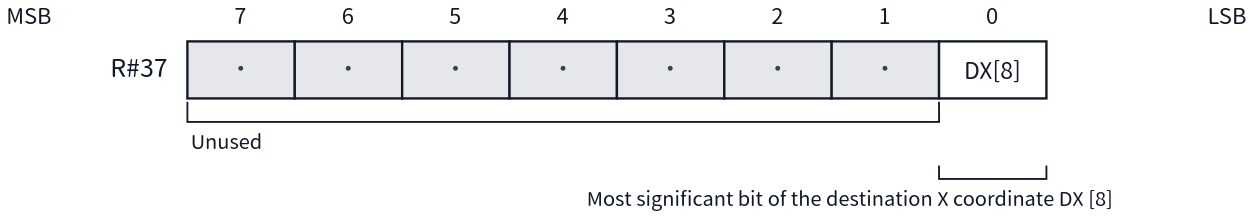



Figure A.34 Bit layout of R#37 Destination X (H)

### R#38 Destination Y (L)

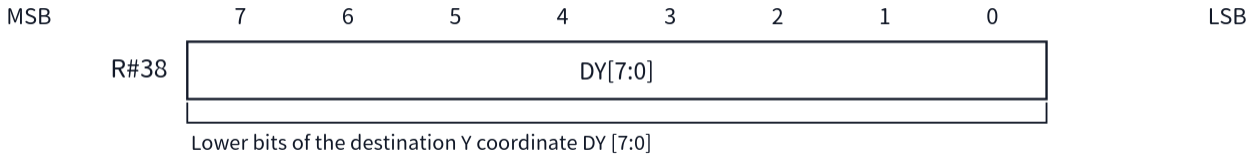



Figure A.35 Bit layout of R#38 Destination Y (L)

### R#39 Destination Y (H)

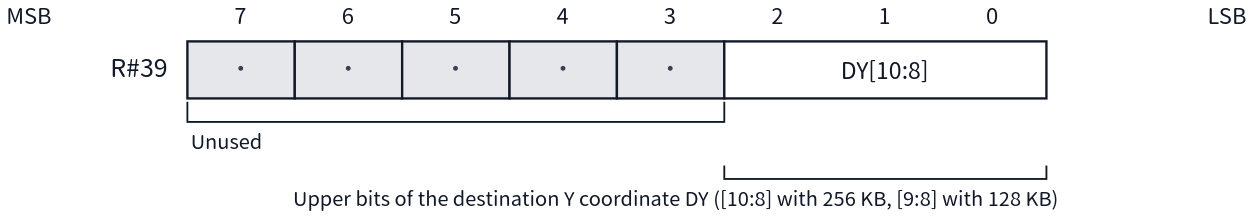



Figure A.36 Bit layout of R#39 Destination Y (H)

### R#40 Number Of Dots X (L)

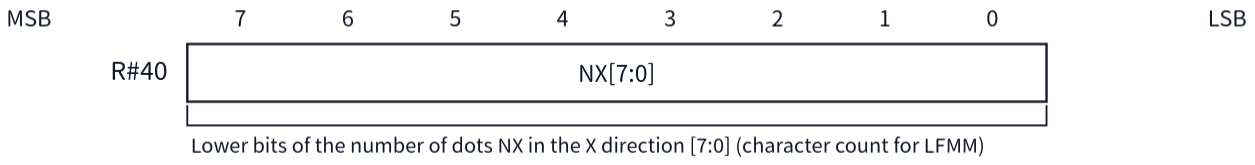



Figure A.37 Bit layout of R#40 Number Of Dots X (L)

### R#41 Number Of Dots X (H)

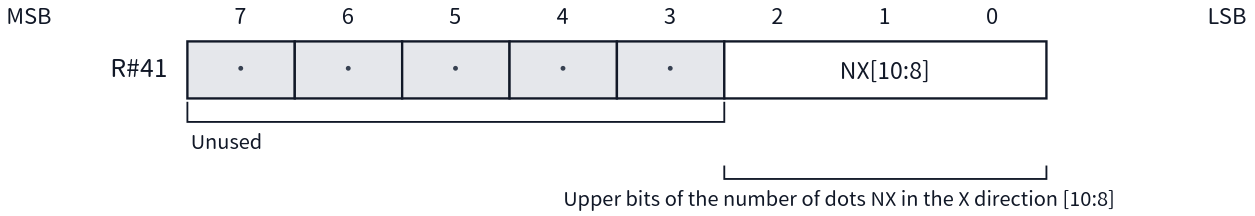



Figure A.38 Bit layout of R#41 Number Of Dots X (H)

### R#42 Number Of Dots Y (L)

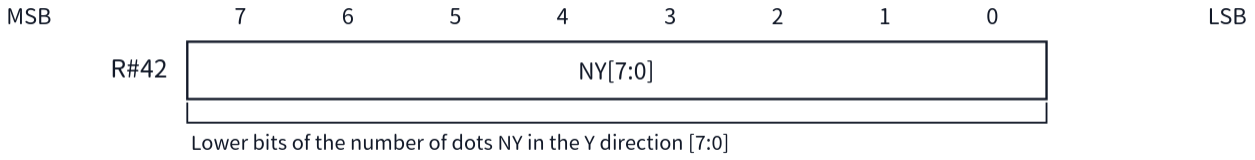



Figure A.39 Bit layout of R#42 Number Of Dots Y (L)

### R#43 Number Of Dots Y (H)

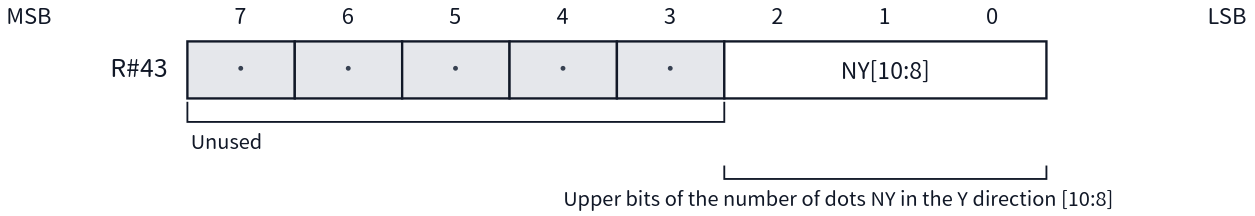



Figure A.40 Bit layout of R#43 Number Of Dots Y (H)

### R#44 Color

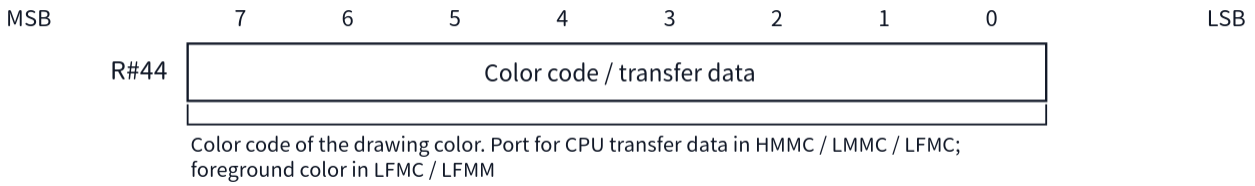



Figure A.41 Bit layout of R#44 Color

## R#45 Argument

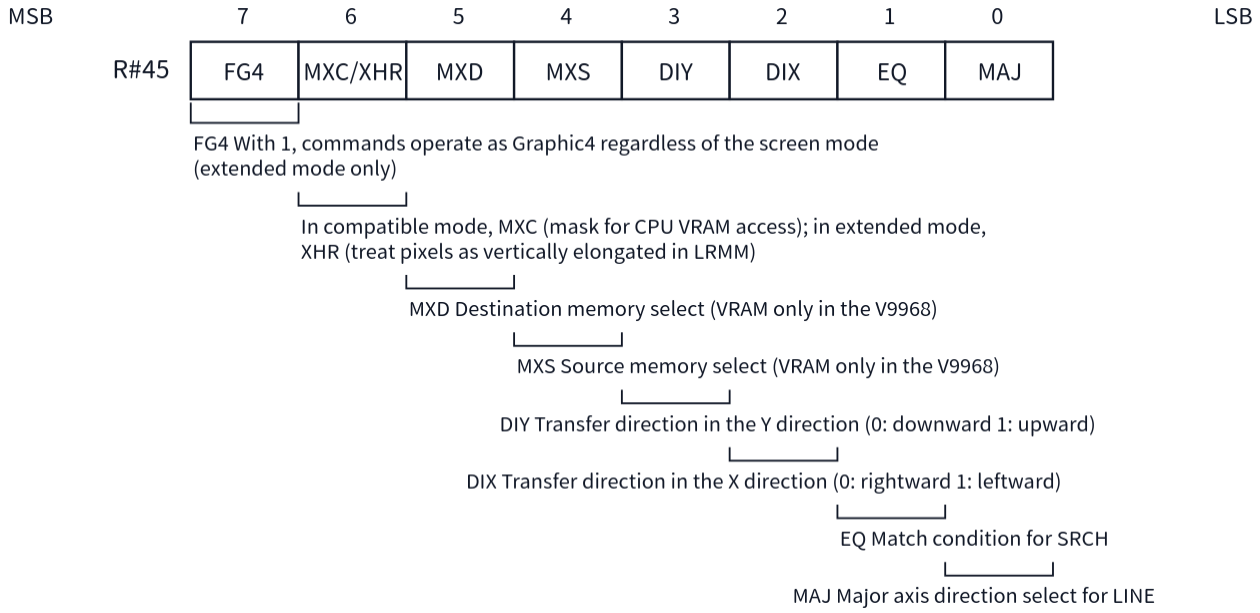



Figure A.42 Bit layout of R#45 Argument

## R#46 Command

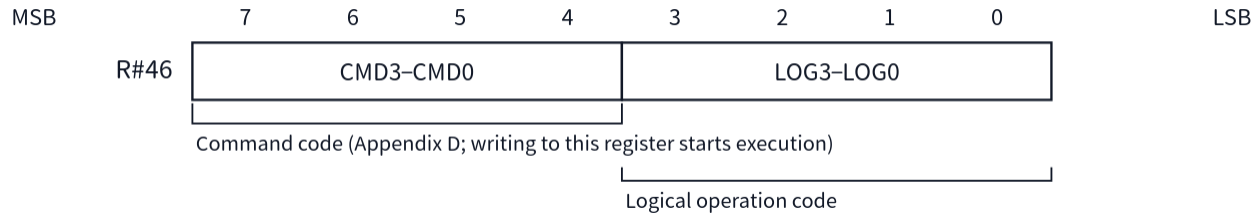



Figure A.43 Bit layout of R#46 Command

## R#47 Rotation Vector X (L)

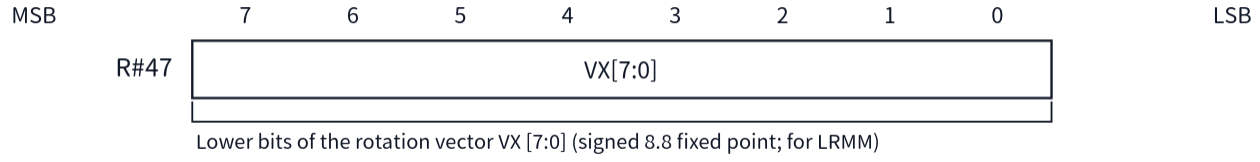



Figure A.44 Bit layout of R#47 Rotation Vector X (L)

### R#48 Rotation Vector X (H)

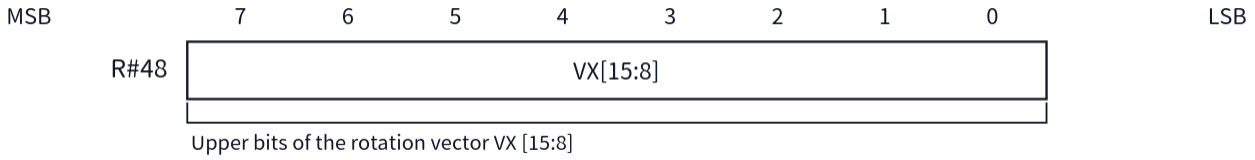



Figure A.45 Bit layout of R#48 Rotation Vector X (H)

### R#49 Rotation Vector Y (L)

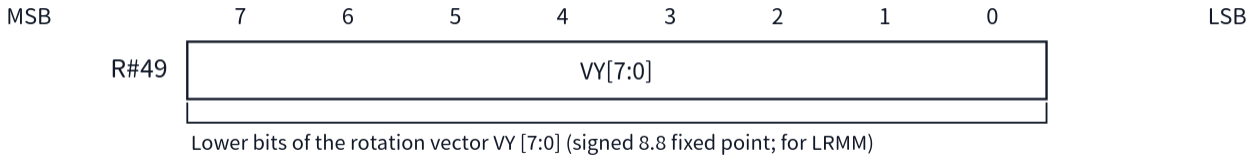



Figure A.46 Bit layout of R#49 Rotation Vector Y (L)

### R#50 Rotation Vector Y (H)

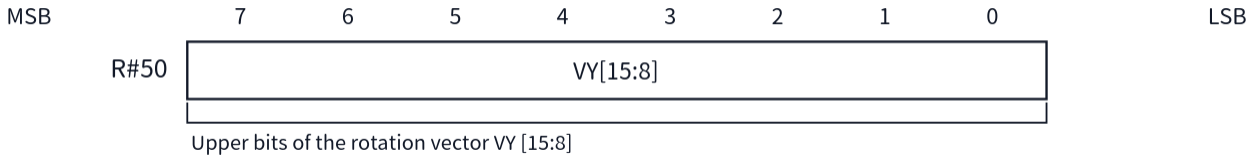



Figure A.47 Bit layout of R#50 Rotation Vector Y (H)

### R#51 Window Start X (L)

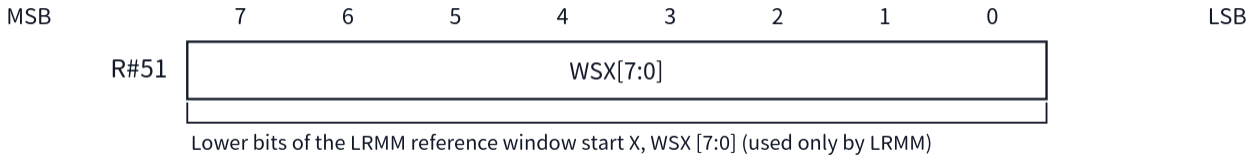



Figure A.48 Bit layout of R#51 Window Start X (L)

### R#52 Window Start X (H)

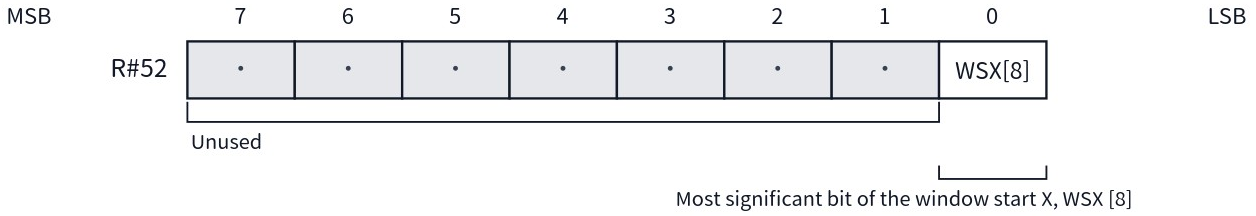



Figure A.49 Bit layout of R#52 Window Start X (H)

### R#53 Window Start Y (L)

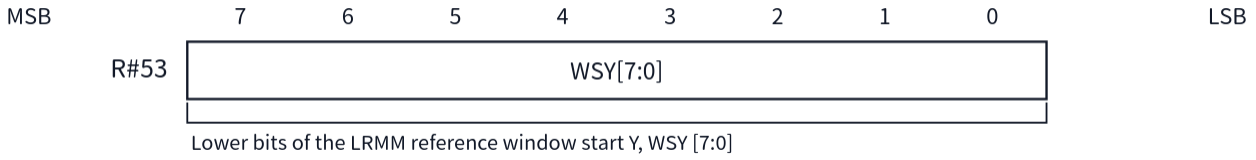



Figure A.50 Bit layout of R#53 Window Start Y (L)

### R#54 Window Start Y (H)

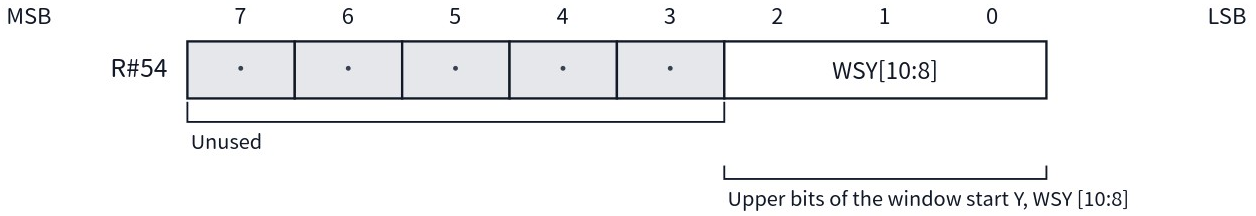



Figure A.51 Bit layout of R#54 Window Start Y (H)

### R#55 Window End X (L)

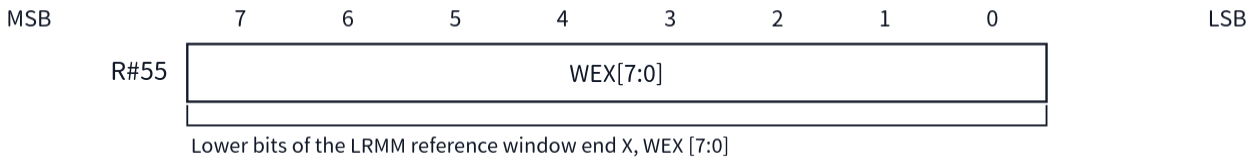



Figure A.52 Bit layout of R#55 Window End X (L)

### R#56 Window End X (H)

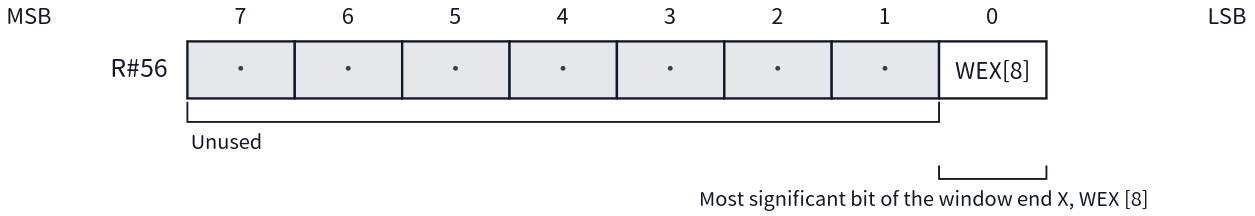



Figure A.53 Bit layout of R#56 Window End X (H)

### R#57 Window End Y (L)

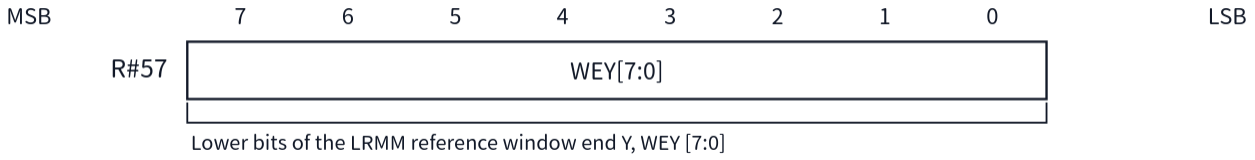



Figure A.54 Bit layout of R#57 Window End Y (L)

### R#58 Window End Y (H)

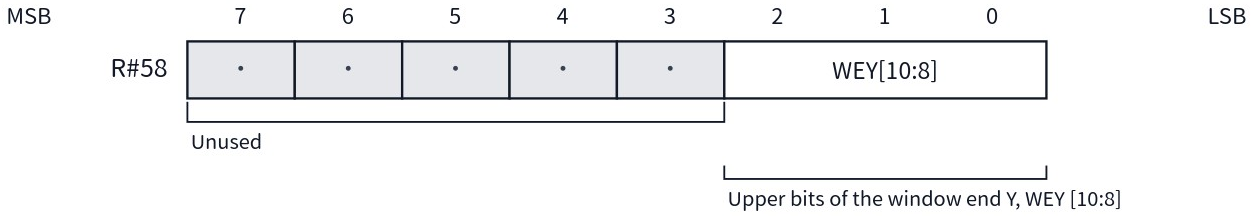



Figure A.55 Bit layout of R#58 Window End Y (H)

## Appendix B Status Register Summary

All are read-only. Specify the number with R#15 and read Port#1 (see Chapters 2 and 4).

| S# | Description | Side Effects on Read |
|---|---|---|
| S#0 | bit7: F (frame interrupt) / bit6: 5S (display limit exceeded) / bit5: C (sprite collision) / bit4–0: 5TH (plane number that exceeded the limit) | F / C / 5S are cleared |
| S#1 | bit5–1: ID (compatible mode 00010 / extended mode 00011) / bit0: FH (line | FH is cleared |

| S# | Description | Side Effects on Read |
|---|---|---|
| | interrupt). bit7–6 are fixed at 0 | |
| S#2 | bit7: TR / bit6: VR / bit5: HR / bit4: BD / bit3: fixed at 1 / bit2: fixed at 1 / bit1: EO / bit0: CE | None |
| S#3 | Sprite collision X coordinate [7:0] | None |
| S#4 | Collision X[8] (bit0). bit7–1 are fixed at 1 | None |
| S#5 | Sprite collision Y coordinate [7:0] | The collision coordinates (X / Y) are cleared |
| S#6 | Collision Y[9:8] (bit1–0). bit7–2 are fixed at 1 | None |
| S#7 | Pixel value read out by POINT / LMCM | Used for LMCM transfer control |
| S#8 | SRCH boundary X coordinate [7:0] | None |
| S#9 | Boundary X[8] (bit0). bit7–1 are fixed at 1 | BD (S#2 bit4) is cleared |
| S#10 – S#15 | Not implemented | Always returns FFh |

## B.1 Bit Layout Diagrams by Register (S#0–S#9)

The bit layout of each status register is shown in numerical order. The diagrams are read in the same way as in Appendix A. For the side effects on read, see the table in this Appendix B and Chapter 4. S#10–S#15 are not implemented, and always return FFh when read.

**S#0**

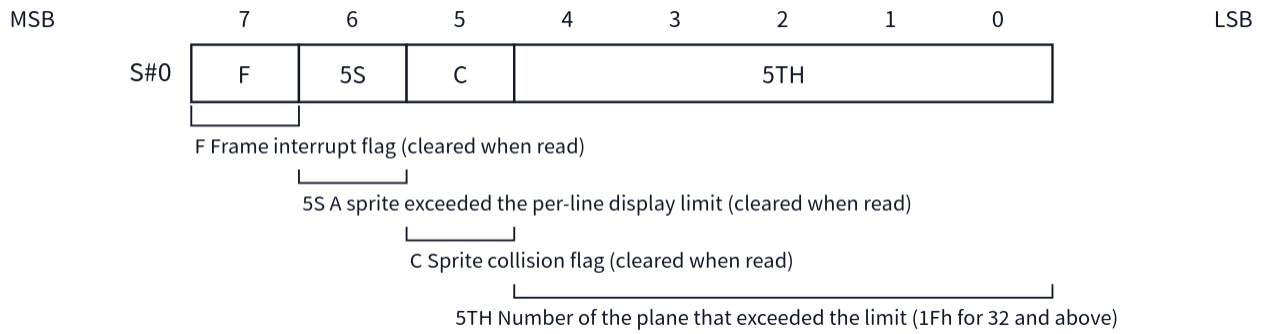



Figure B.1 Bit layout of S#0

**S#1**

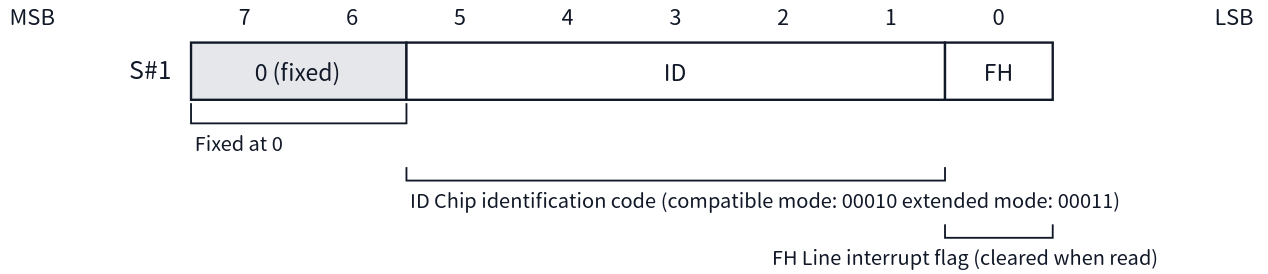



Figure B.2 Bit layout of S#1

**S#2**

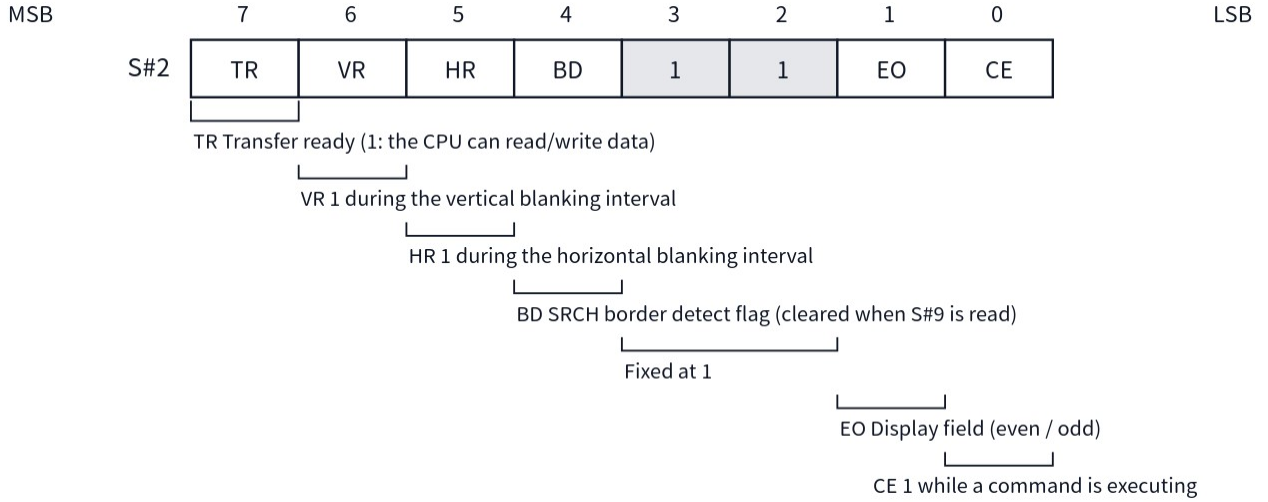



Figure B.3 Bit layout of S#2

### S#3

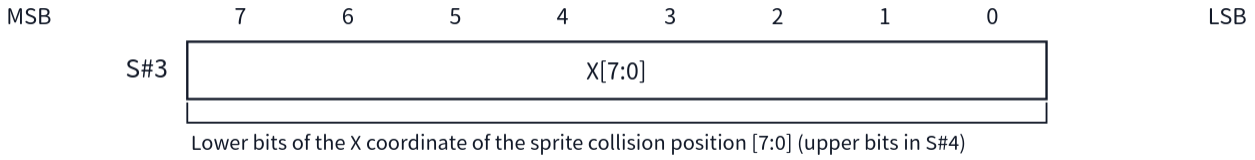



Figure B.4 Bit layout of S#3

### S#4

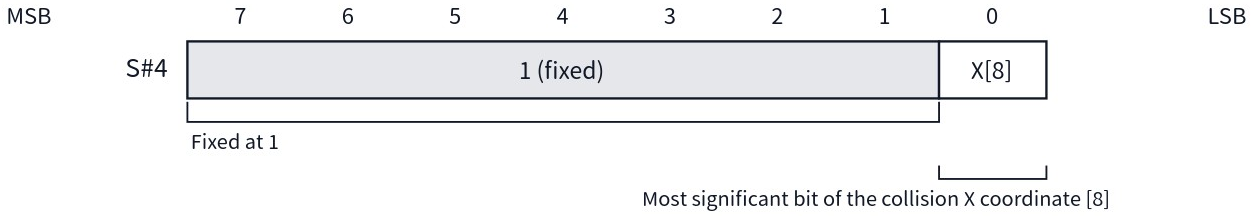



Figure B.5 Bit layout of S#4

### S#5

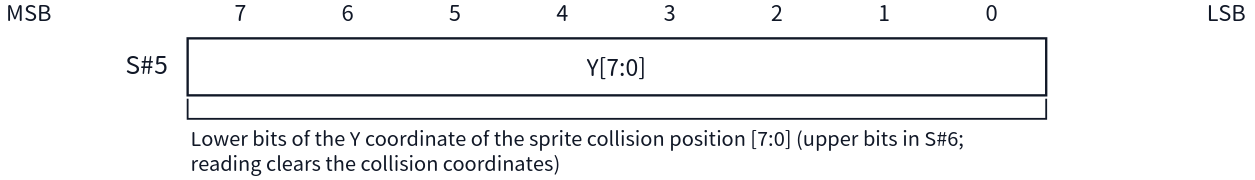



Figure B.6 Bit layout of S#5

### S#6

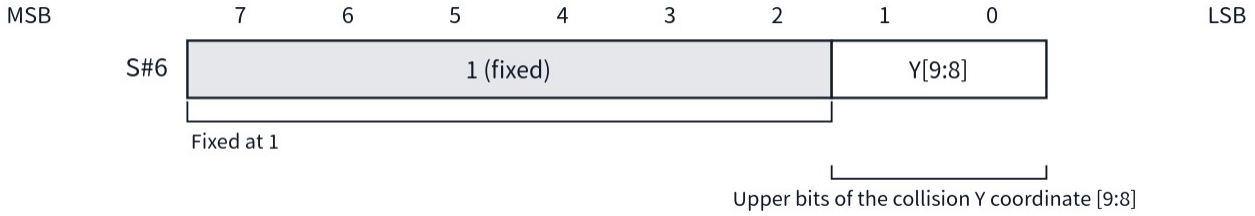



Figure B.7 Bit layout of S#6

**S#7**

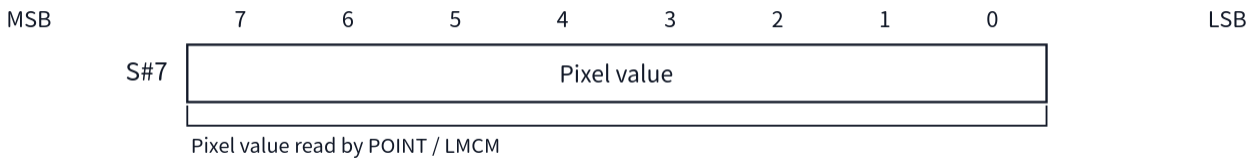



Figure B.8 Bit layout of S#7

**S#8**

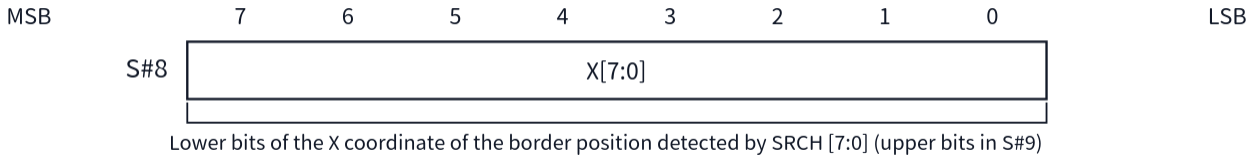



Figure B.9 Bit layout of S#8

**S#9**

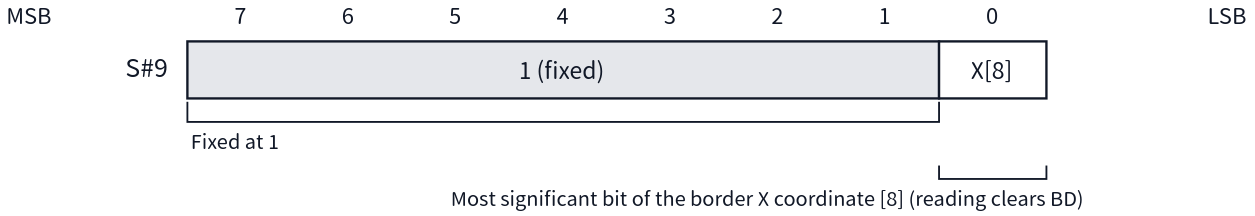



Figure B.10 Bit layout of S#9

# Appendix C Screen Mode Summary

The mode is selected by the combination of M5 / M4 / M3 in R#0 and M1 / M2 in R#1 (see Chapter 5). In Text1 and Multicolor, M3 is a don't-care (the values in the table are representative values; the same mode results even when M3 = 1).

| Mode | Abbreviation | M5 M4 M3 M2 M1 | BASIC SCREEN | Screen | Colors | Sprites |
|---|---|---|---|---|---|---|
| Graphic1 | G1 | 0 0 0 0 0 | SCREEN 1 | 32×24 patterns | 16 palette colors | Mode 1 |

| Mode | Abbreviation | M5 M4 M3 M2 M1 | BASIC SCREEN | Screen | Colors | Sprites |
|---|---|---|---|---|---|---|
| Text1 | T1 | 0 0 x 0 1 | SCREEN 0 (WIDTH 40) | 40×24 characters (6×8 dots) | 2 colors | None |
| Multicolor | MC | 0 0 x 1 0 | SCREEN 3 | 64×48 blocks | 16 palette colors | Mode 1 |
| Graphic2 | G2 | 0 0 1 0 0 | SCREEN 2 | 256×192 dots | 16 palette colors | Mode 1 |
| Graphic3 | G3 | 0 1 0 0 0 | SCREEN 4 | 256×192 dots | 16 palette colors | Mode 2 |
| Graphic4 | G4 | 0 1 1 0 0 | SCREEN 5 | 256×212 dots | 16 palette colors | Mode 2 |
| Text2 | T2 | 0 1 0 0 1 | SCREEN 0 (WIDTH 80) | 80×24 characters (6×8 dots) | 2 colors + blink colors | None |
| Graphic5 | G5 | 1 0 0 0 0 | SCREEN 6 | 512×212 dots | 4 palette colors | Mode 2 |
| Graphic6 | G6 | 1 0 1 0 0 | SCREEN 7 | 512×212 dots | 16 palette colors | Mode 2 |
| Graphic7 | G7 | 1 1 1 0 0 | SCREEN 8 | 256×212 dots | 256 colors (direct specification) | Mode 2 |
| Graphic7-YAE | G7-YAE | 1 1 1 0 0 (+ YAE in R#25) | SCREEN 10 / 11 | 256×212 dots | YJK + 16 palette colors | Mode 2 |
| Graphic7-YJK | G7-YJK | 1 1 1 0 0 (+ YJK in R#25) | SCREEN 12 | 256×212 dots | YJK (approx. 19,268 colors) | Mode 2 |

The Sprites column shows the correspondence in the conventional modes, except when Sprite Mode 3 is selected (SP3 in R#20; see Chapter 6).

Below, for each mode, the standard VRAM layout (VRAM map) and the register settings for that layout are shown. The standard layout here means the layout when MSX-BASIC initializes each mode with a SCREEN statement, and it matches the VRAM maps in the appendix of the MSX2 Technical Handbook. The register values were derived from the base addresses of each table in accordance with the bit layouts in Chapter 3, and have also been confirmed to match the values measured after MSX-BASIC initialization.

The heights of the bands in the diagrams are approximate and do not represent area ratios. All addresses are logical addresses. Note that the palette save area in the Technical Handbook VRAM maps is an area that MSX-BASIC uses to save palette values, and is not a table whose position is determined by a VDP register, so it is not included in the diagrams in this appendix. In extended mode (V58 = 0), the region 20000h–3FFFFh can also be used, but the standard layouts fit within the 128KB range in every mode.

Notes on how to read each table are collected below.

- The values of R#1 include the display enable (BL = 1) and frame interrupt enable (IE0 = 1) set by MSX-BASIC. Only M1 / M2 are required for selecting the mode itself. SI / MAG are both 0 (sprites 8×8 dots, unmagnified).
- The values of R#7 are the MSX-BASIC initial values (the values from the initial settings of the COLOR statement). In text modes, the upper nibble represents the foreground color and the lower nibble the background color. In the other modes, the lower nibble is the border color.
- Unless otherwise noted, the number of display lines in the bitmap modes is for the 212-line case (LN = 1 in R#9).

## C.1 Text1 (SCREEN 0, WIDTH 40)

Sprites and the color table are not used. This is the simplest layout, with only the pattern name table and the pattern generator table.




Figure C.1 Standard VRAM map of Text1

| Register | Standard Value | Description |
|---|---|---|
| R#0 | M5 = 0, M4 = 0, M3 = 0 | Mode selection bits (M3 is a don't-care) |
| R#1 | 70h | BL = 1, IE0 = 1, M1 = 1, M2 = 0 |
| R#2 | 00h | Pattern name table = 00000h |
| R#4 | 01h | Pattern generator table = 00800h |

| Register | Standard Value | Description |
|---|---|---|
| R#7 | F4h | Foreground color 15 / background color 4 |

## C.2 Text2 (SCREEN 0, WIDTH 80)

A blink table is added. The standard number of rows is 24 (LN = 0 in R#9). If LN = 1 (26.5 rows) is set, the pattern name table expands to 2,160 bytes (00000h–0086Fh) and the blink table to 270 bytes (00800h–0090Dh).




Figure C.2 Standard VRAM map of Text2

| Register | Standard Value | Description |
|---|---|---|
| R#0 | M5 = 0, M4 = 1, M3 = 0 | Mode selection bits |
| R#1 | 70h | BL = 1, IE0 = 1, M1 = 1, M2 = 0 |
| R#2 | 03h | Pattern name table = 00000h (A17–A12 affect the base; the lower 2 bits are set to 1) |
| R#3 | 27h | Blink table = 00800h (combined with R#10, specifies A16–A9. The lower bits are set to 1) |
| R#10 | 00h | Upper bits of the blink table base address |
| R#4 | 02h | Pattern generator table = 01000h |
| R#7 | F4h | Foreground color 15 / background color 4 (the blink color is in R#12, the period in R#13) |
| R#9 | LN = 0 | 24 rows (26.5 rows when LN = 1) |

## C.3 Multicolor (SCREEN 3)

The pattern generator table also serves as the color data. There is no color table. Sprites are in Mode 1.




Figure C.3 Standard VRAM map of Multicolor

| Register | Standard Value | Description |
|---|---|---|
| R#0 | M5 = 0, M4 = 0, M3 = 0 | Mode selection bits (M3 is a don't-care) |
| R#1 | 68h | BL = 1, IE0 = 1, M1 = 0, M2 = 1, SI = 0, MAG = 0 |
| R#2 | 02h | Pattern name table = 00800h |
| R#4 | 00h | Pattern generator table (color data) = 00000h |

| Register | Standard Value | Description |
|---|---|---|
| R#5 | 36h | Sprite attribute table = 01B00h |
| R#11 | 00h | Upper bits of the sprite attribute table base address |
| R#6 | 07h | Sprite pattern generator table = 03800h |
| R#7 | 07h | Border color 7 |

## C.4 Graphic1 (SCREEN 1)

Sprites are in Mode 1. The color table is small, at 32 bytes, and there is a large unused region immediately after the pattern name table.




Figure C.4 Standard VRAM map of Graphic1

| Register | Standard Value | Description |
|---|---|---|
| R#0 | M5 = 0, M4 = 0, M3 = 0 | Mode selection bits |
| R#1 | 60h | BL = 1, IE0 = 1, M1 = 0, M2 = 0, SI = 0, MAG = 0 |
| R#2 | 06h | Pattern name table = 01800h |
| R#3 | 80h | Color table = 02000h |

| Register | Standard Value | Description |
|---|---|---|
| R#10 | 00h | Upper bits of the color table base address |
| R#4 | 00h | Pattern generator table = 00000h |
| R#5 | 36h | Sprite attribute table = 01B00h |
| R#11 | 00h | Upper bits of the sprite attribute table base address |
| R#6 | 07h | Sprite pattern generator table = 03800h |
| R#7 | 07h | Border color 7 |

## C.5 Graphic2 (SCREEN 2)

The pattern generator table and the color table have a three-bank configuration, each using 6,144 bytes.

Sprites are in Mode 1.




Figure C.5 Standard VRAM map of Graphic2

| Register | Standard Value | Description |
|---|---|---|
| R#0 | M5 = 0, M4 = 0, M3 = 1 | Mode selection bits |
| R#1 | 60h | BL = 1, IE0 = 1, M1 = 0, M2 = 0, SI = 0, MAG = 0 |
| R#2 | 06h | Pattern name table = 01800h |

| Register | Standard Value | Description |
|---|---|---|
| R#3 | FFh | Color table = 02000h (C13 affects the base; the lower bits are set to 1 as a mask. See Section 5.7) |
| R#10 | 00h | Upper bits of the color table base address |
| R#4 | 03h | Pattern generator table = 00000h (F13 affects the base; the lower 2 bits are set to 1 as a mask. See Section 5.7) |
| R#5 | 36h | Sprite attribute table = 01B00h |
| R#11 | 00h | Upper bits of the sprite attribute table base address |
| R#6 | 07h | Sprite pattern generator table = 03800h |
| R#7 | 07h | Border color 7 |

## C.6 Graphic3 (SCREEN 4)

The VRAM layout is almost the same as Graphic2; the only differences are that the sprites are in Mode 2 and, accordingly, the positions of the sprite attribute table and the sprite color table change (Section 5.8).




Figure C.6 Standard VRAM map of Graphic3

| Register | Standard Value | Description |
|---|---|---|
| R#0 | M5 = 0, M4 = 1, M3 = 0 | Mode selection bits |
| R#1 | 60h | BL = 1, IE0 = 1, M1 = 0, M2 = 0, SI = 0, MAG = 0 |
| R#2 | 06h | Pattern name table = 01800h |

| Register | Standard Value | Description |
|---|---|---|
| R#3 | FFh | Color table = 02000h (same as Graphic2) |
| R#10 | 00h | Upper bits of the color table base address |
| R#4 | 03h | Pattern generator table = 00000h (same as Graphic2) |
| R#5 | 3Fh | Sprite attribute table = 01E00h (the sprite color table is automatically placed at 01C00h. See Section 6.4.2) |
| R#11 | 00h | Upper bits of the sprite attribute table base address |
| R#6 | 07h | Sprite pattern generator table = 03800h |
| R#7 | 07h | Border color 7 |

## C.7 Graphic4 / Graphic5 (SCREEN 5 / 6)

This is a bitmap mode. VRAM is divided into pages of 32KB each, and the sprite-related tables are placed at the rear of page 0. The sprite-related tables are common even when the display page is switched, and only the bitmap is placed in page 1 and beyond. In the 192-line case (LN = 0), the bitmap is 24,576 bytes (00000h–05FFFh within the page). The layout and register values of Graphic4 and Graphic5 are identical except for the mode selection bits.

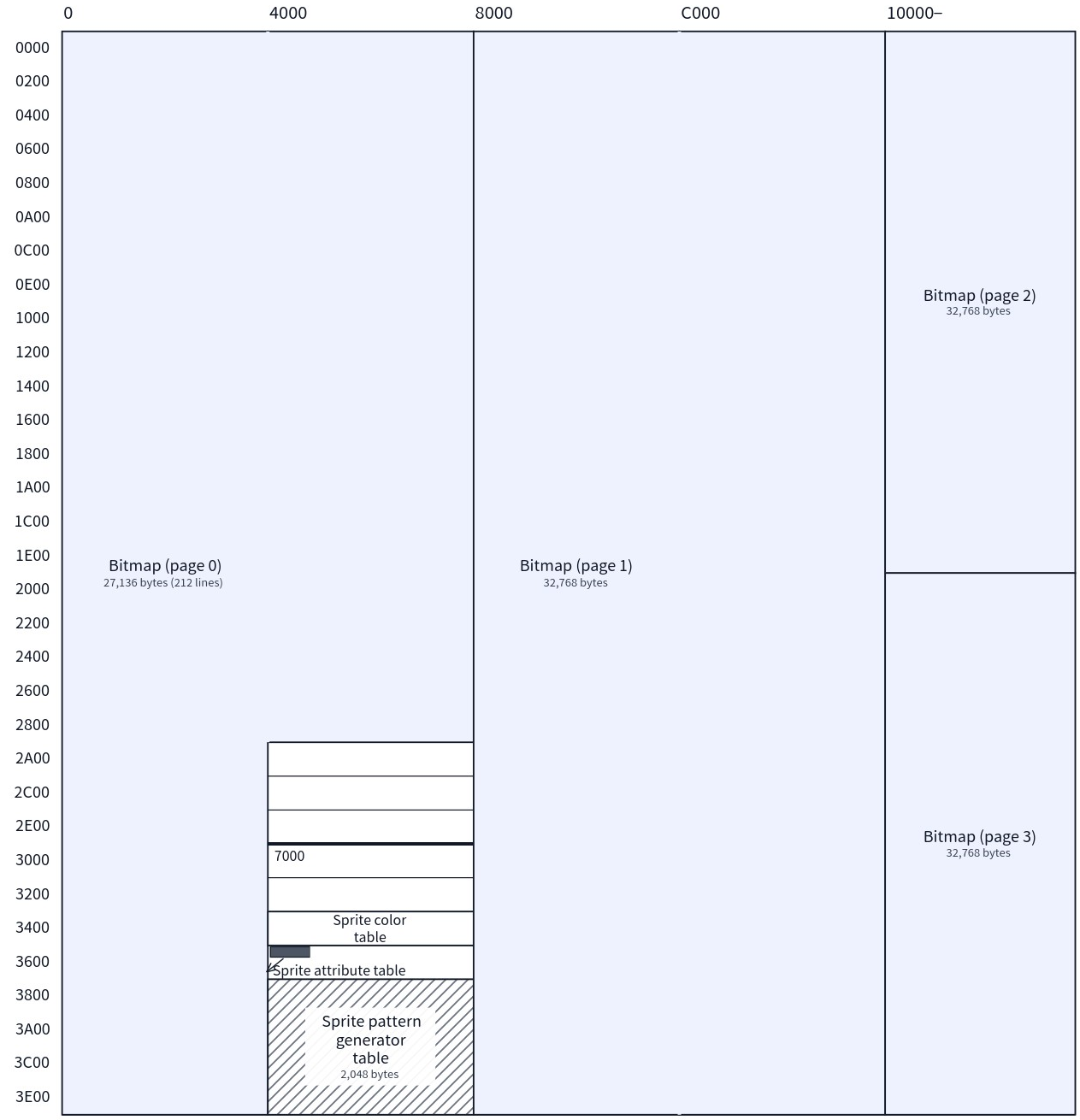



Figure C.7 Standard VRAM map of Graphic4 / Graphic5

| Register | Standard Value | Description |
|---|---|---|
| R#0 | Graphic4: M5 = 0, M4 = 1, M3 = 1 / Graphic5: M5 = 1, M4 = 0, M3 = 0 | Mode selection bits |
| R#1 | 60h | BL = 1, IE0 = 1, M1 = 0, M2 = 0, SI = 0, MAG = 0 (Sprite Mode 2) |
| R#2 | 1Fh | Bitmap page 0 = 00000h (the page is selected by A17–A15; the |

| Register | Standard Value | Description |
|---|---|---|
| | | lower 5 bits are set to 1) |
| R#5 | EFh | Sprite attribute table = 07600h (the sprite color table is automatically placed at 07400h. See Section 6.4.2) |
| R#11 | 00h | Upper bits of the sprite attribute table base address |
| R#6 | 0Fh | Sprite pattern generator table = 07800h |
| R#7 | 07h | Border color 7 |
| R#9 | LN = 1 | 212 lines (192 lines when LN = 0) |

## C.8 Graphic6 / Graphic7 / Graphic7-YAE / Graphic7-YJK (SCREEN 7 / 8 / 10 / 11 / 12)

This is a bitmap mode. Pages are in units of 64KB, and the sprite-related tables are placed at the rear of page 0. The layout and register values are common to the four modes; the only differences are the mode selection bits and the YAE / YJK selection (R#25). In the 192-line case (LN = 0), the bitmap is 49,152 bytes (00000h–0BFFFh within the page).

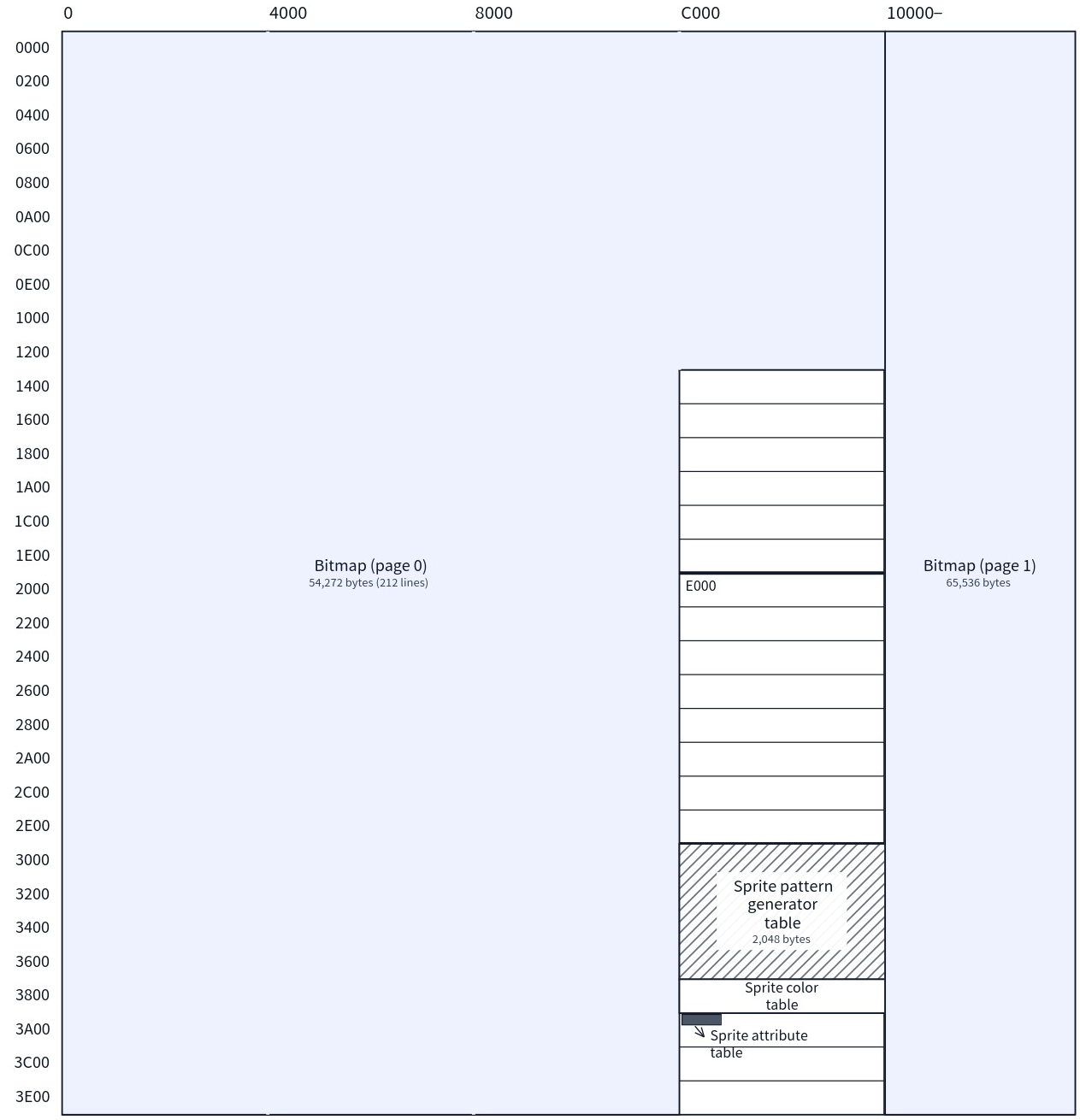



Figure C.8 Standard VRAM map of Graphic6 / Graphic7 (SCREEN 10–12 use the same layout)

| Register | Standard Value | Description |
|---|---|---|
| R#0 | Graphic6: M5 = 1, M4 = 0, M3 = 1 / Graphic7 family: M5 = 1, M4 = 1, M3 = 1 | Mode selection bits |
| R#1 | 60h | BL = 1, IE0 = 1, M1 = 0, M2 = 0, SI = 0, MAG = 0 (Sprite Mode 2) |
| R#2 | 1Fh | Bitmap page 0 = 00000h (the page is selected by A17 / |

| Register | Standard Value | Description |
|---|---|---|
| | | A16; the lower 5 bits are set to 1) |
| R#5 | F7h | Sprite attribute table = 0FA00h (the sprite color table is automatically placed at 0F800h. See Section 6.4.2) |
| R#11 | 03h | Upper bits of the sprite attribute table base address |
| R#6 | 1Eh | Sprite pattern generator table = 0F000h |
| R#7 | 07h | Border color 7 (in Graphic7, the interpretation of colors differs from the other modes. See Section 5.12) |
| R#9 | LN = 1 | 212 lines (192 lines when LN = 0) |
| R#25 | SCREEN 8: YAE = 0, YJK = 0 / SCREEN 10 / 11: YAE = 1, YJK = 1 / SCREEN 12: YAE = 0, YJK = 1 | Selection of the color representation format (Graphic7 variants only) |

Note that, because this group consists of modes of SCREEN 7 and above, the physical VRAM layout is interleaved (even addresses in DRAM0, odd addresses in DRAM1). Figure C.8 shows the logical address layout. Also, when Sprite Mode 3 (SP3 in R#20) is used, the physical layout returns to flat. For details of the physical layout, and for a note that switching changes the arrangement of the data, see Section 11.1.

# Appendix D VDP Command Code Summary

Specify the command code in bit7–4 of R#46. bit3–0 is the logical operation (see Chapter 8). The three commands new in V9968 are valid only in extended mode (V58 = 0); in compatible mode they are treated as STOP.

| Code | Command | Category | Code | Command | Category |
|---|---|---|---|---|---|
| 1111 | HMMC | CPU→VRAM high-speed transfer | 0111 | LINE | Line drawing |

| Code | Command | Category | Code | Command | Category |
|---|---|---|---|---|---|
| 1110 | YMMM | VRAM→VRAM high-speed transfer (Y direction) | 0110 | SRCH | Color search |
| 1101 | HMMM | VRAM→VRAM high-speed transfer | 0101 | PSET | Dot drawing |
| 1100 | HMMV | VRAM high-speed fill | 0100 | POINT | Dot read |
| 1011 | LMMC | CPU→VRAM logical transfer | 0011 | LRMM | Logical transfer with rotation and enlargement / reduction |
| 1010 | LMCM | VRAM→CPU logical transfer | 0010 | LFMC | CPU→VRAM font drawing |
| 1001 | LMMM | VRAM→VRAM logical transfer | 0001 | LFMM | VRAM→VRAM font drawing |
| 1000 | LMMV | VRAM logical fill | 0000 | STOP | Stop |

# Appendix E I/O Port Summary

The base address is the value stored at BIOS 0006h / 0007h (normally 98h) (see Chapter 2).

| Port | Write | Read |
|---|---|---|
| Port#0 (base+0) | VRAM data write (after writing, the address is incremented) | VRAM data read |
| Port#1 (base+1) | Two-byte pair protocol (register write / VRAM address setting) | The status register pointed to by R#15 |
| Port#2 | Palette data write | FFh (no function) |

| Port | Write | Read |
|---|---|---|
| (base+2) | | |
| Port#3<br>(base+3) | Indirect register write via the R#17 pointer | FFh (no function) |
| Port#4<br>(base+4) | Interrupt flag clear + LockExtReg setting (clears only the sources for which 1 is written) | Interrupt status [LockExtReg][0][0][0][0][0][CI][LI][FI] |

## Revision History

| Edition | Date | Description |
|---|---|---|
| 1st edition | — | First edition |

# Index

An index of the important terms used in this manual and the pages that mainly describe them. The page numbers are links: selecting one takes you to the corresponding passage.


| Term | Page |
|---|---|
| 5S / 5TH | 71 |
| 9958 Compatible | 140 |
| CEIE | 16, 129 |
| Collision detection | 70 |
| Command completion interrupt | 129 |
| Compatible mode | 139 |
| EPAL | 16, 74 |
| Extended mode | 139 |
| FG4 | 28 |
| FIL | 16 |
| Frame interrupt | 128 |
| G3BASIC | 122 |
| Geo3D | 113 |
| Graphic7 | 50 |
| HMMC | 87 |

| Term | Page |
| --- | --- |
| HMMM | 89 |
| HMMV | 90 |
| HS | 16, 138 |
| ILNS | 16 |
| Immediate mode | 114 |
| Interrupts | 127 |
| LFMC | 101 |
| LFMM | 102 |
| LINE | 95 |
| Line interrupt | 128 |
| LMCM | 92 |
| LMMC | 91 |
| LMMM | 93 |
| LMMV | 94 |
| LockExtReg | 11 |
| LRMM | 99 |
| MXC | 28 |
| MXD | 28 |

| Term | Page |
|---|---|
| MXS | 28 |
| Palette | 73 |
| Palette RAM | 73 |
| Palette set | 62 |
| Phase 2 engine | 116 |
| POINT | 98 |
| Port#4 | 10 |
| PSET | 97 |
| R#20 | 16 |
| R#21 | 17 |
| R#25 | 18 |
| R#45 | 28 |
| R#46 | 30 |
| Ring buffer | 121 |
| Rotation vector (R#47–R#50) | 31 |
| S16 | 16, 72 |
| Scaling | 68 |
| SP3 | 16, 57 |

| Term | Page |
|---|---|
| Sprite mode 1 | 59 |
| Sprite mode 2 | 60 |
| Sprite mode 3 | 64 |
| SPS | 18, 72 |
| SRCH | 96 |
| Status registers | 34 |
| STOP | 104 |
| SVNS | 16 |
| Texture | 120 |
| V58 | 17 |
| VRAM | 135 |
| Window registers (R#51–R#58) | 32 |
| XHR | 28 |
| YAE | 53, 77 |
| YJK | 54, 77 |
| YMMM | 88 |

# V9968 MSX-VIDEO TECHNICAL DATA BOOK

October 31, 2026

Written by  Takayuki Hara (HRA!)